

Guia de estudo para a arguição

Projeto **Diários Oficiais** — coleta, leitura e análise de diários oficiais municipais (Java puro)

O que é este PDF. Explica cada parte do código: objetivo, como funciona passo a passo, por que foi feito assim, que conceito de Java aparece, o que o professor pode perguntar e onde o código é fraco. Acompanha o PDF 1 (*código-fonte completo*), com os mesmos números de linha.

Arguição: quarta-feira, 07/10/2026. **Este material:** gerado em 05/10/2026 a partir do código que está no repositório, **compilado e executado** para conferir os números citados (85 testes passando; saídas reais dos diários de Belo Horizonte, Recife e Aracaju).

Sumário

Sumário	2
Como usar este material	4
Resumo em uma página	6
1. Visão geral do projeto	7
1.1 O problema e a solução	7
1.2 O pipeline	7
1.3 A vida de um diário: Belo Horizonte, passo a passo	8
1.4 Arquitetura: um cérebro, duas portas	9
1.5 Mapa dos pacotes	9
1.6 As decisões de projeto (o "porquê")	10
1.7 Como o programa é iniciado	10
2. Pacote modelo e pacote erros	11
Fato.java — "achei isto, exatamente aqui"	11
Ato.java — a unidade de trabalho	12
Diario.java — o arquivo de um dia inteiro	13
Contrato.java — uma contratação	14
Relacao.java — uma aresta do grafo	15
Achado.java — uma frase da interpretação	15
DiariollegivelException.java e ColetaException.java — exceções próprias	15
3. Pacote extratores	17
Extrator.java — a classe abstrata	17
ExtratorCnpj.java — o mais simples	19
ExtratorData.java — datas numéricas e por extenso	19
ExtratorValor.java — valores em reais	20
ExtratorProcesso.java — número do processo	21
ExtratorReferencia.java — citações de normas	22
Como tudo se conecta: Extracao.java	23
4. Pacote etapas e Aplicacao.processarPasta()	25
Aplicacao.processarPasta() — o esqueleto	25
Ingestao.java — etapa 1: ler a pasta	26
Limpeza.java — etapa 2: limpar e, se preciso, recusar	27
Fragmentacao.java — etapa 3: cortar em atos	28
Portao.java — etapa 5: dar a nota	30
Privacidade.java — mascarar CPF	32
Relatorio.java — gravar arquivos	33
Visualizador.java — navegador de arquivos do terminal	33
PaginaHtml.java — o HTML do painel	33
5. Pacote analise	34
Assunto.java — de que trata o ato?	34
PainelContratos.java — quem foi contratado, por quanto	35
GrafoNormas.java — quem altera, revoga, prorroga ou só cita	37
LeituraDoAto.java — o que este ato faz, em uma frase	40
Interpretacao.java — "o que esses dados dizem"	41
Cobertura.java — o que foi lido e o que não foi	42
6. Pacote coleta	43
Coletor.java — falar com a API e baixar os textos	43
Json.java — leitor de JSON feito à mão	46
Municipio.java e Fonte.java — o catálogo	49
EstadoFonte.java — a situação de uma fonte	49
Fontes.java — a lista de fontes que lembra	50
ResultadoColeta.java e Rodada.java — o histórico	50
7. Servidor, Aplicacao, Principal e Testes	52
Aplicacao.java (continuação) — coleta, threads e estado	52

Servidor.java — o HttpServer do JDK	55
Formulario.java — ler parâmetros de URL e de formulário	56
PainelWeb.java — as páginas "Painel" e "Coleta e fontes"	57
PaginaDiario.java — diário, ato e busca	57
Principal.java — o menu de terminal	58
Testes.java — como sei que funciona	59
Arquivos de configuração e dados	60
8. Mapa de conceitos de Java	61
Orientação a objetos	61
Exceções	61
Texto, listas e arquivos	62
Algoritmos	62
Fora do conteúdo de aula (saiba a resposta curta)	62
9. Pontos de atenção	64
1. Espécies com acento nunca são reconhecidas (defeito)	64
2. Os "129 títulos sem corpo" de BH: a maioria tem corpo	65
3. Um valor e vários CNPJs: valor contado de novo	66
4. "R\$ 7,02 milhões" vira R\$ 7,02	66
5. O ramo REJEITADO nunca acontece	66
6. Normas: ano descartado e autocitação	66
7. Nomes de empresa	66
8. Dados de página e CNPJ mascarado	66
9. Imperfeições pequenas (para não se surpreender)	67
10. Documentação × código	67
10. Banco de perguntas e exercícios	68
10.1 Visão geral	68
10.2 Orientação a objetos	69
10.3 Exceções e arquivos	69
10.4 Texto e extração	70
10.5 Fragmentação, portão e análise	71
10.6 Fora da aula	72
10.7 Perguntas de "mostre no código"	72
10.8 Exercícios: "altere o código aqui na minha frente"	73
10.9 Autoteste de 10 minutos (em voz alta)	74
11. Plano de estudo e roteiro de apresentação	75
Plano em três blocos	75
Como depurar no IntelliJ (o melhor jeito de aprender)	75
Roteiro de demonstração (3 minutos)	76
Quem defende o quê (sugestão para 4 pessoas)	76
Glossário	77

Como usar este material

Leia esta página primeiro

O professor pode abrir **qualquer arquivo** e perguntar. Este material foi montado para que você consiga responder sobre qualquer parte seguindo sempre a mesma lógica: **o que a parte faz → por que foi feita assim → onde está no código → o que ela não faz bem**.

Os dois PDFs

PDF	para que serve
1 • Código-fonte completo	Todos os 50 arquivos do repositório (44 de código Java, pom.xml, ABRIR_PAINEL.bat, LEIA-ME.txt e os 3 documentos originais, exceto os diários de exemplo em dados/) na íntegra, com os números de linha reais (iguais aos do IntelliJ), índice clicável e favoritos. Use para "achar a linha".
2 • Este guia	Explica cada parte, na ordem em que o código deve ser estudado. As referências Arquivo.java:NN apontam para linhas do PDF 1.

Quanto tempo você tem?

tempo	o que ler
15 minutos	"Resumo em uma página" (próximo capítulo) e o capítulo "Pontos de atenção".
1 hora	o acima + capítulo 1 (visão geral) + Aplicacao.processarPasta() + Fato, Ato e Extrator.
até quarta	o Plano de estudo (último capítulo): dois dias, com ordem e tempo de cada bloco.

Convenções

- `Limpeza.java:17` é arquivo e linha. Abra no PDF 1 (cada cartão de arquivo aqui diz a página).
- Os blocos de código deste guia são **cópias exatas** do arquivo, com os números de linha originais.
- Caixas coloridas: **PERGUNTA PROVÁVEL**, **ATENÇÃO**, **PONTO FRACO** (limitação real do código) e **DICA**.
- **P / R** é uma pergunta e uma resposta curta, para dizer em voz alta.

Como conferi o que está escrito aqui

Compilei o projeto (JDK 21), executei os testes (**85 passam, 0 falham**) e rodei o pipeline inteiro sobre os 4 diários de `dados/diarios`. As saídas reais aparecem como exemplos. Onde digo "verifiquei rodando", foi exatamente isso. Os números do README para Belo Horizonte **batem** (96 atos; 66 aceitos, 30 duvidosos, 0 rejeitados; 19 contratações somando R\$ 8.323.312,07; 6 alterações de norma).

ATENÇÃO — Três divergências entre a documentação e o código

- O README diz **75 testes**; o código tem e executa **85** (`LEIA-ME.txt` e `PREPARO_ARGUICA0.md` já dizem 85).
- O README diz "1 segundo entre requisições"; o código usa `PAUSA_MS = 1500`, ou seja, **1,5 s** (`Coletor.java:24`).
- O README fala em dois diários no corpus, mas a pasta tem **4** (Aracaju, Belo Horizonte e duas edições de Recife). Os números "do painel" (129 atos, 25 contratações, R\$ 9.373.590,71) são dos 4 diários; os números do README são só de BH. Diga sempre de qual conjunto está falando.

PONTO FRACO — SEJA HONESTO — Leia o capítulo "Pontos de atenção" antes de quarta

Ao ler e executar o código encontrei **um defeito real** (títulos com acento, como "RESOLUÇÃO", não são reconhecidos pela fragmentação), **uma suposição do README que não se sustenta** (a maioria dos "títulos sem corpo" descartados em BH são atos reais) e outras limitações. Nada disso foi alterado no código; estão descritos com evidência e com a resposta honesta pronta, porque o professor pode encontrar a mesma coisa.

Resumo em uma página

Se só der para ler uma coisa

O que o programa é. Java puro (JDK 17+, nenhuma biblioteca externa). Coleta diários oficiais de municípios (arquivos de texto enormes), **corta cada diário em atos** (decretos, portarias, extratos...), **acha dentro de cada ato dados concretos** (datas, valores em R\$, CNPJ, número de processo, normas citadas) e produz informação que não está escrita em documento nenhum: quanto cada município contratou e com quem, quais normas foram alteradas ou revogadas, o que cada ato faz em uma frase e **o que o programa não conseguiu ler**. Tudo é determinístico: mesma pasta, mesmos números, nenhuma IA.

O pipeline — o coração está em `Aplicacao.java:69-111`:

Ingestão → Limpeza (pode RECUSAR) → Fragmentação → Extração → Portão → Análise

As cinco ideias que explicam o projeto

- O texto bruto nunca é alterado.** `Diario` guarda `textoBruto` e `textoLimpo` lado a lado.
- Recusar é melhor que fingir que leu.** PDF escaneado vira `DiarioIlegivelException` (com motivo e evidência), não "0 atos".
- Todo fato guarda a posição** (`inicio`, `fim`) no texto. `Fato.provar()` recorta o trecho original: é a prova de que nada foi inventado.
- O portão tem três saídas** (ACEITO, DUVIDOSO, REJEITADO) e sempre grava o motivo. O "duvidoso" fica contável.
- Na dúvida, o lado conservador.** Fragmentação não corta sem evidência de corpo; o grafo classifica como CITA (a relação mais fraca) quando não tem certeza.

Conceitos de aula e onde aparecem: herança, classe abstrata e polimorfismo → `Extrator` e as 5 filhas, e o laço em `Extracao` · encapsulamento e `equals/hashCode` → `Fato`, `Ato`, `Relacao` · exceção própria → `DiarioIlegivelException`, `ColetaException` · arquivos e `try-with-resources` → `Ingestao`, `Relatorio` · `ArrayList`, `String`, `StringBuilder` → em todo lugar (não há `HashMap`, `enum`, `interface` nem `regex` nos extratores) · `Scanner` e `switch` → `Principal.menu()`.

Fora da aula (tenha a resposta honesta pronta): `HttpClient` e `HttpServer` (vêm no JDK), `Thread`, `synchronized`, `volatile`, `lambda () ->`, `this::método`, `Locale.ROOT`, `java.time`, e no `Coletor` `Optional.map(this::...)` e `replaceAll` com `regex`.

Números de Belo Horizonte para decorar

etapa	número
limpeza	16.338 linhas, 5.478 removidas como lixo
fragmentação	225 títulos candidatos, 129 descartados, 96 atos
extração	968 fatos (524 datas, 140 valores, 78 CNPJ, 73 processos, 153 normas)
portão	66 ACEITO · 30 DUVIDOSO · 0 REJEITADO
análise	19 contratações = R\$ 8.323.312,07 · 6 alterações atingindo 4 portarias
testes	85, todos passando

Fraquezas para conhecer (capítulo "Pontos de atenção"): acentos em maiúsculas na fragmentação · 107 dos 129 "descartados" de BH são atos reais de linhas curtas · valor com 2 CNPJs é contado duas vezes · "R\$ 7,02 milhões" é lido como R\$ 7,02 · REJEITADO nunca é alcançado.

1. Visão geral do projeto

O problema, o pipeline, a arquitetura e as decisões

1.1 O problema e a solução

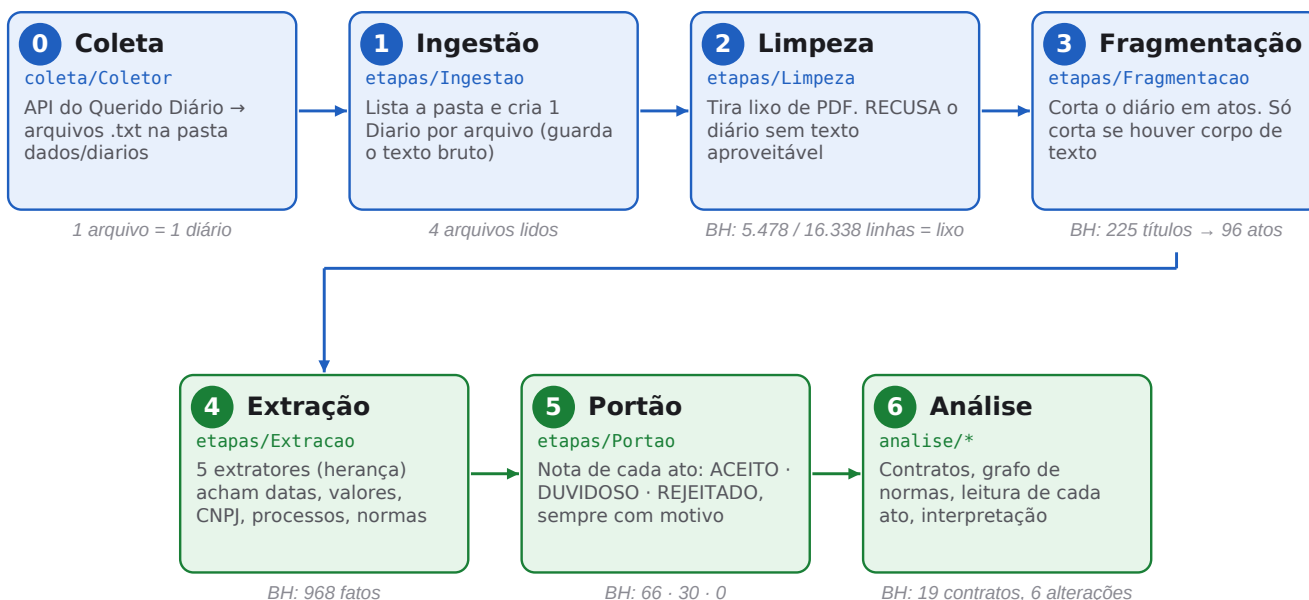
Um **diário oficial** municipal é um documento diário, em PDF, com dezenas ou milhares de páginas: decretos, portarias, nomeações, extratos de contrato, avisos de licitação. O projeto Querido Diário (Open Knowledge Brasil) converte esses PDFs em **arquivos de texto** e os expõe por uma API pública. O texto que chega é bagunçado: colunas quebradas em linhas curtas, cabeçalhos repetidos em toda página, uma marca d'água vertical ("Documento assinado digitalmente") que sai picotada, uma ou duas letras por linha.

O programa responde a quatro perguntas que **nenhum documento individual responde**:

1. Quanto cada município contratou, com quem, e quais fornecedores se repetem?
2. Quais normas (leis, decretos, portarias) foram alteradas, revogadas ou prorrogadas por atos novos?
3. O que cada ato faz, em uma frase em português?
4. O que o programa **não** conseguiu ler, e por quê?

Tudo é feito com a biblioteca padrão do Java (JDK 17+). O painel é uma página HTML servida pelo próprio programa (`com.sun.net.httpserver`), sem JavaScript: cada botão é um formulário comum.

1.2 O pipeline



Tudo é orquestrado por `Aplicacao.processarPasta()` — etapas 1 a 6 em ~30 linhas. A coleta (0) roda antes, em outra thread.

Cada etapa é uma classe pequena de `etapas/`, que recebe um `Diario` e o enriquece. A etapa **0 (coleta)** é separada: ela só baixa arquivos `.txt` para `dados/diarios/`. O pipeline propriamente dito lê essa pasta.

etapa	classe	entrada → saída
1 Ingestão	Ingestao	pasta de <code>.txt</code> → lista de <code>Diario</code> com <code>textoBruto</code>
2 Limpeza	Limpeza	<code>Diario</code> → <code>textoLimpo</code> (ou recusa com <code>DiarioIlegivelException</code>)

etapa	classe	entrada → saída
3 Fragmentação	Fragmentacao	textoLimpo → lista de Ato dentro do Diario
4 Extração	Extracao + 5 Extratores	cada Ato → lista de Fato (com posição)
5 Portão	Portao	cada Ato → decisão + motivo
6 Análise	PainelContratos, GrafoNormas, LeituraDoAto, Interpretacao, Cobertura	atos e fatos → contratos, grafo, frases

1.3 A vida de um diário: Belo Horizonte, passo a passo

Tudo abaixo foi **medido rodando o código** sobre `belo-horizonte_2026-03-25.txt`.

Arquivo. 341 KB, 16.338 linhas. O nome carrega a informação: `belo-horizonte + 2026-03-25`. `Ingestao.municipioDe` pega o que vem antes do primeiro `_`; `Ingestao.dataDe` pega os 10 caracteres depois dele.

Limpeza. Linhas com 3 caracteres ou menos (a marca d'água picotada) e "mobília de página" (cabeçalhos, CHAVE DE ACESSO DA MATÉRIA, endereços `https://`) são descartadas: **5.478 linhas**. Sobram bem mais de 10 linhas longas (> 80 caracteres), então o diário é aceito.

Fragmentação. O programa procura linhas que começam com uma espécie de ato ("DECRETO", "PORTARIA", "EXTRATO"...). Acha **225 títulos candidatos**; só **96** viram atos (os outros 129 foram descartados por não terem "corpo de texto" suficiente; veja a nuance no capítulo "Pontos de atenção").

espécie	atos	assunto (Assunto.de)	atos
EXTRATO	32	OUTRO	37
PORTARIA	18	PESSOAL	21
ATO	11	CONTRATACAO	15
EDITAL	9	LICITACAO	14
COMUNICADO	8	NORMA	5
DECRETO · TERMO	6 + 6	ORCAMENTO	4
LEI · AVISO	3 + 3		

Extração. Os cinco extratores rodam em cada ato e produzem **968 fatos**: 524 datas, 140 valores em R\$, 78 CNPJ, 73 processos, 153 citações de normas.

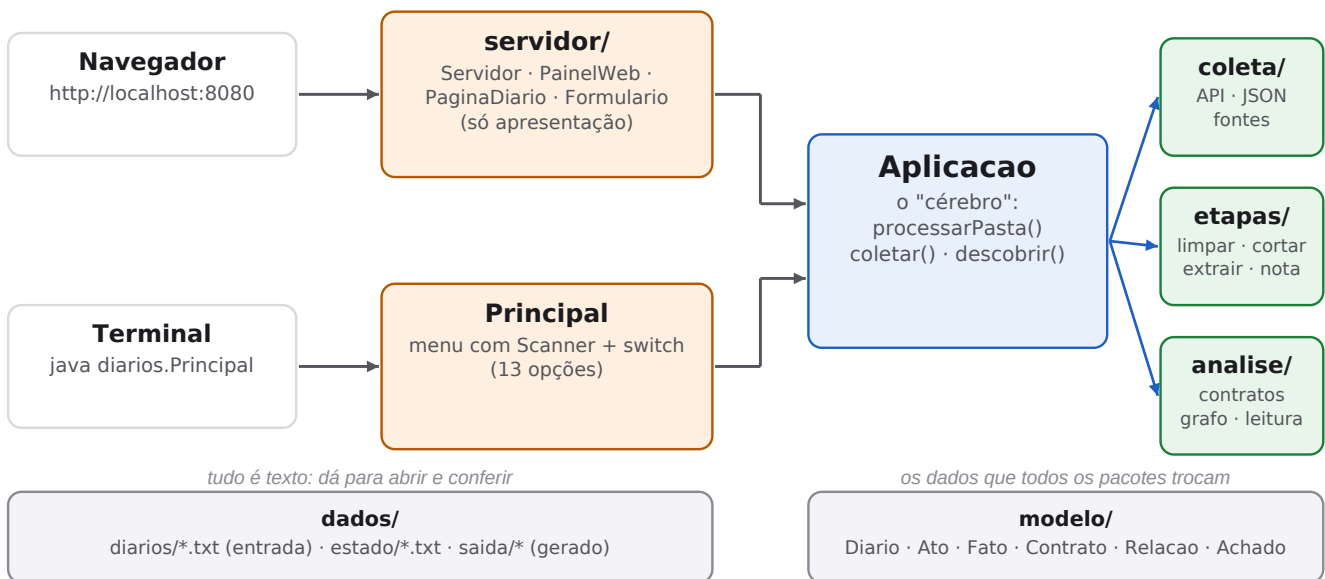
Portão. 66 ACEITO, 30 DUVIDOSO, 0 REJEITADO. Os motivos dos 30 duvidosos: 15 "sem número, sem valor/CNPJ e sem norma citada", 9 "poucos fatos extraídos", 6 "nenhuma data encontrada no ato".

Análise. 19 contratações (R\$ 8.323.312,07), 123 relações no grafo (6 ALTERA, 0 REVOGA, 0 PRORROGA, 117 CITA), e uma frase de leitura para cada um dos 96 atos.

Aracaju é o contraexemplo: 1.715 linhas, mas só **1** linha de corpo. É um PDF escaneado de 269 páginas, o texto extraído tem só cabeçalhos. A limpeza **recusa** o diário e registra: "PDF sem texto extraível (provavelmente escaneado) — 1 linha(s) de corpo em 1715 linhas do arquivo; o mínimo esperado é 10".

1.4 Arquitetura: um cérebro, duas portas

Duas "portas", um só cérebro: os números do painel e do menu são sempre os mesmos.



Aplicacao concentra todo o trabalho. O painel web (**servidor/**) e o menu de terminal (**Principal**) **só chamam métodos dela**, e por isso os números são sempre os mesmos nos dois. Isto responde à pergunta clássica "por que separou assim?": a lógica não depende da forma de mostrar.

A IDEIA CENTRAL — Regra para não se perder no código

Se você está lendo **servidor/** e não entende alguma coisa, **pare**: aquela pasta é só apresentação (montar HTML). O coração do projeto está em **etapas/** e **extratores/**, e o "o que isso significa" está em **analise/**.

1.5 Mapa dos pacotes

pacote	pergunta que responde	classes	conteúdo de aula
modelo/	o que é o dado?	Diario, Ato, Fato, Contrato, Relacao, Achado	classes, encapsulamento, equals/hashCode
erros/	o que pode dar errado ?	DiarioIlegivelException, ColetaException	exceção personalizada
extratores/	o que achar no texto?	Extrator (abstrata) + 5 filhas	herança e polimorfismo
etapas/	como limpar e separar ?	Ingestao, Limpeza, Fragmentacao, Extracao, Portao, Privacidade, Relatorio, Visualizador, PaginaHtml	String, ArrayList, arquivos
analise/	o que isso significa ?	Assunto, PaineContratos, GrafoNormas, LeituraDoAto, Interpretacao, Cobertura	agregação, ordenação, percurso em grafo
coleta/	de onde vem o dado?	Coletor, Json, Municipio, Fonte, EstadoFonte, Fontes, ResultadoColeta, Rodada	String, ArrayList, arquivos, exceções
servidor/	como mostrar ?	Servidor, Formulario, PaineWeb, PaginaDiario	StringBuilder (+ HttpServer, fora da aula)
raiz	orquestração	Aplicacao, Principal	Scanner, switch (+ Thread)
src/test/	como sei que funciona ?	Testes	métodos estáticos, comparação de valores

1.6 As decisões de projeto (o "porquê")

O código não tem comentários; as razões estão no README, e aqui estão ligadas ao ponto exato do código. **Estas são as respostas para as perguntas de "por que você fez assim?"**.

decisão	onde está	o porquê em uma frase
1. O texto bruto nunca é alterado	Diario: Diario.java:10 e Diario.java:13	Se a limpeza tiver defeito, conserta-se o código e roda de novo; alterar o original é a única perda irreversível.
2. Recusar é melhor que fingir que leu	Limpeza.java:40	"0 atos" seria indistinguível de "dia sem publicação".
3. Descartar lixo, nunca remontar	Limpeza.java:28	Remontar a marca d'água colava o título de um ato no parágrafo anterior e fazia perder atos.
4. Só fragmentar com evidência de corpo	Fragmentacao.java:78-92	Perder um ato é ruim, mas inventar atos vazios é pior: o erro não aparece, vira número no relatório.
5. Todo fato guarda a posição	Fato.java:28-32	"Acredite em mim" vs "veja a prova".
6. Portão com três saídas	Portao.java:18-62	Com duas saídas o caso duvidoso é empurrado para um lado e some; com três ele fica contável.
7. Procurar a palavra ≠ classificar	GrafoNormas.java:47-75	"altera" aparece 145 vezes no diário de BH, quase todas em cláusula de contrato, não em norma.
8. Aceitar sigla de órgão na citação	ExtratorReferencia.java:74	Portaria SMC nº 073/2025 não era achada sem isso; só em MAIÚSCULA no original, para "lei que trata do artigo 5" não virar citação.
9. Agrupar por CNPJ, não por nome	PainelContratos.java:279-317	O nome da empresa aparece escrito de várias formas; o CNPJ é estável.
10. Identidade única dentro do diário	Ato.java:36-40 e Fragmentacao.java:56	Duas portarias com a mesma primeira linha geravam lei:13019 duas vezes e a leitura de uma herdava a alteração da outra.
11. Janela de 90 caracteres depois da citação	GrafoNormas.java:16	Com 60 a alteração real se perdia; com 90 aparece.
12. O papel do CNPJ (quem multa, quem é multado)	PainelContratos.java:31	O programa somava multas como contratos; nasceu de um erro real.
13. CPFs mascarados na tela	Privacidade.java:7-28	Diários publicam CPF completo; a máscara mantém o tamanho, então as posições dos fatos continuam valendo.
14. Determinismo, sem IA	todas as análises	Mesma pasta ⇒ mesmos números e frases; dá para conferir e testar.

1.7 Como o programa é iniciado

Há duas formas de entrar, ambas montam uma `Aplicacao`:

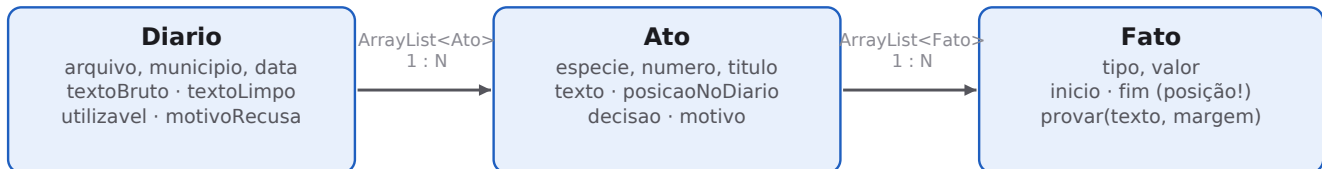
- 1. Painel web** (`Servidor.main`, a classe principal do `pom.xml`): cria `Aplicacao`, chama `processarPasta()` para ler o que já está em disco, sobe o `HttpServer`, abre o navegador e dispara `coletarEmSegundoPlano(1)` (um dia para trás). Opções de linha de comando: `--sem-coleta`, `--sem-navegador`, `--porta=9090`.
- 2. Menu de terminal** (`Principal.main`): cria `Aplicacao`, chama `processarPasta()` e mostra o menu de 13 opções. Com `--tudo` imprime os relatórios e grava os arquivos de saída sem menu.

No Windows, `ABRIR_PAINEL.bat` roda `_sistema\java\bin\java.exe -jar _sistema\diarios.jar` (um Java portátil e o programa já compilado), com `-Dfile.encoding=UTF-8` para acentos corretos.

2. Pacote modelo e pacote erros

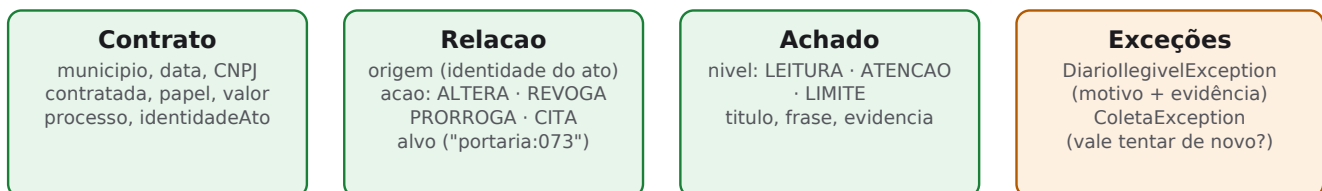
As classes de dados e as duas exceções próprias

Comece por aqui. As classes de **modelo** são simples (atributos, construtor, getters) e **todo o resto do projeto trafega esses objetos**. Se você entende **Fato**, **Ato** e **Diario**, o pipeline fica claro.



Os fatos ficam **DENTRO** do ato; a decisão (**ACEITO/DUVIDOSO/REJEITADO**) também.

Produzidos pela análise a partir de atos e fatos:



azul = o que o pipeline produz e carrega adiante · verde = resultados da análise · laranja = erros próprios

Fato.java — "achei isto, exatamente aqui"

Fato.java

Um dado achado no texto (CNPJ, data, valor...) + a POSIÇÃO exata onde foi achado.

pacote diarios.modelo
55 linhas · PDF 1, **pág. 7**

Objetivo. Representar **um dado achado no texto** (um CNPJ, uma data, um valor...) **junto com o lugar exato onde foi achado**. É a classe mais simples e a mais importante do projeto: a ideia central do trabalho ("todo fato guarda a posição") está nela.

Atributos (todos `private final`: um fato não muda depois de criado):

atributo	exemplo	significado
tipo	"VALOR"	uma das cinco constantes: DATA, VALOR, CNPJ, PROCESSO, REFERENCIA
valor	"R\$ 700.000,00"	o texto exatamente como estava no diário (para REFERENCIA, a forma padronizada "portaria:073")
inicio	60830	índice do primeiro caractere
fim	60843	índice depois do último (mesma convenção do <code>substring(inicio, fim)</code>)

O método que justifica a classe: `provar`.

Fato.java · linhas 28–32

```

28 public String provar(String textoOriginal, int margem) {
29     int de = Math.max(0, inicio - margem);
30     int ate = Math.min(textoOriginal.length(), fim + margem);
31     return textoOriginal.substring(de, ate).replace('\n', ' ').trim();
32 }
  
```

- `Math.max(0, inicio - margem)` e `Math.min(textoOriginal.length(), fim + margem)` impedem sair do texto quando o fato está no começo ou no fim.
- `.replace('\n', ' ')` junta as linhas para o trecho caber numa linha de tela; `.trim()` tira espaços nas pontas.

- Resultado real, em BH: ...Valor total: R\$ 700.000,00 Prazo:120(cento e vinte) dias a partir.... Esse recorte é a **prova** que a página do ato mostra para cada fato.

iguais e hashCode. (Fato.java:35-44) Dois fatos são iguais se tiverem o mesmo tipo e o mesmo valor; a **posição é ignorada**. A ideia é comparar *conteúdo*, não *referência* (Aula 04). O hashCode usa os mesmos dois campos, como manda o contrato.

NOTA — Honestidade sobre o equals

Na versão atual do projeto nenhum código chama Fato.equals nem Ato.equals (verifiquei: os únicos contains sobre objetos do modelo são sobre Relacao). Eles existem por boa prática e por serem conteúdo de aula. Se perguntarem "onde o equals é usado?", a resposta honesta é Relacao.equals, em GrafoNormas.processar (arestas.contains(nova)).

P Por que os atributos são private final?

R Encapsulamento: o fato é criado pelo extrator com todos os valores e **ninguém pode alterá-lo depois** (não há setters). Se alguém precisa de outro valor, cria outro Fato. Isso também protege a "prova": a posição que foi guardada é a que continua valendo.

P Por que guardar inicio e fim em vez de só o valor?

R Sem posição, o programa pede para acreditar nele ("achei a data 24/03/2026"). Com posição ele mostra a prova ("achei 24/03/2026 entre os caracteres 60337 e 60347, olhe o trecho"). Além disso, GrafoNormas e PainelContratos **usam as posições** para olhar o texto em volta do fato (por exemplo, achar a palavra "passa a vigorar" logo depois de uma citação).

Ato.java — a unidade de trabalho

Ato.java

Unidade de trabalho: um decreto/portaria/extrato do diário, com seus fatos e a decisão do portão.

pacote diarios.modelo
99 linhas · PDF 1, **pág. 8**

Objetivo. Um **ato** é um decreto, uma portaria, um extrato de contrato, um aviso: um pedaço do diário que "faz uma coisa". É o objeto que o pipeline passa de etapa em etapa.

O que ele guarda:

- Dados de origem: `municipio`, `dataDiario`, `especie` ("PORTARIA"), `numero` ("016"), `titulo` (a linha de título), `texto` (o bloco de texto do ato) e `posicaoNoDiario` (em que caractere do `textoLimp` o ato começa). Todos `private final`.
- Resultados que as etapas vão preenchendo: a lista `fatos` (Ato.java:19), a `decisao` (começa `DUVIDOSO`) e o `motivo` (começa "ainda nao avaliado").
- Três constantes de texto para a decisão: `ACEITO`, `DUVIDOSO`, `REJEITADO`.

Identidade. Ato.java:36-40

Ato.java · linhas 36–40

```
36 public String getIdentidade() {
37     String num = numero.isEmpty() ? "s-num@" + posicaoNoDiario : numero;
38     return municipio + ":" + especie.toLowerCase() + ":" + num + ":" + dataDiario
39         + desambiguador;
40 }
```

A identidade é montada **a partir do conteúdo**: `municipio:especie:numero:data`. Exemplos reais:

identidade	observação
belo-horizonte:portaria:016:2026-03-25	caso normal
belo-horizonte:extrato:s-num@60058:2026-03-25	título sem número: usa s-num@ + a posição no diário
belo-horizonte:lei:13019:2026-03-25#2	duas portarias com a mesma primeira linha: desambiguar(2) acrescenta #2

identidade	observação
recife:lei:14728:2026-09-29	outro município

Por que não um contador (1, 2, 3...)? Com contador, rodar o programa de novo poderia gerar identificadores diferentes para o mesmo ato (se a ordem mudar), e o grafo e as leituras dependem dessa identidade (`Relacao.origem` e `Contrato.identidadeAto` guardam esse texto). A identidade por conteúdo é **estável e determinística**.

Por que existe `desambiguar`? Foi um bug real (decisão 10 do README): várias portarias de BH começavam com a mesma frase ("Lei Federal nº 13.019/2014 e ao Decreto...") e a fragmentação tirava o número dessa linha. Duas portarias diferentes ficavam com a mesma identidade `lei:13019`, e a leitura de uma herdava a alteração da outra. Hoje, quando `Fragmentacao` vê uma identidade repetida no mesmo diário, chama `novo.desambiguar(repeticoes + 1)`.

Outros métodos: `getFatosDoTipo(tipo)` filtra a lista (usado por todo o código de análise: `ato.getFatosDoTipo(Fato.CNPJ)`); `decidir(decisao, motivo)` grava os dois juntos (**não existe decisão sem motivo**); `equals/hashCode` comparam a identidade.

P	Por que a lista de fatos fica dentro do <code>Ato</code> e não solta?
R	Porque cada fato pertence a um ato; e é o ato quem "decide" (o portão olha os fatos do ato para dar a nota). Agrupar assim também permite que a posição de cada fato seja interpretada dentro do contexto do ato.
P	<code>getFatos()</code> devolve a lista interna. Isso não quebra o encapsulamento?
R	Em parte, sim: quem chama poderia alterar a lista. O projeto aceita esse trade-off pela simplicidade; na prática só <code>Extracao</code> acrescenta fatos, e é por <code>adicionarFato</code> . Uma versão mais rígida devolveria uma cópia.

Diario.java — o arquivo de um dia inteiro

Diario.java O arquivo de um dia inteiro: texto bruto e texto limpo lado a lado, mais os atos.	pacote <code>diarios.modelo</code> 80 linhas · PDF 1, pág. 10
---	---

Objetivo. Representa um arquivo de diário e **tudo que se descobre sobre ele ao longo do pipeline**. É o objeto raiz: contém os atos, e cada ato contém os fatos.

Detalhes que valem lembrar:

- `textoBruto` (`Diario.java:10`) é final: **nunca muda**. `textoLimpo` é preenchido por `registrarLimpeza`. Os dois ficam lado a lado (decisão 1).
- O construtor calcula `linhasOriginais = textoBruto.split("\n").length`.
- Contadores de funil: `linhasRemovidas` (preenchido na limpeza) e `titulosCandidatos/titulosDescartados` (preenchidos na fragmentação). A página do painel mostra o "funil" a partir deles.
- `recusar(motivo)` marca `utilizavel = false`. **Atenção:** a limpeza chama `registrarLimpeza` antes de decidir recusar, então um diário recusado também tem `textoLimpo` preenchido.
- `contar(decisao)` percorre os atos e conta quantos têm aquela decisão (usado por `Cobertura` e pelas páginas).
- Os métodos `registrar...` e `recusar` existem para que cada etapa **informe** o que fez sem precisar mexer nos atributos diretamente.

P Por que guardar o bruto e o limpo, em vez de limpar "no lugar"?

R Se a limpeza ou a fragmentação tiverem defeito, conserta-se o código e roda de novo; se tivéssemos sobrescrito o original, **essa perda nenhum conserto futuro alcança**. Também permite mostrar os dois textos no painel ("como veio" e "depois da limpeza") e é por isso que as posições dos fatos valem sobre o `textoLimpo`.

Contrato.java — uma contratação

Contrato.java

Uma contratação (quem, quanto, processo) e o formatador de reais.

pacote diarios.modelo
59 linhas · PDF 1, **pág. 11**

Objetivo. Guardar **uma contratação identificada**: município, data, CNPJ, nome da empresa (contratada), papel (FORNECEDOR, INDEFINIDO, NOTIFICADO...), valor (double), processo e a identidadeAto do ato de onde saiu. Quem cria é `PainelContratos`.

O método estático `formatarReal(double)` converte `8323312.07` em `"R$ 8.323.312,07"` **sem** usar `NumberFormat`:

Contrato.java · linhas 36–52

```
36 public static String formatarReal(double v) {
37     String s = String.format(java.util.Locale.ROOT, "%.2f", v);
38     int ponto = s.indexOf('.');
39     String inteiro = s.substring(0, ponto);
40     String centavos = s.substring(ponto + 1);
41
42     StringBuilder comPonto = new StringBuilder();
43     int digitos = 0;
44     for (int i = inteiro.length() - 1; i >= 0; i--) {
45         comPonto.insert(0, inteiro.charAt(i));
46         digitos++;
47         if (digitos % 3 == 0 && i > 0) {
48             comPonto.insert(0, '.');
49         }
50     }
51     return "R$ " + comPonto + "," + centavos;
52 }
```

1. `String.format(Locale.ROOT, "%.2f", v)` gera `"8323312.07"` com **ponto decimal sempre**.
2. Separa a parte inteira (`"8323312"`) dos centavos (`"07"`) pelo `indexOf('.')`.
3. Percorre a parte inteira **da direita para a esquerda**, inserindo cada dígito no início de um `StringBuilder` e um `'.'` a cada 3 dígitos (`digitos % 3 == 0 && i > 0`).
4. Monta `"R$ " + inteiroComPontos + "," + centavos`.

Por que `Locale.ROOT`? Foi um erro real que só aparecia no Windows em português: sem `Locale.ROOT`, `String.format("%.2f")` usa a vírgula como separador decimal (`"8323312,07"`); então `indexOf('.')` devolve `-1` e `substring(0, -1)` lança exceção. O teste (`Testes.java:42`) força `Locale.setDefault(pt-BR)` justamente para reproduzir isso.

NOTA — double para dinheiro

Em sistema financeiro de verdade usaria-se `BigDecimal`. Aqui os valores servem para somar poucas dezenas de contratações e exibir; o `double` é suficiente e é o que foi dado em aula. Se perguntarem, reconheça o limite.

Relacao.java — uma aresta do grafo

Relacao.java

Aresta do grafo: ato --(ALTERA | REVOGA | PRORROGA | CITA)--> norma.

pacote diarios.modelo
31 linhas · PDF 1, **pág. 12**

Objetivo. Uma relação **ato** → **norma**: origem (a identidade do ato), **acao** (ALTERA, REVOGA, PRORROGA ou CITA), **alvo** (a norma, em forma padronizada como "portaria:073") e **posicao** (onde está a citação no texto).

Impressa assim: belo-horizonte:portaria:016:2026-03-25 --ALTERA --> portaria:074.

iguais **ignora a posição** (Relacao.java:28-37): duas relações iguais em origem, ação e alvo são a mesma, mesmo citadas em pontos diferentes do texto. Este é o **equals** **que o programa realmente usa**: GrafoNormas.processar só acrescenta a aresta se !arestas.contains(nova), evitando duplicatas.

Achado.java — uma frase da interpretação

Achado.java

Uma frase da interpretação, com nível, título e evidência.

pacote diarios.modelo
31 linhas · PDF 1, **pág. 13**

Objetivo. Uma "descoberta" gerada por Interpretacao: nivel (LEITURA, ATENCAO ou LIMITE), titulo, frase e evidencia (os números que sustentam a frase). toString imprime os quatro. O painel agrupa por nível em "Leitura do período", "Onde vale a pena olhar" e "Limites desta leitura".

DiarioIlegivelException.java e ColetaException.java — exceções próprias

DiarioIlegivelException.java

Exceção própria: diário sem texto aproveitável (carrega motivo e evidência).

pacote diarios.erros
16 linhas · PDF 1, **pág. 14**

DiarioIlegivelException.java · linhas 3–16

```
3 public class DiarioIlegivelException extends Exception {
4
5     private final String arquivo;
6     private final String evidencia;
7
8     public DiarioIlegivelException(String arquivo, String motivo, String evidencia) {
9         super(motivo);
10        this.arquivo = arquivo;
11        this.evidencia = evidencia;
12    }
13
14    public String getArquivo() { return arquivo; }
15    public String getEvidencia() { return evidencia; }
16 }
```

Objetivo. DiarioIlegivelException representa um **resultado possível**, não um bug: o diário veio sem texto aproveitável. Por isso é uma exceção *própria* (conteúdo da Aula 05), que carrega **o motivo** (a mensagem, via super(motivo)) **e a evidência numérica** ("1 linha(s) de corpo em 1715 linhas...").

- **Quem lança:** Limpeza.limpar (Limpeza.java:43).
- **Quem captura:** Aplicacao.processarPasta (Aplicacao.java:82), que registra a recusa em Cobertura e **continua** com o próximo diário (o continue). O programa nunca cai por causa de um diário ilegível.
- É **extends Exception** (exceção *verificada*): quem chama **é obrigado** a tratar ou declarar **throws**. Isso é de propósito: o compilador impede que alguém "esqueça" que um diário pode ser recusado.

ColetaException.java

Exceção própria: falha ao falar com a API (diz se vale tentar de novo).

pacote diarios.erros
23 linhas · PDF 1, **pág. 15**

ColetaException é o equivalente para a coleta: a requisição à API falhou. Além do motivo guarda a url, valeTentarDeNovo e esperaSugeridaMs. Tem **dois construtores**: o curto chama o longo com this(url,

`motivo, false, 0)` (sobrecarga de construtor). `Coletor` usa esses campos para decidir se espera e tenta de novo (429 e 5xx e falhas de rede: sim; outros: não).

P Por que lançar exceção em vez de devolver um número ou `false`?

R "Diário ilegível" não é um erro do programa: é uma saída possível que **precisa ser tratada**. A exceção carrega junto o motivo e a evidência, e obriga quem chama a decidir o que fazer. Um `return false` poderia ser ignorado sem querer, e o diário viraria "0 atos", indistinguível de um dia sem publicação.

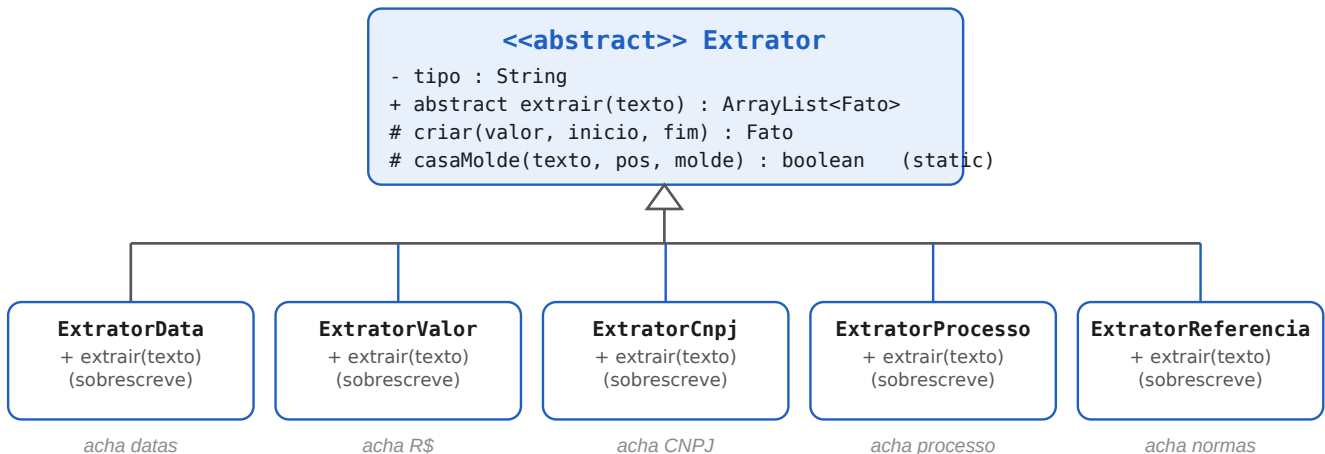
P Qual a diferença entre exceção verificada e não verificada? Qual vocês usaram?

R Verificadas (`extends Exception`) o compilador obriga a tratar ou declarar; não verificadas (`RuntimeException`) não. As duas exceções do projeto são **verificadas**, porque representam situações esperadas que o chamador deve tratar.

3. Pacote extratores

Herança e polimorfismo: como o programa "acha" os dados

Este é o **pacote** que o professor mais provavelmente vai cobrar, porque é onde estão **herança**, **classe abstrata** e **polimorfismo** (Aula 02). Todo extrator recebe um texto e devolve uma lista de Fato. **Nenhum usa expressão regular**: só `charAt`, `indexOf`, `substring` e laços (Aulas 03 e 04).



Extracao guarda um `ArrayList<Extrator>` e chama `e.extrair(...)` sem saber qual filha é — polimorfismo.
Para achar CEPs: uma classe nova + uma linha em `Extracao`. Nada mais muda.

Extrator.java — a classe abstrata

Extrator.java

Classe ABSTRATA: contrato "extrair" + utilitários (`casaMolde`, `semAcento...`).

pacote diarios.extratores
56 linhas · PDF 1, **pág. 16**

Objetivo. Definir o **contrato** que todo extrator cumpre ("receba um texto, devolva `ArrayList<Fato>`") e reunir os **utilitários comuns** (reconhecer dígito, comparar com um molde, tirar acento). É **abstract**: **não se cria um** `Extrator`, só as filhas.

Extrator.java · linhas 6–22

```

6 public abstract class Extrator {
7
8     private final String tipo;
9
10    protected Extrator(String tipo) {
11        this.tipo = tipo;
12    }
13
14    public String getTipo() {
15        return tipo;
16    }
17
18    public abstract ArrayList<Fato> extrair(String texto);
19
20    protected Fato criar(String valor, int inicio, int fim) {
21        return new Fato(tipo, valor, inicio, fim);
22    }
23 }
  
```

- `protected Extrator(String tipo)`: o construtor é `protected` para que só as filhas o chamem (`super(Fato.CNPJ)`). Cada filha informa de qual tipo de fato ela cuida.
- `public abstract ArrayList<Fato> extrair(String texto)`: o **contrato**. Sem corpo, cada filha é obrigada a implementar.
- `protected Fato criar(valor, inicio, fim)`: atalho reaproveitado por todas as filhas: monta um `Fato` já com o tipo certo.

Utilitários:

Extrator.java · linhas 36–48

```

36 protected static boolean casaMolde(String texto, int posicao, String molde) {
37     if (posicao + molde.length() > texto.length()) {
38         return false;
39     }
40     for (int i = 0; i < molde.length(); i++) {
41         char esperado = molde.charAt(i);
42         char achado = texto.charAt(posicao + i);
43         if (esperado == '9' ? !ehDigito(achado) : achado != esperado) {
44             return false;
45         }
46     }
47     return true;
48 }

```

Como `casaMolde` funciona. O "molde" é um texto onde `9` significa "qualquer dígito" e qualquer outro caractere **tem que ser idêntico**. Exemplo com o molde `99.999.999/9999-99` sobre o texto `48.356.311/0001-14`:

molde :	9	9	.	9	9	9	.	9	9	9	/	9	9	9	9	-	9	9
texto :	4	8	.	3	5	6	.	3	1	1	/	0	0	0	1	-	1	4
vale? :	d	d	=	d	d	d	=	d	d	d	=	d	d	d	d	=	d	d

d = é dígito, confere
= = caractere idêntico, confer

e

Se **qualquer** posição falhar, devolve `false` imediatamente. Se chegar ao fim do molde, `true`. A primeira verificação (`posicao + molde.length() > texto.length()`) evita estourar o texto.

- `ehDigito(c)` compara `c >= '0' && c <= '9'` (poderia ser `Character.isDigit`, mas assim fica restrito aos dígitos ASCII).
- `contarDigitos(texto, posicao)` conta quantos dígitos seguidos existem a partir de uma posição (usado para exigir "4 dígitos" do ano).
- `semAcento(String)` é `public static`: troca ç ã á â à é ê í ó ô õ ú pelas letras sem acento. Muitas classes a usam. **Ela só troca letras minúsculas**; por isso, em todo o código se chama `toLowerCase()` antes de `semAcento(...)` (a única exceção é `Fragmentacao`, e é um defeito: veja o capítulo "Pontos de atenção").
- Como `semAcento` e `toLowerCase` trocam **caractere por caractere**, o texto transformado tem o **mesmo tamanho** do original. É isso que permite achar uma posição na versão "sem acento" e usá-la no texto original.

P Onde está a herança no projeto?

R `ExtratorCnpj` extends `Extrator` (e as outras quatro filhas também). Também `DiarioIlegivelException` extends `Exception` e `ColetaException` extends `Exception` são herança, de uma classe do JDK.

P Por que `Extrator` é abstrata?

R Não faz sentido criar um "extrator genérico": ele não sabe o que procurar. A classe abstrata define o **contrato** (`extrair`) e reaproveita código comum (`criar`, `casaMolde`, `semAcento`). O compilador impede `new Extrator(...)` e obriga cada filha a implementar `extrair`.

P O que é o `@Override`?

R Uma anotação que avisa o compilador "este método sobrescreve o da classe mãe". Se a assinatura estiver errada (por exemplo, um parâmetro diferente), o compilador acusa erro em vez de criar, sem querer, um método novo.

P Onde está o polimorfismo?

R Em `Extracao` (`Extracao.java:28`): uma lista `ArrayList<Extrator>` com cinco classes diferentes, e o laço chama `e.extrair(...)` **sem saber qual filha é**. Cada uma responde do seu jeito. (É o mesmo princípio do `FazerSom()` do Cachorro e do Gato, na Aula 02.)

ExtratorCnpj.java — o mais simples

ExtratorCnpj.java

Filha: acha CNPJ pelo molde 99.999.999/9999-99.

pacote diarios.extratores
25 linhas · PDF 1, **pág. 17**

Objetivo. Achar CNPJ no formato com pontuação, `NN.NNN.NNN/NNNN-NN`.

ExtratorCnpj.java · linhas 15–24

```
15 public ArrayList<Fato> extrair(String texto) {
16     ArrayList<Fato> achados = new ArrayList<>();
17     for (int i = 0; i + MOLDE.length() <= texto.length(); i++) {
18         if (casaMolde(texto, i, MOLDE)) {
19             achados.add(criar(texto.substring(i, i + MOLDE.length()), i, i + MOLDE.length()));
20             i += MOLDE.length() - 1;
21         }
22     }
23     return achados;
24 }
```

Passo a passo. O `for` anda por todas as posições `i` do texto (enquanto cabe um molde inteiro: `i + MOLDE.length() <= texto.length()`). Em cada uma pergunta `casaMolde`. Quando casa, cria o `Fato` com o trecho e as posições `[i, i + 18)` e **pula** `MOLDE.length() - 1` posições (o `i++` do laço pula a última), para não achar CNPJ "dentro" do CNPJ que acabou de achar.

NOTA — Limitações (verificadas)

- Só confere o **formato**, não os dígitos verificadores: 11.111.111/0001-11 é aceito.
- CNPJ **sem pontuação** não é achado.
- Os diários às vezes **mascaram** o CNPJ (07.XXX.XXX/XXXX-58, comum nos extratos de BH): o X não é dígito, então o molde não casa e a empresa fica sem CNPJ.
- Não confunde CPF (069.555.516-20) com CNPJ, porque o CPF não tem / (há um teste para isso).

ExtratorData.java — datas numéricas e por extenso

ExtratorData.java

Filha: acha datas numéricas (dd/mm/aaaa) e por extenso ("24 de março de 2026").

pacote diarios.extratores
82 linhas · PDF 1, **pág. 18**

Objetivo. Achar datas em dois formatos: `24/03/2026` (ou `24/03/26`) e por extenso, `20 de março de 2026`.

ExtratorData.java · linhas 25–35

```
25 private void acharNumericas(String texto, ArrayList<Fato> achados) {
26     for (int i = 0; i < texto.length(); i++) {
27         if (casaMolde(texto, i, "99/99/9999")) {
28             achados.add(criar(texto.substring(i, i + 10), i, i + 10));
29             i += 9;
30         } else if (casaMolde(texto, i, "99/99/99")) {
31             achados.add(criar(texto.substring(i, i + 8), i, i + 8));
32             i += 7;
33         }
34     }
35 }
```

- Tenta primeiro o molde longo `99/99/9999`; se não casar, o curto `99/99/99`. Cada molde que casa avança `i` para não sobrepor.
- **Não valida** a data: `99/99/9999` seria aceito.

Por extenso é mais engenhoso: parte do **nome do mês** e anda para os dois lados.

ExtratorData.java · linhas 37–50

```

37 private void acharPorExtenso(String texto, ArrayList<Fato> achados) {
38     String base = semAcento(texto.toLowerCase());
39     for (String mes : MESES) {
40         int p = base.indexOf(mes);
41         while (p >= 0) {
42             int inicio = recuarAteDia(base, p);
43             int fim = avancarAteAno(base, p + mes.length());
44             if (inicio >= 0 && fim > 0) {
45                 achados.add(criar(texto.substring(inicio, fim), inicio, fim));
46             }
47             p = base.indexOf(mes, p + mes.length());
48         }
49     }
50 }

```

1. `base = semAcento(texto.toLowerCase())`: tudo em minúsculas e sem acento, assim "MARÇO", "Março" e "março" viram `marco`.
2. Para cada mês da lista `MESES`, acha todas as ocorrências (`indexOf` repetido).
3. `recuarAteDia`: da posição do mês, volta e confere que antes dele vem " de " e, antes disso, **dígitos** (aceitando `0`, como em "1º de março"); devolve onde o dia começa, ou `-1`.
4. `avancarAteAno`: depois do mês exige " de " e **exatamente 4 dígitos**; devolve onde o ano termina, ou `-1`.
5. Se os dois deram certo, o fato é `texto.substring(inicio, fim)` **do texto original** (com a grafia original, ex.: 25 DE MARÇO DE 2026).

NOTA — Datas de rodapé

No diário de BH aparecem datas como 24/03/2026 18:39:28 que são **rodapés de página do PDF** (Dom7466.indd), e entram como fatos DATA do ato onde estavam. Isso infla a contagem de datas e é uma das razões de "tem data" não ser sinal forte de qualidade.

ExtratorValor.java — valores em reais

ExtratorValor.java

Filha: acha valores em reais ("R\$ 1.234,56") e converte para número.

pacote diarios.extratores
64 linhas · PDF 1, **pág. 19**

Objetivo. Achar R\$ 1.234,56 e convertê-lo em número.

ExtratorValor.java · linhas 28–46

```

28 private int medir(String texto, int inicioRS) {
29     int p = inicioRS + 2;
30     while (p < texto.length() && (texto.charAt(p) == ' ' || texto.charAt(p) == '\u00a0')) {
31         p++;
32     }
33     if (contarDigitos(texto, p) == 0) {
34         return -1;
35     }
36     while (p < texto.length() && (ehDigito(texto.charAt(p)) || texto.charAt(p) == '.')) {
37         p++;
38     }
39     boolean temCentavos = p + 2 < texto.length() && texto.charAt(p) == ',' &&
40         ehDigito(texto.charAt(p + 1)) && ehDigito(texto.charAt(p + 2));
41     if (!temCentavos) {
42         return -1;
43     }
44     boolean digitoExtra = p + 3 < texto.length() && ehDigito(texto.charAt(p + 3));
45     return digitoExtra ? -1 : p + 3;
46 }

```

Regras do `medir`, depois de achar "R\$":

1. Pula espaços (o espaço comum e o espaço não separável, `U+00A0`, que aparece em texto vindo de PDF).
2. Exige **pelo menos um dígito**.
3. Consome dígitos e pontos (1.234).

4. Exige **vírgula + exatamente 2 dígitos** (,56). Se não tiver, devolve -1: R\$ 18 é ignorado (há teste: "ignora R\$ sem centavos").
5. Se houver um **terceiro** dígito depois dos centavos, também recusa (-1).

ExtratorValor.java · linhas 48–63

```

48 public static double paraNumero(String valorTexto) {
49     StringBuilder limpo = new StringBuilder();
50     for (int i = 0; i < valorTexto.length(); i++) {
51         char c = valorTexto.charAt(i);
52         if (c >= '0' && c <= '9') {
53             limpo.append(c);
54         } else if (c == ',') {
55             limpo.append('.');
56         }
57     }
58     try {
59         return Double.parseDouble(limpo.toString());
60     } catch (NumberFormatException e) {
61         return 0.0;
62     }
63 }

```

paraNumero guarda só dígitos, troca a vírgula por ponto ("R\$ 184.500,00" → "184500.00") e usa Double.parseDouble; se der NumberFormatException, devolve 0.0.

PONTO FRACO — SEJA HONESTO — "R\$ 7,02 milhões" vira R\$ 7,02

Testei: ExtratorValor lê R\$ 7,02 milhões como o valor **7,02**. Ele não olha a palavra seguinte ("milhões", "mil"). Um diário que escreva valores por extenso em notas (como nas notícias do diário do Recife) produz valores pequenos demais. Resposta honesta: "foi desenhado para os extratos, que escrevem o valor completo, como R\$ 700.000,00".

ExtratorProcesso.java — número do processo

ExtratorProcesso.java

Filha: acha o número do processo depois da palavra "processo".

pacote diarios.extratores
62 linhas · PDF 1, **pág. 20**

Objetivo. Achar o número que vem depois da palavra "processo" (Processo: 31.00204095/2026-45).

ExtratorProcesso.java · linhas 16–30

```

16 public ArrayList<Fato> extrair(String texto) {
17     ArrayList<Fato> achados = new ArrayList<>();
18     int p = texto.indexOf(MARCA);
19     while (p >= 0) {
20         int inicio = pularAteNumero(texto, p + MARCA.length());
21         if (inicio > 0) {
22             int fim = medirNumero(texto, inicio);
23             if (fim - inicio >= TAMANHO_MINIMO) {
24                 achados.add(criar(texto.substring(inicio, fim), inicio, fim));
25             }
26         }
27         p = texto.indexOf(MARCA, p + MARCA.length());
28     }
29     return achados;
30 }

```

- A marca é "rocesso" (sem o P/p inicial) para pegar "**Processo**" e "**processo**" de uma só vez, sem converter o texto para minúsculas.
- pularAteNumero: nos **até 12 caracteres** seguintes aceita só "enfeites" (:, espaço, n, N, º, °, .), como em Processo nº 31... Se aparecer outra coisa antes de um dígito, desiste (-1): assim "processo seletivo 123456" não gera fato.
- medirNumero: consome dígitos e ., -, /; depois **recua** os caracteres finais que não são dígitos (o ponto que fecha a frase, por exemplo).
- Só aceita se o número tiver **no mínimo 6 caracteres** (TAMANHO_MINIMO).

ExtratorReferencia.java — citações de normas

ExtratorReferencia.java

Filha: acha citações de normas (lei, decreto, portaria...) e as padroniza ("lei:11065").

pacote diarios.extratores
106 linhas · PDF 1, **pág. 21**

Objetivo. Achar menções a leis, decretos, portarias, resoluções e instruções normativas e **padronizá-las** (lei:11065, portaria:073), para que a mesma norma escrita de jeitos diferentes seja a mesma coisa no grafo. É o extrator mais cuidadoso do pacote.

ExtratorReferencia.java · linhas 19–43

```
19 public ArrayList<Fato> extrair(String texto) {
20     ArrayList<Fato> achados = new ArrayList<>();
21     String base = semAcento(texto.toLowerCase());
22
23     for (String especie : ESPECIES) {
24         int p = base.indexOf(especie);
25         while (p >= 0) {
26             boolean ehPrefixoDeOutra = especie.equals("lei")
27                 && base.startsWith("lei complementar", p);
28             if (!ehPrefixoDeOutra) {
29                 int inicioNumero = pularAteNumero(base, texto, p + especie.length());
30                 if (inicioNumero > 0) {
31                     int fimNumero = medirNumero(base, inicioNumero);
32                     if (fimNumero > inicioNumero) {
33                         achados.add(criar(
34                             canonica(especie, texto.substring(inicioNumero, fimNumero)),
35                             p, fimNumero));
36                     }
37                 }
38             }
39             p = base.indexOf(especie, p + especie.length());
40         }
41     }
42     return achados;
43 }
```

Pontos importantes:

- A lista ESPECIES começa por "lei complementar" e só depois "lei". E há um teste explícito, `ehPrefixoDeOutra`, para que "Lei Complementar 123" não seja lida também como "Lei".
- **Padronização** (`canonica`): junta a espécie (lei-complementar) com os dígitos do número **antes da barra**: Lei nº 11.065 e LEI MUNICIPAL 11065 dão ambas lei:11065 (há teste "grafias diferentes, mesma norma").
- O fato cobre do início da palavra da espécie até o fim do número (p a `fimNumero`).

ExtratorReferencia.java · linhas 59–81

```
59 private int pularAteNumero(String base, String original, int posicao) {
60     int limite = Math.min(base.length(), posicao + 28);
61     for (int i = posicao; i < limite; i++) {
62         char c = base.charAt(i);
63         if (ehDigito(c)) {
64             return i;
65         }
66         if (" .:-/,ººnoum".indexOf(c) >= 0) {
67             continue;
68         }
69         int pulo = tamanhoDaEsfera(base, i);
70         if (pulo > 0) {
71             i += pulo - 1;
72             continue;
73         }
74         boolean siglaDeOrgao = Character.isLetter(c) && i < original.length()
75             && Character.isUpperCase(original.charAt(i));
76         if (!siglaDeOrgao) {
77             return -1;
78         }
79     }
80     return -1;
81 }
```

`pularAteNumero` (janela de 28 caracteres) aceita entre a espécie e o número: espaços e pontuação, nº/no/num, a **esfera** (municipal, federal, estadual, conjunta) e uma **sigla de órgão em MAIÚSCULA** no

texto original. Isso é a decisão 8: sem a sigla, Portaria SMC nº 073/2025 não seria achada, e essa era a **única** alteração de norma real do diário. A sigla só vale se for maiúscula no original: aceitar minúscula faria "lei que trata do artigo 5" virar citação da lei 5.

PONTO FRACO — SEJA HONESTO — Duas limitações da padronização

- canônica **descarta o ano** (para na barra): Portaria 073/2025 e Portaria 073/2026 viram o mesmo portaria:073. Como portarias de órgãos reiniciam a numeração a cada ano, duas normas diferentes podem colapsar no grafo.
- O **título do próprio ato** ("PORTARIA Nº 016/2026") também é lido como referência, e vira uma relação CITA do ato para si mesmo (aparece no grafo real: decreto:19524 --CITA--> decreto:19524). É ruído inofensivo, mas é ruído.

Como tudo se conecta: Extracao.java

Extracao.java

Etapa 4: roda todos os extratores em cada ato (polimorfismo).

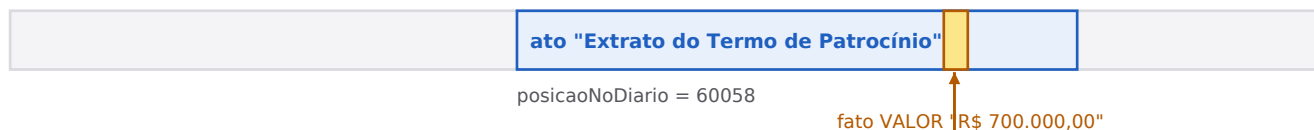
pacote diarios.etapas
37 linhas · PDF 1, **pág. 27**

Objetivo. Rodar **todos os extratores em todos os atos** de um diário e guardar os fatos. É a classe que mostra o polimorfismo.

Extracao.java · linhas 14–37

```
14 public class Extracao {
15
16     private final ArrayList<Extrator> extratores = new ArrayList<>();
17
18     public Extracao() {
19         extratores.add(new ExtratorData());
20         extratores.add(new ExtratorValor());
21         extratores.add(new ExtratorCnpj());
22         extratores.add(new ExtratorProcesso());
23         extratores.add(new ExtratorReferencia());
24     }
25
26     public void extrair(Diario diario) {
27         for (Ato ato : diario.getAtos()) {
28             for (Extrator e : extratores) {
29                 for (Fato f : e.extrair(ato.getTexto())) {
30                     ato.adicionarFato(new Fato(f.getTipo(), f.getValor(),
31                                             f.getInicio() + ato.getPosicao(),
32                                             f.getFim() + ato.getPosicao()));
33                 }
34             }
35         }
36     }
37 }
```

1. O construtor monta a lista `extratores` com as cinco filhas (todas vistas como `Extrator`).
2. `extrair(Diario)` percorre cada `Ato` e cada `Extrator`; chama `e.extrair(ato.getTexto())`.
3. **Detalhe essencial:** os extratores devolvem posições **relativas ao texto do ato**. `Extracao` cria um **novo** `Fato` somando `ato.getPosicao()` às duas posições, e então elas passam a ser **relativas ao diário**. Por isso depois `Fato.provar(d.getTextoLimp(), ...)` funciona e `GrafoNormas` consegue olhar o texto do diário em volta da citação.

textoLimpdo do diário de BH**O que cada classe faz com a posição****ExtratorValor.extrair(ato.getTexto())**

acha "R\$ 700.000,00" em 772 .. 785
relativo ao ATO (o extrator só enxerga o texto do ato)

Extracao.extrair(diario)

soma ato.getPosicao(): 772 + 60058 = 60830 .. 60843
agora relativo ao DIÁRIO — é o Fato que fica guardado

Fato.provar(textoLimpdo, margem)

textoLimpdo.substring(60830 - m, 60843 + m)
recorta o original: "...Valor total: R\$ 700.000,00 Prazo..."

PaginaDiario.destacar(ato)

inicio - ato.getPosicao() = 772 .. 785
volta a ser relativo ao ATO, para pôr <mark> no texto do ato

Por isso *GrafoNormas* e *PainelContratos*, que usam *d.getTextoLimpdo()*, comparam posições de fatos diretamente com o texto do diário inteiro.

DICA — Se pedirem "acrescente um extrator na minha frente"

Veja o Exercício A no capítulo de perguntas: constante nova em Fato, classe nova ExtratorCep
extends Extrator, **uma linha** em Extracao. Nada mais muda. Esse é o argumento do polimorfismo.

4. Pacote etapas e Aplicacao.processarPasta()

O pipeline, etapa por etapa

`Aplicacao.processarPasta()` **é o projeto inteiro em ~40 linhas**. Todos precisam saber explicá-lo. Leia-o devagar e depois siga cada chamada até a classe correspondente.

Aplicacao.processarPasta() — o esqueleto

Aplicacao.java

O "cérebro": `processarPasta()`, `coletar()`, `descobrir()` e a coleta em segundo plano.

pacote diarios
306 linhas · PDF 1, **pág. 81**

Aplicacao.java · linhas 69–111

```
69 public void processarPasta() {
70     ArrayList<String> falhas = new ArrayList<>();
71     ArrayList<Diario> lidos = new Ingestao(PASTA_DIARIOS).lerTodos(falhas);
72
73     ArrayList<Diario> novos = new ArrayList<>();
74     Cobertura novaCobertura = new Cobertura();
75     Limpeza limpeza = new Limpeza();
76     Extracao extracao = new Extracao();
77     Portao portao = new Portao();
78
79     for (Diario d : lidos) {
80         try {
81             limpeza.limpar(d);
82         } catch (DiarioIlegivelException e) {
83             novaCobertura.registrarRecusa(e.getArquivo(), e.getMessage(), e.getEvidencia());
84             novos.add(d);
85             continue;
86         }
87         new Fragmentacao().fragmentar(d);
88         extracao.extrair(d);
89         portao.avaliar(d);
90         novaCobertura.registrarProcessado(d);
91         novos.add(d);
92     }
93
94     PaineisContratos novoPainel = new PaineisContratos();
95     novoPainel.processar(novos);
96     GrafoNormas novoGrafo = new GrafoNormas();
97     novoGrafo.processar(novos);
98     Interpretacao novaInterpretacao = new Interpretacao();
99     novaInterpretacao.interpretar(novos, novaCobertura, novoPainel, novoGrafo);
100
101     synchronized (this) {
102         diarios = novos;
103         cobertura = novaCobertura;
104         painel = novoPainel;
105         grafo = novoGrafo;
106         interpretacao = novaInterpretacao;
107         falhasDeLeitura = falhas;
108         ultimoProcessamento = agora();
109     }
110     fontes.atualizarAcervo(PASTA_DIARIOS);
111 }
```

Passo a passo:

- Lê a pasta.** `new Ingestao(PASTA_DIARIOS).lerTodos(falhas)` devolve a lista de `Diario` (e anota em `falhas` o que não conseguiu abrir).
- Prepara as peças:** `Cobertura` nova, `Limpeza`, `Extracao`, `Portao`. A `Extracao` e o `Portao` são criados **uma vez** e reaproveitados para todos os diários; a `Fragmentacao` é criada **uma por diário** (ela guarda contadores).
- Para cada diário:**

- tenta `limpeza.limpar(d)`. Se lançar `DiarioIlegivelException`, **registra a recusa** em `Cobertura`, guarda o diário na lista (ele aparece no painel como RECUSADO) e faz `continue`: segue para o próximo, sem fragmentar nem extrair;
 - se a limpeza passou: **fragmenta** → **extrai** → **avalia no portão** → registra o diário como processado.
- 4. Análise sobre o conjunto:** `PainelContratos.processar(novos)`, `GrafoNormas.processar(novos)`, `Interpretacao.interpretar(...)`. Todas só olham os diários utilizáveis.
- 5. Troca atômica:** tudo foi montado em **variáveis locais** (`novos`, `novaCobertura`, `novoPainel...`). Só no fim, dentro de `synchronized (this)`, os campos da aplicação são trocados de uma vez. Assim, quem está lendo os dados (a página do painel, em outra thread) nunca vê um estado "pela metade".
- 6.** `fontes.atualizarAcervo(...)` conta quantos arquivos cada fonte tem na pasta.

P	Explique <code>processarPasta()</code> em ordem .
R	Lê a pasta em <code>Diarios</code> ; para cada um, limpa (e se o diário for ilegível, registra a recusa e passa ao próximo), fragmenta em atos, extrai os fatos de cada ato e dá a nota no portão. Depois, com todos os diários processados, monta o painel de contratos, o grafo de normas e a interpretação, e só então troca os resultados da aplicação de uma vez (<code>synchronized</code>), para a outra thread nunca ver dados pela metade.
P	Por que o try/catch fica dentro do laço?
R	Para que um diário ruim não derrube os outros . A exceção é tratada por diário, e o <code>continue</code> pula só aquele.

Ingestao.java — etapa 1: ler a pasta

Ingestao.java

Etapa 1: lista a pasta e cria um `Diario` por arquivo `.txt`.

pacote `diarios.etapas`
60 linhas · PDF 1, **pág. 23**

Objetivo. Transformar os arquivos `.txt` de uma pasta em objetos `Diario`.

`Ingestao.java` · linhas 21–46

```

21 public ArrayList<Diario> lerTodos(ArrayList<String> falhas) {
22     ArrayList<Diario> lidos = new ArrayList<>();
23     ArrayList<Path> arquivos = new ArrayList<>();
24
25     try (DirectoryStream<Path> fluxo = Files.newDirectoryStream(Paths.get(pasta), "*.txt")) {
26         for (Path p : fluxo) {
27             arquivos.add(p);
28         }
29     } catch (IOException e) {
30         falhas.add("nao foi possivel abrir a pasta '" + pasta + "': " + e.getMessage());
31         return lidos;
32     }
33
34     Collections.sort(arquivos);
35
36     for (Path p : arquivos) {
37         String nome = p.getFileName().toString();
38         try {
39             lidos.add(new Diario(nome, municipioDe(nome), dataDe(nome),
40                 Files.readString(p, StandardCharsets.UTF_8)));
41         } catch (IOException e) {
42             falhas.add(nome + ": falha de leitura - " + e.getMessage());
43         }
44     }
45     return lidos;
46 }
```

- `Files.newDirectoryStream(Paths.get(pasta), "*.txt")` lista só os `.txt`. Está dentro de um `try-with-resources` (`Ingestao.java:25`), que **fecha o recurso sozinho**, mesmo se der erro (Aula 05).
- `Collections.sort(arquivos)`: ordem alfabética, para que a execução seja **determinística** (o `DirectoryStream` não garante ordem).

- `Files.readString(p, UTF_8)` lê o arquivo inteiro como `String`.
- **Parâmetro de saída** falhas: se a pasta não abre, ou um arquivo falha, a `IOException` é capturada, o motivo vai para `falhas` e o método devolve o que conseguiu (lista vazia no primeiro caso). **O programa não cai** com uma pasta inexistente.
- `municipioDe` e `dataDe` extraem município e data **do nome do arquivo** (`belo-horizonte_2026-03-25.txt`): antes do primeiro `_` e os 10 caracteres depois dele. Funciona também com `recife_2026-09-29-extra116.txt` (edição extra).

Limpeza.java — etapa 2: limpar e, se preciso, recusar

Limpeza.java

Etapa 2: remove lixo de PDF e RECUSA o diário sem texto aproveitável.

pacote diarios.etapas
61 linhas · PDF 1, **pág. 24**

Objetivo. Duas coisas: **tirar o lixo** da conversão de PDF e **recusar** o diário que não tem texto aproveitável.

Limpeza.java · linhas 17–47

```

17 public void limpar(Diario diario) throws DiarioIlegivelException {
18     String[] linhas = diario.getTextoBruto().split("\n");
19     StringBuilder limpo = new StringBuilder();
20     int removidas = 0;
21     int longas = 0;
22
23     for (String linha : linhas) {
24         String s = linha.trim();
25         if (s.isEmpty()) {
26             continue;
27         }
28         if (s.length() <= MINIMO_DE_CARACTERES || ehMobiliaDePagina(s)) {
29             removidas++;
30             continue;
31         }
32         if (s.length() > LINHA_LONGA) {
33             longas++;
34         }
35         limpo.append(s).append('\n');
36     }
37
38     diario.registrarLimpeza(limpo.toString(), removidas);
39
40     if (longas < MINIMO_DE_LINHAS_LONGAS) {
41         String motivo = "PDF sem texto extraivel (provavelmente escaneado)";
42         diario.recusar(motivo);
43         throw new DiarioIlegivelException(diario.getArquivo(), motivo,
44             longas + " linha(s) de corpo em " + linhas.length
45             + " linhas do arquivo; o minimo esperado e " + MINIMO_DE_LINHAS_LONGAS);
46     }
47 }

```

Como funciona:

- `split("\n")` quebra em linhas; `trim()` tira as pontas; linha vazia é simplesmente ignorada (não conta como lixo).
- **Lixo** = linha com **3 caracteres ou menos** (`MINIMO_DE_CARACTERES`) — a marca d'água vertical sai picotada, uma ou duas letras por linha — **ou** "móbia de página" (`ehMobiliaDePagina`): linhas com CHAVE DE INÍCIO/ACESSO DA MATÉRIA, cabeçalho com DIÁRIO OFICIAL + EDIÇÃO N, ou que começam com HTTPS:// ou WWW.. Conta em `removidas`.
- Conta quantas linhas têm **mais de 80 caracteres** (`LINHA_LONGA`): são o "corpo" de verdade.
- `diario.registrarLimpeza(...)` guarda o texto limpo e o número de linhas removidas.
- **Recusa**: se houver **menos de 10 linhas longas** (`MINIMO_DE_LINHAS_LONGAS`), marca o diário como recusado (`diario.recusar`) e **lança** `DiarioIlegivelException` com a evidência numérica.

Por que 10? Aracaju (PDF escaneado) tem **1** linha longa; BH tem mais de mil. Qualquer número entre os dois separa os casos; 10 foi o escolhido.

P	Por que descartar as letras soltas da marca d'água e não remontá-las?
R	Foi testado: remontar colava o título de um ato no parágrafo anterior e o programa passava a perder atos que antes achava. Descartar linha curta é seguro: o corpo do documento está intacto por baixo. (Decisão 3.)
P	Por que lançar exceção em vez de devolver "0 atos"?
R	"0 atos" é indistinguível de "dia sem publicação". São situações diferentes: uma é "não havia nada", outra é "havia, mas não consigo ler". A exceção carrega motivo e evidência.

NOTA — Um detalhe de implementação

`s.length() <= 3` também remove linhas curtas legítimas (um "§ 1" isolado, por exemplo). Aceitável: o que se perde é irrelevante para o resultado e simplifica muito.

Fragmentacao.java — etapa 3: cortar em atos

Fragmentacao.java

Etapa 3: corta o diário em atos; só corta com evidência de corpo de texto.

pacote diarios.etapas
125 linhas · PDF 1, **pág. 25**

Objetivo. Transformar o texto limpo em uma **lista de atos**. É a parte mais delicada do pipeline.

Fragmentacao.java · linhas 23–63

```

23 public void fragmentar(Diario diario) {
24     String texto = diario.getTextoLimpo();
25     ArrayList<Integer> inicios = new ArrayList<>();
26     ArrayList<String> titulos = new ArrayList<>();
27     ArrayList<String> especies = new ArrayList<>();
28
29     int posicao = 0;
30     for (String linha : texto.split("\n")) {
31         String s = linha.trim();
32         String especie = especieDaLinha(s);
33         if (especie != null && s.length() <= TITULO_MAXIMO) {
34             inicios.add(posicao);
35             titulos.add(s);
36             especies.add(especie);
37         }
38         posicao += linha.length() + 1;
39     }
40
41     candidatos = inicios.size();
42     descartados = 0;
43
44     for (int i = 0; i < inicios.size(); i++) {
45         int comeco = inicios.get(i);
46         int fim = (i + 1 < inicios.size()) ? inicios.get(i + 1) : texto.length();
47         String bloco = texto.substring(comeco, fim);
48
49         if (!temCorpo(bloco, titulos.get(i))) {
50             descartados++;
51             continue;
52         }
53         Ato novo = new Ato(diario.getMunicipio(), diario.getData(),
54                             especies.get(i), numeroDoTitulo(titulos.get(i)),
55                             titulos.get(i), bloco, comeco);
56         int repeticoes = contarIdentidade(diario, novo.getIdentidade());
57         if (repeticoes > 0) {
58             novo.desambiguar(repeticoes + 1);
59         }
60         diario.adicionarAto(novo);
61     }
62     diario.registrarFragmentacao(candidatos, descartados);
63 }

```

Em duas fases:

Fase 1 — achar títulos candidatos. Para cada linha, `especieDaLinha(s)` diz se ela **começa** com uma espécie conhecida (lista `ESPECIES`: DECRETO, LEI COMPLEMENTAR, LEI, PORTARIA, RESOLUCAO, EDITAL, INSTRUCAO NORMATIVA, EXTRATO, TERMO, AVISO, ATO, ORDEM DE SERVICO, DELIBERACAO,

COMUNICADO, RETIFICACAO), e se tem **no máximo 120 caracteres** (TITULO_MAXIMO). Anota a posição (posicao acumula linha.length() + 1 para o \n), o título e a espécie.

Fragmentacao.java · linhas 94–109

```

94 private String especieDaLinha(String linha) {
95     String s = Extrator.semAcento(linha.toUpperCase());
96     for (String e : ESPECIES) {
97         if (!s.startsWith(e)) {
98             continue;
99         }
100         if (e.length() >= s.length()) {
101             return e;
102         }
103         char depois = s.charAt(e.length());
104         if (" Nº:-.".indexOf(depois) >= 0) {
105             return e;
106         }
107     }
108     return null;
109 }

```

- A espécie precisa vir **seguida de** espaço, N, º, :, - ou . (ou ser a linha inteira): "EXTRATOS" **não** conta como "EXTRATO".
- A ordem de ESPECIES importa: LEI COMPLEMENTAR **antes de** LEI, porque a busca para no primeiro que casa; se LEI viesse antes, "LEI COMPLEMENTAR Nº 5" seria lida como LEI.

Fase 2 — decidir quais viram ato. O bloco de cada candidato vai **da sua posição até a do próximo candidato** (ou até o fim). Só vira Ato se temCorpo:

Fragmentacao.java · linhas 78–92

```

78 private boolean temCorpo(String bloco, String titulo) {
79     if (bloco.length() <= titulo.length()) {
80         return false;
81     }
82     String semTitulo = bloco.substring(titulo.length());
83     if (semTitulo.trim().length() < CORPO_MINIMO) {
84         return false;
85     }
86     for (String l : semTitulo.split("\n")) {
87         if (l.trim().length() > PARAGRAFO_MINIMO) {
88             return true;
89         }
90     }
91     return false;
92 }

```

A regra do corpo (decisão 4): depois de tirar o título, o que sobra precisa ter **pelo menos 200 caracteres** (CORPO_MINIMO) e pelo menos **uma linha com mais de 60 caracteres** (PARAGRAFO_MINIMO). A ideia: um título que só é seguido de outros títulos é um **sumário** ("DECRETO NUMERADOS - 8512, DECRETO SIMPLES - 2026-02-23..."); um fragmentador que corta sempre que vê um título criaria dezenas de atos vazios, e **toda contagem do programa passaria a mentir**. Na dúvida, **não corta**.

Quando vira ato: numeroDoTitulo extrai o número (primeiro dígito em diante, tolerando .: "Nº 19.524, DE..." → 19524), cria o Ato com posicao = começo, calcula a identidade e, se já houver uma igual no diário, chama **desambiguar** (#2, #3...). No fim registra **candidatos** e **descartados** no Diario, que alimentam o "funil".

PONTO FRACO — SEJA HONESTO — A regra do corpo descarta atos reais (verifiquei rodando)

Em BH, dos **129** candidatos descartados, só **22** têm menos de 200 caracteres (os "vazios", como sumário). Os outros **107** têm corpo grande, mas **nenhuma linha passa de 60 caracteres**, porque o PDF tem colunas estreitas. Exemplos reais descartados: vários "Extrato do Termo de Execução Cultural", com processo, contratado, objeto e Valor total: R\$100.000,00. São atos de verdade que **ficam fora de todos os totais**. Resposta honesta: "a regra privilegia não inventar atos; o custo é deixar passar atos de coluna estreita, e isso aparece no funil (225 → 96)". Mais detalhes no capítulo "Pontos de atenção".

PONTO FRACO — SEJA HONESTO — Defeito: espécies acentuadas nunca são reconhecidas

especieDaLinha faz `Extrator.semAcento(linha.toUpperCase())`: converte para maiúsculas **antes** de tirar o acento, mas `semAcento` só troca letras **minúsculas**. Então "RESOLUÇÃO", "RETIFICAÇÃO", "DELIBERAÇÃO", "INSTRUÇÃO NORMATIVA" e "ORDEM DE SERVIÇO", escritos com acento, **não casam** com as espécies sem acento da lista (RESOLUCAO...). Reproduzi: o texto RESOLUÇÃO Nº 11/2026 não vira ato, enquanto RESOLUCAO Nº 10/2026 vira. No BH real, RESOLUÇÃO CMAS-BH Nº 028/2026 e várias linhas RETIFICAÇÃO ficam de fora. A correção é trocar a ordem (`semAcento(linha.toLowerCase()).toUpperCase()`), mas **não mexi no código**; veja o capítulo "Pontos de atenção".

P Como a fragmentação evita contar o sumário como atos?

R Pela função `temCorpo`: um título só vira ato se depois dele houver texto de verdade (pelo menos 200 caracteres e uma linha com mais de 60). Sumário é lista de títulos curtos, sem corpo.

P Por que a identidade ganha #2?

R Porque duas portarias começavam com a mesma primeira linha e ficavam com a mesma identidade, e a leitura de uma herdava a alteração da outra. Achei olhando a página do ato, não num teste; hoje há um teste que trava isso ("nenhuma identidade repetida no mesmo diário").

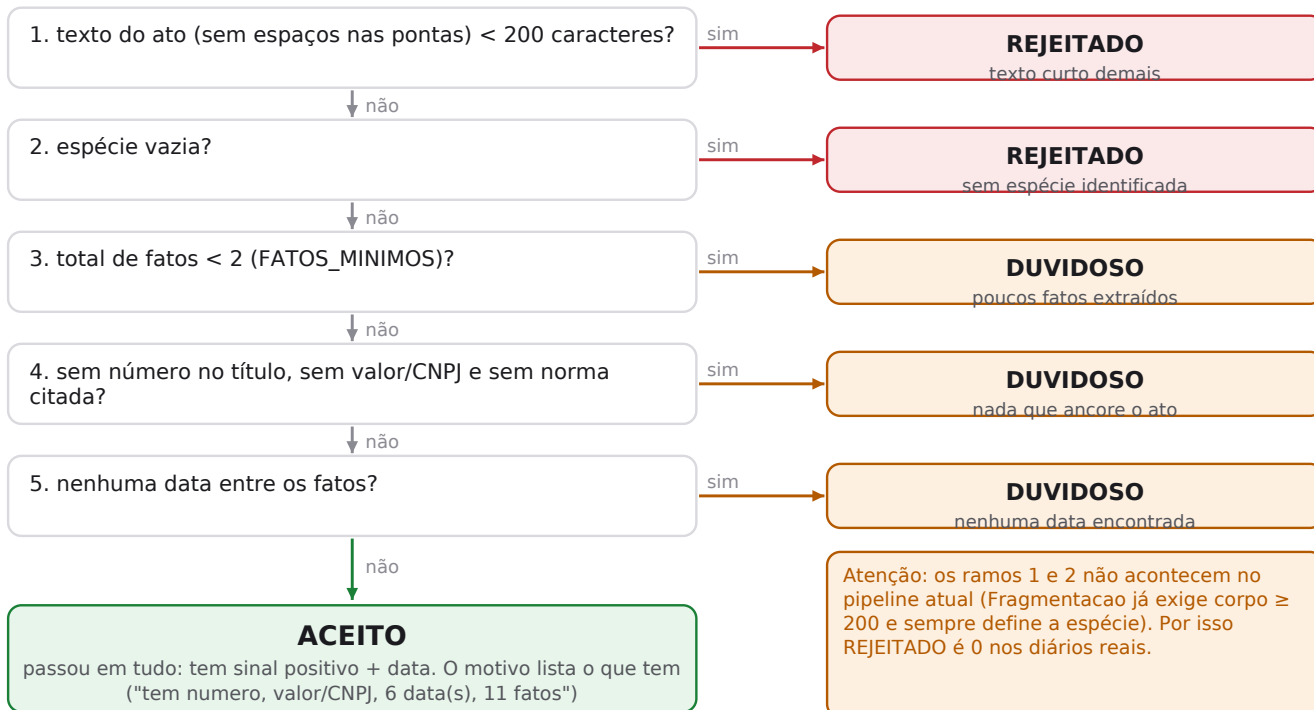
Portao.java — etapa 5: dar a nota

Portao.java

Etapa 5: dá a nota a cada ato — ACEITO, DUVIDOSO ou REJEITADO — com motivo.

pacote diarios.etapas
63 linhas · PDF 1, **pág. 28**

Objetivo. Dar a cada ato uma de **três notas**, sempre com o **motivo**.



Só chega em ACEITO quem passa por TODOS os testes — nenhum caminho aceita "por omissão".
Todo ramo chama `ato.decidir(decisao, motivo)`: o motivo é sempre gravado.

Portao.java · linhas 18–62

```

18 private void avaliarAto(Ato ato) {
19     int tamanho = ato.getTexto().trim().length();
20     if (tamanho < TEXTO_MINIMO) {
21         ato.decidir(Ato.REJEITADO, "texto curto demais (" + tamanho
22             + " caracteres, mínimo " + TEXTO_MINIMO + ")");
23         return;
24     }
25     if (ato.getEspecie().isEmpty()) {
26         ato.decidir(Ato.REJEITADO, "sem especie identificada");
27         return;
28     }
29
30     int datas = ato.getFatosDoTipo(Fato.DATA).size();
31     int total = ato.getFatos().size();
32     boolean temNumero = !ato.getNumero().isEmpty();
33     boolean temDinheiro = !ato.getFatosDoTipo(Fato.VALOR).isEmpty()
34         || !ato.getFatosDoTipo(Fato.CNPJ).isEmpty();
35     boolean temNorma = !ato.getFatosDoTipo(Fato.REFERENCIA).isEmpty();
36
37     if (total < FATOS_MINIMOS) {
38         ato.decidir(Ato.DUVIDOSO, "poucos fatos extraídos (" + total + ")");
39         return;
40     }
41     if (!temNumero && !temDinheiro && !temNorma) {
42         ato.decidir(Ato.DUVIDOSO, "sem numero, sem valor/CNPJ e sem norma citada");
43         return;
44     }
45     if (datas == 0) {
46         ato.decidir(Ato.DUVIDOSO, "nenhuma data encontrada no ato");
47         return;
48     }
49
50     StringBuilder porque = new StringBuilder("tem ");
51     if (temNumero) {
52         porque.append("numero, ");
53     }
54     if (temDinheiro) {
55         porque.append("valor/CNPJ, ");
56     }
57     if (temNorma) {
58         porque.append("norma citada, ");
59     }
60     porque.append(datas).append(" data(s), ").append(total).append(" fatos");
61     ato.decidir(Ato.ACEITO, porque.toString());
62 }

```

Leia como uma sequência de ifs, na ordem do código:

1. Texto do ato (com `trim`) com menos de **200** caracteres → `REJEITADO`.
2. Espécie vazia → `REJEITADO`.
3. Menos de **2** fatos no total (`FATOS_MINIMOS`) → `DUVIDOSO`.
4. Sem **número** no título **e** sem valor/CNPJ **e** sem norma citada → `DUVIDOSO`.
5. **Nenhuma data** entre os fatos → `DUVIDOSO`.
6. Passou por todos → `ACEITO`, com o motivo montado por um `StringBuilder` ("tem numero, valor/CNPJ, 6 data(s), 11 fatos").

O ponto central (decisão 6): só chega em `ACEITO` quem passou por todos os `if`. Nenhum caminho leva a "aceito" por omissão. E toda decisão grava o motivo, inclusive as rejeitadas.

Por que três saídas e não duas?

erro	o que acontece
deixa passar lixo	aparece: alguém lê a saída e vê
descarta coisa boa	não aparece nunca

Com duas saídas, o caso duvidoso seria empurrado para um dos lados e sumiria. Com a terceira, ele fica **contável**: os 30 `DUVIDOSO` de BH são a lista do que o programa ainda não trata.

PONTO FRACO — SEJA HONESTO — REJEITADO nunca acontece no pipeline atual

Os dois primeiros ramos são inalcançáveis hoje: `Fragmentacao.temCorpo` já exige pelo menos 200 caracteres de corpo (então o texto do ato sempre passa de 200) e a espécie nunca é vazia (um título só é candidato se começar com uma espécie). Por isso os relatórios dizem `REJEITADO 0` em todos os diários. O ramo existe como **rede de segurança**: se um dia a fragmentação mudar (ou outro código criar atos), o portão continua barrando texto curto e ato sem espécie. Se o professor perguntar "REJEITADO nunca aparece?", é isto.

P Por que `FATOS_MINIMOS` é uma constante e não um número solto no `if`?

R Porque é um **parâmetro de ajuste**. Mudar de 2 para 3 faz BH passar de 66 aceitos / 30 duvidosos para **58 / 38** (verifiquei). Isso mostra que o limite é uma decisão explícita e que o número de "duvidosos" responde a ele. É o Exercício B.

Privacidade.java — mascarar CPF

Privacidade.java

Mascara CPFs (***,***,***-20) sem mudar o tamanho do texto.

pacote diarios.etapas
49 linhas · PDF 1, **pág. 29**

Objetivo. Diários publicam CPF completo (servidores nomeados, exonerados). O painel mostra

..***-20.

Privacidade.java · linhas 7–28

```

7 public static String mascararCpf(String texto) {
8     if (texto == null) {
9         return "";
10    }
11    StringBuilder saida = new StringBuilder(texto);
12    int i = 0;
13    while (i + MOLDE_CPF.length() <= saida.length()) {
14        if (casa(saida, i) && !digitoAntes(saida, i)
15            && !digitoDepois(saida, i + MOLDE_CPF.length())) {
16            for (int k = 0; k < 11; k++) {
17                char c = saida.charAt(i + k);
18                if (c >= '0' && c <= '9') {
19                    saida.setCharAt(i + k, '*');
20                }
21            }
22            i += MOLDE_CPF.length();
23        } else {
24            i++;
25        }
26    }
27    return saida.toString();
28 }
```

- Percorre um `StringBuilder` procurando o molde `999.999.999-99` (mesma ideia do `casaMolde`).
- Só mascara se **não houver dígito antes nem depois** (para não mexer num número maior que contenha esse formato).
- Troca os dígitos dos primeiros 11 caracteres do molde por `*`, **mantendo** os pontos e o `-20` final.
- **Mantém o tamanho do texto** (cada dígito vira um `*`): por isso as posições dos fatos continuam apontando para o lugar certo (teste "mantem o tamanho").
- A máscara é aplicada **na hora de mostrar** (`PaginaDiario`), não nos dados guardados.

Relatorio.java — gravar arquivos

Relatorio.java

Grava arquivos de saída (BufferedWriter + try-with-resources).

pacote diarios.etapas
33 linhas · PDF 1, **pág. 30**

Objetivo. Gravar uma lista de linhas num arquivo de saída. É o par de `Ingestao`: uma lê, a outra grava.

Relatorio.java · linhas 19–32

```
19 public String gravar(String nomeArquivo, ArrayList<String> linhas) throws IOException {
20     Path pasta = Paths.get(pastaSaida);
21     if (!Files.exists(pasta)) {
22         Files.createDirectories(pasta);
23     }
24     Path destino = pasta.resolve(nomeArquivo);
25     try (BufferedWriter escritor = Files.newBufferedWriter(destino, StandardCharsets.UTF_8)) {
26         for (String l : linhas) {
27             escritor.write(l);
28             escritor.newLine();
29         }
30     }
31     return destino.toString();
32 }
```

`Files.createDirectories` cria a pasta se não existir; `Files.newBufferedWriter` dentro de `try-with-resources` fecha o arquivo sozinho; `escritor.newLine()` usa o separador de linha do sistema. Declara `throws IOException`: quem chama decide o que fazer (`Principal.gravarTudo` captura e imprime "falha ao gravar").

Visualizador.java — navegador de arquivos do terminal

Visualizador.java

Navegador de arquivos do menu de terminal (paginação, busca).

pacote diarios.etapas
236 linhas · PDF 1, **pág. 31**

Objetivo. Dá ao menu de terminal (opção 6) um "explorador": lista os arquivos de uma pasta, abre um, e mostra o texto bruto, o texto limpo, um ato inteiro ou as linhas com uma palavra. A parte mais interessante é `paginar`, que mostra 30 linhas por vez e espera Enter. Recebe o `Scanner` pelo construtor (o mesmo que o menu já usa), o que evita ter dois `Scanner` lendo o mesmo `System.in`.

PaginaHtml.java — o HTML do painel

PaginaHtml.java

Monta as seções HTML do painel e o arquivo `painel.html` (concatenação de `String`).

pacote diarios.etapas
508 linhas · PDF 1, **pág. 34**

Objetivo. Montar as **seções HTML** do painel (tiles, interpretação, cobertura, funil, portão, fornecedores, grafo, tabela) concatenando `String` em um `ArrayList<String>`. Serve a **dois usos**: `gerar(...)` monta a página inteira e grava `painel.html` (via `Relatorio`), e os métodos públicos `secao...` são **reaproveitados** por `PainelWeb` para o painel ao vivo.

- `escapar(String)` troca `&`, `<`, `>` e `"` por entidades HTML. **É a defesa contra injeção de HTML**: o texto vem de diários públicos e não pode virar HTML.
- `estilo()` devolve o CSS (cores claras e escuras via variáveis `--ink`, `--serie...`).
- Barras e gráficos são `divs` com `width:N%` calculado em Java (`etapaFunil`, `faixaPortao`, `secaoFornecedores`): nenhum JavaScript.
- `cortar(String, max)` abrevia com `....`

NOTA — Não é preciso decorar o CSS

O estilo é apresentação. Se perguntarem, diga que a classe monta HTML por concatenação, que o navegador desenha, e que `escapar` protege contra injeção.

5. Pacote analise

O que os dados significam: contratos, grafo, leitura e interpretação

Depois que cada ato tem seus fatos e sua nota, `analise/` responde as perguntas do projeto. Nenhuma classe daqui usa IA: tudo é **regra sobre números medidos no texto**.

Assunto.java — de que trata o ato?

Assunto.java

Classifica o assunto de um ato (contratação, pessoal, norma...).

pacote diarios.analise
71 linhas · PDF 1, **pág. 40**

Objetivo. Classificar um ato em um de seis assuntos: `CONTRATACAO`, `ORCAMENTO`, `PESSOAL`, `LICITACAO`, `NORMA` ou `OUTRO`.

Assunto.java · linhas 36–61

```
36 public static String de(Ato ato) {
37     String texto = Extrator.semAcento(ato.getTexto().toLowerCase());
38     String especie = ato.getEspecie();
39
40     boolean temDinheiro = !ato.getFatosDoTipo(Fato.VALOR).isEmpty();
41     boolean temEmpresa = !ato.getFatosDoTipo(Fato.CNPJ).isEmpty();
42
43     if (temEmpresa && temDinheiro) {
44         return CONTRATACAO;
45     }
46     if (contem(texto, MARCAS_ORCAMENTO)) {
47         return ORCAMENTO;
48     }
49     if (contem(texto, MARCAS_PESSOAL)) {
50         return PESSOAL;
51     }
52     if (contem(texto, MARCAS_LICITACAO) || especie.equals("EDITAL")) {
53         return LICITACAO;
54     }
55     if (contem(texto, MARCAS_NORMA)
56         && (especie.equals("DECRETO") || especie.equals("LEI")
57             || especie.equals("LEI COMPLEMENTAR") || especie.equals("PORTARIA"))) {
58         return NORMA;
59     }
60     return OUTRO;
61 }
```

A ordem dos `if` é a regra. O primeiro que bater decide:

1. tem **CNPJ e valor** → `CONTRATACAO` (não depende de palavras);
2. contém marcas de **orçamento** ("credito adicional", "abre credito"...) → `ORCAMENTO`;
3. marcas de **pessoal** ("nomeia", "exonera", "servidor"...) → `PESSOAL`;
4. marcas de **licitação** ("homologa", "pregao"...) **ou** espécie `EDITAL` → `LICITACAO`;
5. marcas de **norma** ("passa a vigorar", "fica revogado", "decreta:"...) **e** espécie `DECRETO`, `LEI`, `LEI COMPLEMENTAR` ou `PORTARIA` → `NORMA`;
6. senão, `OUTRO`.

O texto é comparado em minúsculas e sem acento (`Extrator.semAcento(texto.toLowerCase())`). Em BH: 21 `PESSOAL`, 15 `CONTRATACAO`, 14 `LICITACAO`, 5 `NORMA`, 4 `ORCAMENTO` e **37 `OUTRO`**. Como as marcas são listas de `String[]` e o método só usa `contains`, é fácil acrescentar uma marca nova.

PainelContratos.java — quem foi contratado, por quanto

PainelContratos.java

Monta a lista de contratações, separa multas, faz o ranking por CNPJ.

pacote diarios.analise
328 linhas · PDF 1, **pág. 41**

Objetivo. Transformar fatos (CNPJ + valor) em **contratações** e produzir o ranking de fornecedores. É a classe com a lógica mais "de negócio" do projeto.

PainelContratos.java · linhas 56–87

```
56 public void processar(ArrayList<Diario> diarios) {
57     for (Diario d : diarios) {
58         if (!d.isUtilizavel()) {
59             continue;
60         }
61         for (Ato ato : d.getAto()) {
62             ArrayList<Fato> cnpjs = ato.getFatosDoTipo(Fato.CNPJ);
63             ArrayList<Fato> valores = ato.getFatosDoTipo(Fato.VALOR);
64             if (cnpjs.isEmpty() || valores.isEmpty()) {
65                 continue;
66             }
67             ArrayList<Fato> processos = ato.getFatosDoTipo(Fato.PROCESSO);
68             String processo = processos.isEmpty() ? "-" : processos.get(0).getValor();
69
70             for (Fato cnpj : cnpjs) {
71                 Fato valor = maisProximo(valores, cnpj.getInicio());
72                 String[] papelENome = papelENomeAntesDe(d.getTextoLimpo(), cnpj.getInicio());
73
74                 Contrato c = new Contrato(d.getMunicipio(), d.getData(),
75                     cnpj.getValor(), papelENome[1], papelENome[0],
76                     ExtratorValor.parNumero(valor.getValor()),
77                     processo, ato.getIdentidade());
78
79                 if (papelENome[0].equals(FORNECEDOR) || papelENome[0].equals(INDEFINIDO)) {
80                     contratos.add(c);
81                 } else {
82                     descartados.add(c);
83                 }
84             }
85         }
86     }
87 }
```

Como `processar` **decide o que é contratação:**

1. Só olha atos que tenham **pelo menos um CNPJ e pelo menos um valor**.
2. Para cada CNPJ do ato, escolhe o **valor mais próximo** em posição (`maisProximo`: menor `|v.inicio - cnpj.inicio|`).
3. Descobre o **papel** daquele CNPJ e o **nome** da empresa (`papelENomeAntesDe`).
4. Se o papel for `FORNECEDOR` ou `INDEFINIDO`, vai para `contratos`; se for `LADO_MUNICIPIO` (o órgão que multa) ou `NOTIFICADO` (a empresa multada), vai para `descartados`.

O papel do CNPJ (decisão 12) nasceu de um erro real: o programa somava **multas** como contratos.

PainelContratos.java · linhas 102–120

```
102 private String[] papelENomeAntesDe(String texto, int posicaoCnpj) {
103     int de = Math.max(0, posicaoCnpj - JANELA_PAPEL);
104     String janela = texto.substring(de, posicaoCnpj).replace('\n', ' ');
105     String maiusculo = janela.toUpperCase();
106
107     int ultimaMarca = -1;
108     String papel = INDEFINIDO;
109     for (String[] par : PAPEIS) {
110         int p = maiusculo.lastIndexOf(par[0]);
111         if (p > ultimaMarca) {
112             ultimaMarca = p;
113             papel = par[1];
114         }
115     }
116     if (ultimaMarca < 0) {
117         return new String[] {INDEFINIDO, nomeAntesDe(texto, posicaoCnpj)};
118     }
119     return new String[] {papel, nomeDepoisDaMarca(janela, ultimaMarca)};
120 }
```

- Olha os **320 caracteres antes do CNPJ** (`JANELA_PAPEL`), em maiúsculas.

- Para cada marca da tabela **PAPEIS** (**NOTIFICANTE** e **CONTRATANTE** → lado do município; **NOTIFICADO** e **AUTUADO** → multado; **CONTRATADA**, **DETENTORA**, **VENCEDORA**, **FORNECEDOR**, **EMPRESA...** → fornecedor), usa `lastIndexOf` e **fica com a marca mais próxima do CNPJ** (a de maior índice).
- Se achou marca: o nome da empresa é o texto **logo depois dela** (`nomeDepoisDaMarca`), cortado na primeira vírgula, `,`, `"sito"`, `"inscrit"`, `"CNPJ"` ou travessão, e só aceito se tiver entre 4 e 70 caracteres; senão `"(nao identificada)"`.
- Se **não** achou marca: papel **INDEFINIDO** e tenta o nome **procurando um sufixo de razão social** (**LTD**A, **S/A**, **EIRELI**, **ME**, **EPP**, **COOPERATIVA**) nos 220 caracteres anteriores (`nomeAntesDe`), e recuando até o início do nome; se nada, `nomePorSeparador`, que exige que o nome seja pelo menos 70% maiúsculas.

O caso do teste (`Testes.java`, "papel de cada CNPJ"): um trecho com **NOTIFICADO**: **ACACIA COMERCIO DE MEDICAMENTOS LTDA.**, **CNPJ** **N 03.945.035/0001-91** e **NOTIFICANTE**: **HOSPITAL METROPOLITANO ODILON BEHRENS ... 16.692.121/0001-81** e **Multa** de **R\$ 2.461,28**. **Resultado**: um CNPJ descartado como **LADO_MUNICIPIO**, outro como **NOTIFICADO**, **zero contratos** e **total 0**: a multa não entra.

Ranking e agrupamento por CNPJ (decisão 9):

`PainelContratos.java` · linhas 279–317

```

279 public ArrayList<String[]> ranking() {
280     ArrayList<String> cnpjs = new ArrayList<>();
281     ArrayList<String> nomes = new ArrayList<>();
282     ArrayList<Double> somas = new ArrayList<>();
283     ArrayList<Integer> vezes = new ArrayList<>();
284
285     for (Contrato c : contratos) {
286         int i = cnpjs.indexOf(c.getCnpj());
287         if (i < 0) {
288             cnpjs.add(c.getCnpj());
289             nomes.add(c.getContratada());
290             somas.add(c.getValor());
291             vezes.add(1);
292         } else {
293             somas.set(i, somas.get(i) + c.getValor());
294             vezes.set(i, vezes.get(i) + 1);
295             if (nomes.get(i).equals("(nao identificada)")) {
296                 nomes.set(i, c.getContratada());
297             }
298         }
299     }
300
301     ArrayList<String[]> linhas = new ArrayList<>();
302     boolean[] usado = new boolean[cnpjs.size()];
303     for (int n = 0; n < cnpjs.size(); n++) {
304         int melhor = -1;
305         for (int i = 0; i < cnpjs.size(); i++) {
306             if (!usado[i] && (melhor < 0 || somas.get(i) > somas.get(melhor))) {
307                 melhor = i;
308             }
309         }
310         usado[melhor] = true;
311         linhas.add(new String[] {
312             cnpjs.get(melhor), nomes.get(melhor),
313             Contrato.formatarReal(somas.get(melhor)), String.valueOf(vezes.get(melhor))
314         });
315     }
316     return linhas;
317 }

```

1. Quatro **ArrayList** **paralelas** (`cnpjs`, `nomes`, `somas`, `vezes`): o índice `i` é o mesmo fornecedor nas quatro. `cnpjs.indexOf(c.getCnpj())` localiza o fornecedor; se não existe, acrescenta; se existe, soma o valor e conta mais uma vez. Se o nome guardado era `"(nao identificada)"` e esta ocorrência trouxe um nome melhor, troca.
2. **Ordenação por seleção**: repete "acha o maior ainda não usado (`boolean[] usado`), marca como usado, acrescenta ao resultado".
3. Devolve `ArrayList<String[]>`, cada linha `{cnpj, nome, "R$ ..." formatado, vezes}`.

Complexidade. Agrupar é $O(n \cdot m)$ (`indexOf` em lista) e a seleção é $O(m^2)$, com m = número de fornecedores. Com dezenas é instantâneo. Com milhares, valeria uma estrutura melhor (`HashMap`, que ainda não foi dada).

`repeticoes(minimo)` filtra o ranking pelos que aparecem `minimo` vezes ou mais (usado para "fornecedor que se repete", com 3). `total()` soma os valores. `contarDescartadosDoPapel(papel)` alimenta o aviso "Nem todo CNPJ com valor ao lado é um contrato".

PONTO FRACO — SEJA HONESTO — Um valor e vários CNPJs: o valor é contado várias vezes

Testei: um ato com **dois fornecedores** (CONTRATADA: ALFA... CNPJ ... e CONTRATADA: BETA... CNPJ ...) e **um único** Valor global: R\$ 100.000,00 produz **2 contratos de R\$ 100.000,00**, total **R\$ 200.000,00**. Cada CNPJ pega "o valor mais próximo", e se só existe um, os dois pegam o mesmo. Atas de registro de preços com várias detentoras podem inflar o total. Resposta honesta: "é uma simplificação: associa cada CNPJ ao valor mais próximo; para atos com vários fornecedores e um só valor global, o total pode estar acima do real".

PONTO FRACO — SEJA HONESTO — Nomes de empresa mal extraídos

Na saída real aparecem "fornecedores" como Item 01, Itens 01 a 04 e licitante/lote adjudicado): BEST CONE.. O nome vem de heurística sobre texto livre e, quando falha, o CNPJ continua certo e o agrupamento por CNPJ continua válido. É exatamente por isso que o ranking agrupa por **CNPJ** e não por nome.

P Por que ordenar por seleção e não usar `Collections.sort`?

R Porque o ranking é uma lista de `String[]` montada à mão, e a ordenação por seleção foi o algoritmo visto em aula. Com dezenas de fornecedores o custo $O(n^2)$ não aparece.

P Como você sabe que um CNPJ é fornecedor e não a prefeitura multando?

R Pelo **papel escrito antes do CNPJ**: NOTIFICANTE, CONTRATANTE → lado do município; NOTIFICADO, AUTUADO → multado; CONTRATADA, DETENTORA... → fornecedor. Isso nasceu de um erro real, em que multas eram somadas como contratos.

GrafoNormas.java — quem altera, revoga, prorroga ou só cita

GrafoNormas.java

Grafo de normas: decide ALTERA / REVOGA / PRORROGA / CITA; percorre vizinhança.

pacote diarios.analise
176 linhas · PDF 1, **pág. 45**

Objetivo. Montar um **grafo dirigido** cujos vértices são atos e normas e cujas arestas são `Relacao`: este ato ALTERA / REVOGA / PRORROGA / CITA *aquela norma*. Responde "quais normas mudaram?".

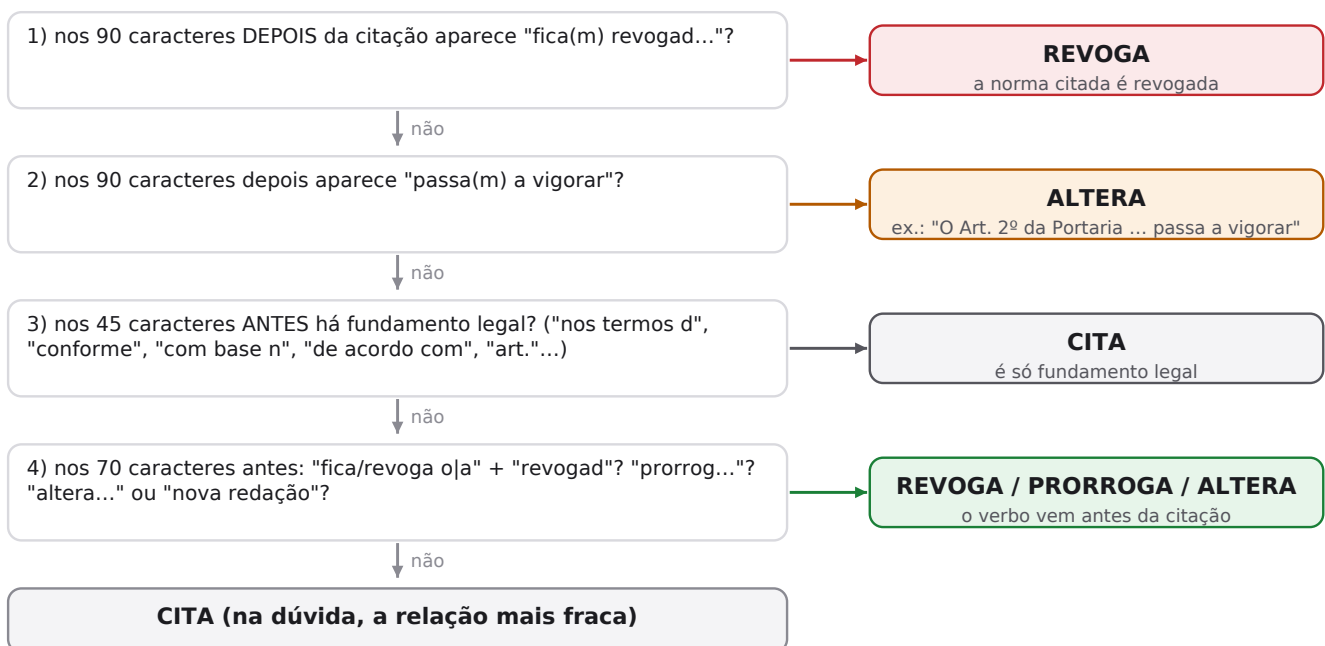
O problema difícil. A palavra "altera" aparece **145 vezes** no diário de BH e "prorrog" **125**, mas quase todas são cláusulas de contrato ("não haverá **alteração** dos preços", "**prorrogação** da vigência da Ata"), não normas. Um classificador que só procura a palavra criaria cerca de **270 relações falsas** num diário com **5** verdadeiras. Por isso o método começa **eliminando**.

GrafoNormas.java · linhas 47–75

```

47 private String classificar(String texto, int inicioCitacao, int fimCitacao) {
48     String antes = trecho(texto, Math.max(0, inicioCitacao - JANELA_ANTES), inicioCitacao);
49     String depois = trecho(texto, fimCitacao,
50         Math.min(texto.length(), fimCitacao + JANELA_DEPOIS));
51
52     String comecoDepois = primeiros(depois, TRECHO_DEPOIS);
53     if (comecoDepois.contains("fica revogad") || comecoDepois.contains("ficam revogad")) {
54         return Relacao.REVOGA;
55     }
56     if (comecoDepois.contains("passa a vigorar") || comecoDepois.contains("passam a vigorar")) {
57         return Relacao.ALTERA;
58     }
59
60     if (ehFundamento(antes)) {
61         return Relacao.CITA;
62     }
63
64     String fim = ultimos(antes, TRECHO_VERBO);
65     if (operou(fim, "revogad", "revoga")) {
66         return Relacao.REVOGA;
67     }
68     if (operou(fim, "prorrogad", "prorroga")) {
69         return Relacao.PRORROGA;
70     }
71     if (operou(fim, "alterad", "altera") || fim.contains("nova redacao")) {
72         return Relacao.ALTERA;
73     }
74     return Relacao.CITA;
75 }

```



A ORDEM importa: em "O Art. 2º da Portaria SMC nº 073/2025 passa a vigorar", o "Art." (passo 3) pareceria fundamento — mas o passo 2 vem antes e acerta ALTERA.

Janelas medidas em caracteres do textoLimpo, em minúsculas e sem acento (Extrator.semAcento).

Passo a passo do classificar (a citação é um fato REFERENCIA com início e fim no texto limpo do diário):

1. Pega o trecho **antes** (até 160 caracteres) e **depois** (até 110) da citação, em minúsculas e sem acento. Dele só usa os **primeiros 90** depois (TRECHO_DEPOIS).
2. Se nos 90 depois aparece **fica revogad/ficam revogad** → **REVOGA**; se aparece **passa a vigorar/passam a vigorar** → **ALTERA**. **Isso vem primeiro.**
3. Senão, se nos **45 caracteres antes** há **fundamento legal** (nos termos d, conforme, com base n, previsto n, de acordo com, na forma d, a que se refere, disposto n, art.) → só **CITA**.
4. Senão olha os **70 caracteres antes** buscando o **verbo** (operou: **fica(m) revogad...**, **revoga o|a**; **prorroga...**; **altera...** ou **nova redacao**) → **REVOGA / PRORROGA / ALTERA**.

5. Se nada bateu → CITA. Na dúvida, a relação mais fraca.

A ordem importa (decisão 7): em 0 Art. 2º da Portaria SMC nº 073/2025 passa a vigorar, o trecho antes começa com "Art.", que é marca de fundamento. Mas o "passa a vigorar" logo depois é evidência **mais forte**, e por isso é testado **primeiro**. Na ordem inversa, a única alteração real do diário viraria apenas uma menção. E em Portaria SMC Nº 074/2025, publicada no DOM em 25 de novembro de 2025, passa a vigorar o verbo fica a ~60 caracteres da citação: com a janela de 60 se perdia; com 90 aparece (decisão 11).

Montagem do grafo:

GrafoNormas.java · linhas 29–45

```
29 public void processar(ArrayList<Diario> diarios) {
30     for (Diario d : diarios) {
31         if (!d.isUtilizavel()) {
32             continue;
33         }
34         for (Ato ato : d.getAto()) {
35             for (Fato ref : ato.getFatosDoTipo(Fato.REFERENCIA)) {
36                 Relacao nova = new Relacao(ato.getIdentidade(),
37                     classificar(d.getTextoLimpo(), ref.getInicio(), ref.getFim()),
38                     ref.getValor(), ref.getInicio());
39                 if (!arestas.contains(nova)) {
40                     arestas.add(nova);
41                 }
42             }
43         }
44     }
45 }
```

Para cada ato de cada diário utilizável e para cada fato REFERENCIA, cria a Relacao e só a acrescenta se !arestas.contains(nova) (usa o equals de Relacao, que ignora a posição): sem duplicatas.

Consultas:

- doTipo(acao) filtra as arestas por ação.
- maisAlteradas() conta, para cada norma-alvo, quantas relações **que não são** CITA a atingem (duas listas paralelas) e ordena por seleção. Resultado real: portaria:074 e portaria:073 com 2 cada; portaria:041, portaria:075, lei:18441 e lei:19021 com 1.
- vizinhanca(norma, niveis): **busca em largura**:

GrafoNormas.java · linhas 150–175

```
150 public ArrayList<String> vizinhanca(String norma, int niveis) {
151     ArrayList<String> visitados = new ArrayList<>();
152     ArrayList<String> fronteira = new ArrayList<>();
153     visitados.add(norma);
154     fronteira.add(norma);
155
156     for (int nivel = 0; nivel < niveis && !fronteira.isEmpty(); nivel++) {
157         ArrayList<String> proxima = new ArrayList<>();
158         for (String atual : fronteira) {
159             for (Relacao r : arestas) {
160                 String outro = null;
161                 if (r.getAlvo().equals(atual)) {
162                     outro = r.getOrigem();
163                 } else if (r.getOrigem().equals(atual)) {
164                     outro = r.getAlvo();
165                 }
166                 if (outro != null && !visitados.contains(outro)) {
167                     visitados.add(outro);
168                     proxima.add(outro);
169                 }
170             }
171         }
172         fronteira = proxima;
173     }
174     return visitados;
175 }
```

- visitados é a lista de nós já vistos, fronteira os nós do nível atual, proxima os do próximo.
- Para cada nó da fronteira percorre as arestas **nos dois sentidos** (se o nó é alvo, o vizinho é a origem; se é origem, o vizinho é o alvo) e acrescenta o que ainda não foi visto.

- Repete por **níveis** níveis. Como as arestas alternam ato ↔ norma, 2 níveis a partir de **portaria:074** chegam nos atos que a alteram e nas normas que esses atos citam. Resultado real: **portaria:074, ...:portaria:016:2026-03-25, ...:lei:13019:2026-03-25, portaria:016, lei:13019.**

P	O que é o grafo, em uma frase?
R	Um conjunto de relações "este ato altera/revoga/prorroga/cita aquela norma", guardado como uma lista de arestas (<code>ArrayList<Relacao></code>), com consultas por ação, ranking das normas mais mexidas e percurso por níveis (busca em largura).
P	Por que não procurar só a palavra "altera"?
R	Porque ela aparece 145 vezes no diário de BH e quase todas são cláusula de contrato. Só pela palavra surgiriam ~270 relações falsas para 5 verdadeiras. O classificador primeiro elimina (fundamento legal, forma não dispositiva) e só depois procura o verbo.
P	Qual a ordem das regras e por que importa?
R	(1) o que vem depois da citação ("passa a vigorar", "fica revogado"); (2) fundamento legal antes ("nos termos do..."); (3) verbo antes. Importa porque em "O Art. 2º da Portaria ... passa a vigorar" o "Art." pareceria fundamento; testando o "passa a vigorar" antes, acerta-se ALTERA.

LeituraDoAto.java — o que este ato faz, em uma frase

LeituraDoAto.java

Escreve, em português, o que cada ato faz.

pacote diarios.analise
156 linhas · PDF 1, **pág. 47**

Objetivo. Escrever em português o que cada ato faz, usando as classificações já feitas. É determinístico: **mesma entrada, mesma frase** (há teste).

LeituraDoAto.java · linhas 33–63

```

33 public String ler(Ato ato, String dataDoDiario) {
34     String texto = Extrator.semAcento(ato.getTexto().toLowerCase());
35     String assunto = Assunto.de(ato);
36     StringBuilder s = new StringBuilder();
37
38     if (assunto.equals(Assunto.CONTRATACAO)) {
39         s.append(lerContratacao(ato));
40     } else if (assunto.equals(Assunto.NORMA)) {
41         s.append(lerNorma(ato));
42     } else if (assunto.equals(Assunto.PESSOAL)) {
43         s.append("Ato de pessoal: ").append(primeiraMarca(texto, PESSOAL, "movimentacao de servid
>> or"))
44         .append(".");
45     } else if (assunto.equals(Assunto.LICITACAO)) {
46         s.append("Ato de licitacao: ").append(primeiraMarca(texto, LICITACAO, "procedimento licit
>> atorio"))
47         .append(".");
48     } else if (assunto.equals(Assunto.ORCAMENTO)) {
49         s.append(lerOrcamento(ato));
50     } else {
51         s.append("Ato sem categoria reconhecida; ").append(ato.getFatos().size())
52         .append(" fato(s) extraido(s).");
53     }
54
55     if (texto.contains("entra em vigor na data de sua publicacao")
56         || texto.contains("entra em vigor na data da sua publicacao")) {
57         s.append(" Vigora desde a publicacao (").append(dataDoDiario).append(").");
58     }
59     if (ato.getDecisao().equals(Ato.DUVIDOSO)) {
60         s.append(" Leitura com ressalva: ").append(ato.getMotivo()).append(".");
61     }
62     return s.toString();
63 }
```

1. `Assunto.de(ato)` escolhe o ramo.
2. `CONTRATACAO` → `lerContratacao`: busca em `painel.getContratos()` os contratos **daquele ato** (comparando `identidadeAto`); se houver 1, "Contrata empresa (CNPJ) por R\$..."; se vários, "N

contratações somando R\$...; a maior com ...". Acrescenta "Processo *número*" se houver. Se não há contrato mas há CNPJ **descartado** (multa), diz "Notificação ou multa envolvendo *N* CNPJ(s); não conta como contratação".

3. NORMA → lerNorma: junta as relações do grafo cuja origem é este ato ("altera portaria:074"), ou diz "Ato normativo que cria regra própria; cita *N* norma(s) como fundamento".
4. PESSOAL / LICITACAO → primeiraMarca procura a primeira marca na tabela (exonera → "exoneracao"...). ORCAMENTO → lerOrcamento ("Ato orçamentário movimentando R\$...").
5. Depois, **sempre**: se o texto contém "entra em vigor na data de sua publicação" acrescenta "Vigora desde a publicação (*data*)"; se a decisão é DUVIDOSO acrescenta "**Leitura com ressalva**: *motivo do portão*".

Exemplos reais:

Contrata AÇÕES PARA A CIDADANIA (13.630.795/0001-63) por R\$ 700.000,00. Processo 31.00204095/2026-45.
 Ato normativo que cria regra propria; cita 2 norma(s) como fundamento. Vigora desde a publicacao (2026-03-25).
 Ato de pessoal: exoneraçao. Leitura com ressalva: poucos fatos extraídos (1).
 Ato orcamentario movimentando R\$ 89.223,82.

Note que o construtor recebe PainelContratos e GrafoNormas: a classe **depende** do resultado das duas análises (por isso Aplicacao.leitor() cria new LeituraDoAto(painel, grafo) com os objetos atuais).

Interpretacao.java — "o que esses dados dizem"

Interpretacao.java

Gera as frases do "o que esses dados dizem" a partir de regras e limites.

pacote diarios.analise
413 linhas · PDF 1, **pág. 49**

Objetivo. Gerar as frases do bloco "O que esses dados dizem": um **resumo** e uma lista de Achado, em três níveis: LEITURA (descreve), ATENCAO ("onde vale a pena olhar") e LIMITE ("limites desta leitura": o que o programa não sabe).

Interpretacao.java · linhas 27–40

```
27 public void interpretar(ArrayList<Diario> diarios, Cobertura cobertura,
28                       PainelContratos painel, GrafoNormas grafo) {
29     limpar();
30     montarResumo(diarios, painel, grafo);
31     perfilDoDiario(diarios);
32     concentracaoDeGasto(painel);
33     contratoDominante(painel);
34     fornecedorRepetido(painel);
35     rastreabilidade(painel);
36     papeisSeparados(painel);
37     normasDesatualizadas(grafo);
38     buracosDeCobertura(cobertura);
39     oQueNaoFoiEntendido(diarios, cobertura);
40 }
```

Cada método abaixo olha números medidos e, **se uma regra for satisfeita**, acrescenta um Achado. Os limiares são constantes no topo da classe.

método	regra	nível
perfilDoDiario	% de atos de gasto (contratação + licitação) × % de atos de norma: se gasto > 3 × norma, "diário de EXECUÇÃO"	LEITURA
concentracaoDeGasto	os 3 maiores fornecedores somam ≥ 60% (CONCENTRACAO_ALTA) do total → "CONCENTRADO", senão "DISTRIBUÍDO" (precisa de ≥ 3 fornecedores)	ATENÇÃO ou LEITURA
contratoDominante	um só contrato ≥ 30% do total (UM_SO_CONTRATO_ALTO)	ATENÇÃO
fornecedorRepetido	fornecedor com ≥ 3 contratos (REPETICAO_MINIMA); texto avisa que é contagem, não acusação	ATENÇÃO
rastreabilidade	≥ 30% dos contratos sem processo (SEM_PROCESSO_ALTO)	LIMITE
papeisSeparados	há CNPJs descartados por papel (multa/órgão): explica por que o total não soma tudo	LIMITE

método	regra	nível
normasDesatualizadas	há normas alteradas/revogadas/prorrogadas (lista as 5 primeiras)	LEITURA
buracosDeCobertura	há diários recusados: diz que a leitura vale só para o resto	LIMITE
oQueNaoFoiEntendido	≥ 25% (DUVIDA_ALTA) dos atos em dúvida; agrupa os motivos	LIMITE

montarResumo produz a frase de abertura ("Em 2026-03-11 a 2026-09-29, 2 municípios publicou 129 atos. Em 17 deles ha 25 contratacoes, somando R\$ 9.373.590,71..."), usando periodoDe (menor e maior data) e os totais.

P	"Nenhuma frase foi escrita à mão": é verdade?
R	Em parte. As frases-modelo (o texto fixo de cada achado) são escritas à mão no código. O que nunca é escrito à mão é qual achado aparece e quais números entram nele : isso sai de regras sobre valores medidos (porcentagens, contagens, limiares). Mesma pasta, mesmas frases. Se o professor apertar, diga isso.
P	Por que existem os níveis LIMITE?
R	Porque o projeto quer ser honesto sobre o que não sabe: PDFs que não leu, atos em dúvida, contratos sem processo, CNPJs que não são contrato. Um relatório que só mostra o que deu certo esconde o tamanho do erro.

Cobertura.java — o que foi lido e o que não foi

Cobertura.java

Relatório do que foi lido, do que foi recusado e dos totais do portão.

pacote diarios.analise
78 linhas · PDF 1, **pág. 54**

Objetivo. Registrar, diário a diário, **o que foi lido, o que foi recusado e os totais do portão**. Tem contadores (lidos, utilizaveis, recusados, totalAtos, aceitos, duvidosos, rejeitados) e uma lista de linhas de texto.

- registrarRecusa(arquivo, motivo, evidencia) conta o diário como lido e recusado e guarda três linhas (nome, motivo, evidência).
- registrarProcessado(Diario) soma os atos e as decisões (d.contar(...)) e guarda as linhas do funil ("limpeza: 16338 linhas → 5478 removidas", "fragmentação: 225 títulos candidatos, 129 descartados (sem corpo), 96 atos", "portão: ACEITO 66 · DUVIDOSO 30 · REJEITADO 0").
- montar() devolve as linhas do **Relatório de cobertura**. Os getters alimentam Interpretacao e o painel.

É o que torna visível a diferença entre "**0 atos**" e "**não consegui ler**".

6. Pacote coleta

De onde vem o dado: API, JSON à mão, fontes e rodadas

O professor ensinou até a Aula 05. A coleta usa **duas peças do JDK que não foram dadas** (`java.net.http.HttpClient` e, no servidor, `com.sun.net.httpserver.HttpServer`) porque sem elas não haveria como buscar o dado real nem mostrar o painel. O restante (ler o JSON, montar a URL, guardar o estado) é feito à mão com `indexOf`, `substring`, `String` e `ArrayList`.

DICA — A resposta pronta para "por que usou isso se não foi dado?"

"Sem `HttpClient` não haveria como buscar o dado real, e sem `HttpServer` não haveria o painel. Usamos só o que já vem no JDK, sem biblioteca externa. Todo o resto é `String`, `ArrayList` e arquivos."

Aplicacao.coletar(dias)

desde = hoje - dias · ate = hoje · alvo = fontes.ligadas()

para cada fonte ligada

estado.jaCobre(desde, ate)? → sim: "pulada" (a não ser que "refazer todas")

Coletor.coletar(municipio, desde, ate)

GET `api.../gazettes?territory_ids=...&published_since=...&size=50`

consultarApi → buscarTexto

HTTP 200? devolve o JSON. 429 / ≥500 / IOException → tenta de novo (espera 5 s, 15 s, 45 s)

Json.objetos(resposta, "gazettes")

quebra a lista em objetos {...} à mão; `Json.campo(obj, "txt_url")` lê cada campo

baixarEdicao (para cada edição)

sem `txt_url` → "só PDF" · arquivo já existe → "já tinha" · senão baixa e grava .txt

estado.registrarResultado(...)

COM_DADO · SEM_EDICAO · SO_PDF · FALHOU (3 falhas seguidas interrompem a rodada)

fontes.salvar() → processarPasta() → registrarRodada()

grava fontes.txt, relê a pasta inteira, acrescenta uma linha em rodadas.txt

Coletor.java — falar com a API e baixar os textos

Coletor.java

Fala com a API do Querido Diário (`HttpClient`), baixa os textos, tenta de novo.

pacote `diarios.coleta`
216 linhas · PDF 1, **pág. 55**

Objetivo. Consultar a API pública do **Querido Diário** (<https://api.queridodiario.org.br/gazettes>), descobrir quais edições existem para um município em um período e **baixar o texto** de cada uma para `dados/diarios/`.

Cuidados que estão no código (educação com um servidor público e gratuito):

- **pausa entre requisições:** `PAUSA_MS = 1500` (1,5 s);
- `User-Agent` **identificando o trabalho** ("`ProjetoFaculdade/1.0 (trabalho academico)`"), em vez de fingir ser navegador;
- **diário que já está na pasta não é baixado de novo** (`Files.exists(destino)`);
- **tenta de novo com espera crescente** quando a API recusa por excesso de pedidos (5 s, 15 s, 45 s).

Coletor.java · linhas 43–69

```

43 public ResultadoColeta coletar(Municipio municipio, String desde, String ate) {
44     ResultadoColeta r = new ResultadoColeta(municipio);
45     String url = "?territory_ids=" + municipio.getCodigo()
46         + "&published_since=" + desde + "&published_until=" + ate
47         + "&size=" + LIMITE;
48
49     String resposta;
50     try {
51         resposta = consultarApi(url, "consulta");
52     } catch (ColetaException e) {
53         r.marcarFalhaDeRede(e.getMessage());
54         r.anoar(municipio.getNome() + ": " + e.getMessage());
55         return r;
56     }
57
58     ArrayList<String> gazetas = Json.objetos(resposta, "gazettes");
59     if (gazetas.isEmpty()) {
60         r.anoar(municipio.getNome() + ": nenhuma edicao de " + desde + " a " + ate);
61         return r;
62     }
63
64     for (String gazeta : gazetas) {
65         r.contarEdicao();
66         baixarEdicao(municipio, gazeta, r);
67     }
68     return r;
69 }

```

1. Monta os parâmetros: `territory_ids` (código IBGE do município), `published_since`, `published_until`, `size=50`.
2. `consultarApi(...)`. Se falhar (`ColetaException`), marca `r.marcarFalhaDeRede` e **devolve o resultado**, sem lançar: a rodada continua com as outras fontes.
3. `Json.objetos(resposta, "gazettes")` separa as edições. Lista vazia → anota "nenhuma edição".
4. Para cada edição, `r.contarEdicao()` e `baixarEdicao`.

Coletor.java · linhas 97–128

```

97 private void baixarEdicao(Municipio municipio, String gazeta, ResultadoColeta r) {
98     String data = Json.campo(gazeta, "date");
99     String urlTexto = Json.campo(gazeta, "txt_url");
100     String nome = municipio.getNome() + " " + data;
101
102     if (urlTexto.isEmpty()) {
103         r.contarSemTexto();
104         r.anoar(nome + ": so PDF, sem texto");
105         return;
106     }
107
108     Path destino = Paths.get(pastaDestino, municipio.getApelido() + "_" + data
109         + sufixoDeEdicao(gazeta) + ".txt");
110     if (Files.exists(destino)) {
111         r.contarJaExistia();
112         r.anoar(nome + ": ja estava na pasta");
113         return;
114     }
115
116     try {
117         String texto = buscarTexto(urlTexto, "download de " + data);
118         Files.createDirectories(destino.getParent());
119         Files.writeString(destino, texto, StandardCharsets.UTF_8);
120         r.contarBaixado();
121         r.anoar(nome + ": " + (texto.length() / 1024) + " KB -> " + destino.getFileName());
122     } catch (IOException e) {
123         r.anoar(nome + ": falha ao gravar - " + e.getMessage());
124     } catch (ColetaException e) {
125         r.anoar(nome + ": " + e.getMessage());
126     }
127     esperar();
128 }

```

- Lê `date` e `txt_url` da edição (`Json.campo`).
- **Sem** `txt_url`: a edição só existe em PDF (sem texto) → `contarSemTexto` ("só PDF, sem texto").
- O nome do arquivo é `apelido_data.txt` (belo-horizonte_2026-03-25.txt), com o sufixo `-extra116` quando é **edição extra** (`sufixoDeEdicao`): é por isso que existe `recife_2026-09-29-extra116.txt` ao lado de `recife_2026-09-29.txt`.

- Arquivo já existe → `contarJaExistia`. Senão baixa (`buscarTexto`), cria a pasta (`Files.createDirectories`), grava (`Files.writeString`) e conta `contarBaixado`.
- Qualquer `IOException` ou `ColetaException` vira uma linha de registro; **nunca derruba a coleta**.

O mecanismo de nova tentativa:

Coletor.java · linhas 139–157

```
139 private String consultarApi(String parametros, String oQue) throws ColetaException {
140     String url = API + parametros;
141     for (int tentativa = 0; ; tentativa++) {
142         try {
143             String resposta = buscarTexto(url, oQue);
144             esperar();
145             return resposta;
146         } catch (ColetaException e) {
147             boolean acabaram = tentativa >= ESPERAS_ENTRE_TENTATIVAS_MS.length;
148             if (!e.podeTentarDeNovo() || acabaram) {
149                 throw new ColetaException(url, e.getMessage()
150                     + (tentativa > 0 ? " (depois de " + (tentativa + 1) + " tentativas)" : ""
151 >> ),
152                     e.podeTentarDeNovo(), e.getEsperaSugeridaMs());
153             long espera = Math.max(ESPERAS_ENTRE_TENTATIVAS_MS[tentativa], e.getEsperaSugeridaMs(
154 >> ));
155             esperar(Math.min(espera, ESPERA_MAXIMA_MS));
156         }
157     }
```

- Um `for` infinito (`for (int tentativa = 0; ; tentativa++)`) que só sai por `return` (deu certo) ou `throw` (desistiu).
- Se `buscarTexto` lançar `ColetaException`: `acabaram = tentativa >= ESPERAS.length (3)`. Se **não vale tentar de novo** (`!e.podeTentarDeNovo()`) **ou** as tentativas acabaram, lança uma exceção nova, acrescentando "(depois de N tentativas)".
- Senão espera `max(ESPERAS[tentativa], esperaSugerida)`, limitada a 60 s (`ESPERA_MAXIMA_MS`), e tenta de novo. Total: **4 tentativas** (a primeira + 3 esperas: 5 s, 15 s, 45 s).
- Exemplo **real**, gravado em `dados/estado/fontes.txt`: Fortaleza ficou **FALHOU** com "*consulta respondeu HTTP 503 [no available server] (depois de 4 tentativas)*".

Coletor.java · linhas 159–187

```
159 private String buscarTexto(String url, String oQue) throws ColetaException {
160     HttpRequest pedido = HttpRequest.newBuilder()
161         .uri(URI.create(url))
162         .header("User-Agent", AGENTE)
163         .header("Accept", "application/json, text/plain, /*/*")
164         .timeout(Duration.ofSeconds(90))
165         .GET()
166         .build();
167     try {
168         HttpResponse<String> resposta =
169             cliente.send(pedido, HttpResponse.BodyHandlers.ofString(StandardCharsets.UTF_8));
170         int status = resposta.statusCode();
171         if (status != 200) {
172             boolean passageiro = status == 429 || status >= 500;
173             long sugerida = resposta.headers().firstValue("Retry-After")
174                 .map(this::segundosParaMs).orElse(0L);
175             throw new ColetaException(url, oQue + " respondeu HTTP " + status
176                 + (status == 429 ? " (limite de pedidos da API)" : "")
177                 + " [" + trechoDoCorpo(resposta.body()) + "]", passageiro, sugerida);
178         }
179         return resposta.body();
180     } catch (IOException e) {
181         throw new ColetaException(url, oQue + " falhou: " + e.getClass().getSimpleName()
182             + " " + e.getMessage(), true, 0);
183     } catch (InterruptedException e) {
184         Thread.currentThread().interrupt();
185         throw new ColetaException(url, oQue + " foi interrompido");
186     }
187 }
```

- Monta um `HttpRequest` GET com os cabeçalhos e timeout de 90 s; `cliente.send(...)` com `BodyHandlers.ofString(UTF_8)`.

- Se o status **não for 200**: é "passageiro" (vale tentar de novo) quando for **429** (limite de pedidos) ou $\geq$ **500**; lê o cabeçalho `Retry-After` para sugerir a espera (aqui aparece `Optional.map(this::segundosParaMs)`: fora da aula). Lança `ColetaException` com um trecho do corpo (`trechoDoCorpo`, que tira tags HTML com `replaceAll`).
- `IOException` (rede) → `ColetaException` com "vale tentar de novo = true".
- `InterruptedException` → **restaura a flag de interrupção** (`Thread.currentThread().interrupt()`) e lança sem nova tentativa.

`descobrir(desde, ate)`: em vez de adivinhar quais municípios têm texto, pergunta à API quem publicou nos últimos dias. Página até **3 páginas de 100** edições; guarda cada `territory_id` **uma vez** (`codigos.contains`) e **só se a edição tem** `txt_url`; devolve `ArrayList<Municipio>`. Para quando uma página vem com menos de 100.

Proxy. O construtor, se não houver `https.proxyHost`, liga `java.net.useSystemProxies`, e o `HttpClient` usa `ProxySelector.getDefault()` (para funcionar em redes de faculdade/empresa).

P	E se a API estiver fora do ar?
R	<code>buscarTexto</code> lança <code>ColetaException</code> (503, timeout...); <code>consultarApi</code> tenta até 4 vezes com 5 s, 15 s e 45 s de espera; se continuar falhando, <code>Coletor.coletar</code> marca a fonte como FALHOU com o motivo e devolve, e a rodada segue para a próxima fonte. Em <code>Aplicacao.coletar</code> , depois de 3 fontes seguidas com falha, a rodada para ("interrompida depois de 3 falhas seguidas") para não martelar um servidor que caiu.
P	Como o JSON é lido sem biblioteca?
R	Por <code>Json.java</code> , à mão: objetos percorre a lista caractere a caractere contando chaves <code>{}</code> e ignorando o que está dentro de aspas; <code>campo</code> procura "chave", pula o <code>:</code> e lê o valor, tratando <code>\n</code> e <code>\uXXXX</code> .

Json.java — leitor de JSON feito à mão

Json.java

Leitor de JSON feito à mão (`indexOf` / `substring`).

pacote `diarios.coleta`
109 linhas · PDF 1, **pág. 58**

Objetivo. Extrair, de uma resposta JSON da API, **uma lista de objetos** e **o valor de um campo**, usando só `String`.

Json.java · linhas 7–52

```
7 public static ArrayList<String> objetos(String resposta, String lista) {
8     ArrayList<String> partes = new ArrayList<>();
9     String marca = "\"" + lista + "\"";
10    int p = resposta.indexOf(marca);
11    if (p < 0) {
12        return partes;
13    }
14    int abre = resposta.indexOf('[', p);
15    if (abre < 0) {
16        return partes;
17    }
18
19    int profundidade = 0;
20    int inicio = -1;
21    boolean dentroDeTexto = false;
22
23    for (int i = abre; i < resposta.length(); i++) {
24        char c = resposta.charAt(i);
25
26        if (dentroDeTexto) {
27            if (c == '\\') {
28                i++;
29            } else if (c == '"') {
30                dentroDeTexto = false;
31            }
32            continue;
33        }
34        if (c == '"') {
35            dentroDeTexto = true;
36        } else if (c == '{') {
37            if (profundidade == 0) {
38                inicio = i;
39            }
40            profundidade++;
41        } else if (c == '}') {
42            profundidade--;
43            if (profundidade == 0 && inicio >= 0) {
44                partes.add(resposta.substring(inicio, i + 1));
45                inicio = -1;
46            }
47        } else if (c == ']' && profundidade == 0) {
48            break;
49        }
50    }
51    return partes;
52 }
```

`objetos(resposta, "gazettes")` acha "gazettes", depois o [, e varre caractere a caractere com três pedaços de estado: `profundidade` (sobe em {, desce em }), `inicio` (onde o objeto de nível 0 começou) e `dentroDeTexto` (para ignorar chaves e colchetes dentro de aspas; um \ pula o caractere seguinte, que é "escapado"). Quando a profundidade volta a 0, o trecho {...} é um objeto completo e vai para a lista. Um] com profundidade 0 encerra.

Json.java · linhas 54–99

```

54 public static String campo(String objeto, String chave) {
55     String marca = "\"" + chave + "\"";
56     int p = objeto.indexOf(marca);
57     if (p < 0) {
58         return "";
59     }
60     int i = objeto.indexOf(':', p + marca.length());
61     if (i < 0) {
62         return "";
63     }
64     i++;
65     while (i < objeto.length() && objeto.charAt(i) == ' ') {
66         i++;
67     }
68     if (i >= objeto.length()) {
69         return "";
70     }
71     if (objeto.charAt(i) != '"') {
72         return valorSemAspas(objeto, i);
73     }
74     i++;
75
76     StringBuilder valor = new StringBuilder();
77     while (i < objeto.length()) {
78         char c = objeto.charAt(i);
79         if (c == '\\' && i + 1 < objeto.length()) {
80             char seguinte = objeto.charAt(i + 1);
81             if (seguinte == 'n') {
82                 valor.append('\n');
83             } else if (seguinte == 'u' && i + 5 < objeto.length()) {
84                 valor.append((char) Integer.parseInt(objeto.substring(i + 2, i + 6), 16));
85                 i += 4;
86             } else {
87                 valor.append(seguinte);
88             }
89             i += 2;
90             continue;
91         }
92         if (c == '"') {
93             break;
94         }
95         valor.append(c);
96         i++;
97     }
98     return valor.toString();
99 }

```

`campo(objeto, "date")` acha "date", vai ao :, pula espaços. Se o valor **não começa com aspas** (número, true, null), `valorSemAspas` lê até ,, } ou], e null vira "". Se começa com aspas, lê até a aspa de fechamento, convertendo \n em quebra de linha, \uXXXX no caractere correspondente (`Integer.parseInt(hex, 16)`) e \", \\ nos próprios caracteres.

Os testes cobrem: separar duas edições; ler data e URL; null vira ""; chave ausente vira ""; Maceió escrito como Maceiô; número e booleano sem aspas.

NOTA — Limitação aceitável

`campo` procura a **primeira** ocorrência de "chave" dentro do objeto, o que funciona para os objetos planos desta API. Num JSON com objetos aninhados ou texto contendo a chave, poderia pegar o campo errado. Um leitor completo seria uma biblioteca; aqui o escopo é pequeno e conhecido.

Municipio.java e Fonte.java — o catálogo

Municipio.java

Código IBGE, nome e UF; gera o "apelido" usado nos nomes de arquivo.

pacote diarios.coleta
44 linhas · PDF 1, **pág. 60**

Municipio guarda `codigo` (código IBGE de 7 dígitos), `nome` e `uf`. O método interessante é `getApelido()`: o nome em minúsculas, espaço → -, só letras e dígitos, **sem acento**. Gera os nomes de arquivo: São Paulo → `sao-paulo`; Belo Horizonte → `belo-horizonte`. Para remover acento usa uma tabela local ("`àáãäåêëíóôõùç`" → "`aaaaeeiooouc`") em vez de `Extrator.semAcento`, o que é uma pequena duplicação.

Fonte.java

Catálogo das 27 capitais e do conjunto padrão de 5 fontes.

pacote diarios.coleta
72 linhas · PDF 1, **pág. 61**

Fonte é o **catálogo das 27 capitais** (matriz `String[][]` com código, nome e UF). `porNome(busca)` aceita **começo do nome, começo do apelido** ou **o código exato** ("`belo`", "`2611606`"). `padrao()` devolve as **5 fontes ligadas na primeira vez**: Belo Horizonte, Recife, Fortaleza, Curitiba e Aracaju.

EstadoFonte.java — a situação de uma fonte

EstadoFonte.java

Estado de uma fonte: ligada?, status da última tentativa, período coberto.

pacote diarios.coleta
86 linhas · PDF 1, **pág. 62**

Objetivo. Guardar, para cada município, se está **ligado**, o **status** da última tentativa e **qual período já foi coberto**.

status	significa
NUNCA_TENTADA	ainda não foi coletada
COM_DADO	baixou diário com texto, ou já tinha na pasta
SEM_EDICAO	a API respondeu, mas não houve edição no período
SO_PDF	houve edição, mas só em PDF escaneado (sem texto)
FALHOU	a rede ou a API não respondeu (o motivo é guardado)

EstadoFonte.java · linhas 27–46

```

27 public void registrarResultado(String quando, ResultadoColeta r, String desde, String ate) {
28     ultimaTentativa = quando;
29     motivo = "";
30     coberturaDesde = "";
31     coberturaAte = "";
32     if (r.isFalhouRede()) {
33         status = FALHOU;
34         motivo = r.getMotivoFalha();
35         return;
36     }
37     coberturaDesde = desde;
38     coberturaAte = ate;
39     if (r.getBaixados() > 0 || r.getJaExistiam() > 0) {
40         status = COM_DADO;
41     } else if (r.getSemTexto() > 0) {
42         status = SO_PDF;
43     } else {
44         status = SEM_EDICAO;
45     }
46 }

```

`registrarResultado` traduz o `ResultadoColeta`: falha de rede → `FALHOU` (e **limpa a cobertura**); senão grava o período coberto e escolhe `COM_DADO` (baixou ou já existia), `SO_PDF` (houve edição sem texto) ou `SEM_EDICAO`.

EstadoFonte.java · linhas 48–53

```

48 public boolean jaCobre(String desde, String ate) {
49     boolean resolvida = status.equals(COM_DADO) || status.equals(SEM_EDICAO)
50         || status.equals(SO_PDF);
51     return resolvida && !coberturaDesde.isEmpty()
52         && coberturaDesde.compareTo(desde) <= 0 && coberturaAte.compareTo(ate) >= 0;
53 }

```

jaCobre(desde, ate) é o que evita refazer trabalho: só vale se o status for "resolvido" (COM_DADO, SEM_EDICAO ou SO_PDF) e o período pedido estiver **dentro** do já coberto (coberturaDesde <= desde e coberturaAte >= ate). As datas são String no formato aaaa-mm-dd, que **se comparam em ordem correta com** compareTo. Se falhou ou nunca foi tentada, **não pula**. Há 10 testes só para essas regras.

Fontes.java — a lista de fontes que lembra

Fontes.java

Lista de fontes + persistência em dados/estado/fontes.txt.

pacote diarios.coleta
156 linhas · PDF 1, **pág. 63**

Objetivo. Manter a lista de EstadoFonte e **lembrar entre execuções**, gravando em dados/estado/fontes.txt.

Fontes.java · linhas 91–106

```
91 public synchronized void salvar() {
92     ArrayList<String> linhas = new ArrayList<>();
93     for (EstadoFonte e : estados) {
94         Municipio m = e.getMunicipio();
95         linhas.add(String.join(";", m.getCodigo(), limpo(m.getNome()), m.getUf(),
96             String.valueOf(e.isLigada()), e.getStatus(), limpo(e.getUltimaTentativa()),
97             limpo(e.getMotivo()), String.valueOf(e.isDescoberta()),
98             e.getCoberturaDesde(), e.getCoberturaAte()));
99     }
100     try {
101         Files.createDirectories(arquivo.getParent());
102         Files.write(arquivo, linhas, StandardCharsets.UTF_8);
103     } catch (IOException ex) {
104         System.out.println("aviso: nao consegui salvar " + arquivo + ": " + ex.getMessage());
105     }
106 }
```

- **Formato:** uma linha por fonte, 10 campos separados por ;:
codigo;nome;uf;ligada;status;ultimaTentativa;motivo;descoberta;coberturaDesde;coberturaAte.
Exemplo real: 2304400;Fortaleza;CE>true;FALHOU;2026-09-30 18:53:44;consulta respondeu HTTP 503 [no available server] (depois de 4 tentativas);false;;.
- limpo(s) troca ; por , e quebras de linha por espaço, para que um texto livre (o motivo da falha) não **quebre o formato**.
- carregar() lê as linhas, divide com split(";", -1) (o -1 **mantém campos vazios** no fim) e restaura cada EstadoFonte; linhas com menos de 8 campos são ignoradas, e municípios que não são capitais entram como "descoberta".
- No construtor: se **não há arquivo**, liga só as 5 do padrao(); se há, carrega o estado salvo.
- **Todos os métodos públicos são** synchronized, porque a thread da coleta e a do servidor mexem na mesma lista.
- adicionarDescobertas acrescenta os municípios novos **desligados** e conta só os novos; atualizarAcervo conta, para cada fonte, os arquivos apelido_*.txt na pasta.

ResultadoColeta.java e Rodada.java — o histórico

ResultadoColeta.java

Contadores e registro do que aconteceu com UMA fonte numa rodada.

pacote diarios.coleta
42 linhas · PDF 1, **pág. 65**

ResultadoColeta é um **saco de contadores** para **uma fonte numa rodada** (baixados, semTexto, jaExistiam, edicoes, falhouRede, pulada) mais o registro (lista de linhas de texto: <município> <data>: <N> KB -> <arquivo>.txt). Quase só getters e métodos contar...

Rodada.java

Uma coleta (ou descoberta) inteira; grava/relê uma linha em rodadas.txt.

pacote diarios.coleta
126 linhas · PDF 1, **pág. 66**

Rodada representa **uma coleta (ou descoberta) inteira**: início, tipo, dias, período, a lista de ResultadoColeta, fim e observação. O que importa:

- `paraLinha()` transforma a rodada em **uma linha de 12 campos** para `dados/estado/rodadas.txt` (`inicio;tipo;dias;desde;ate;fim;fontes;baixados;jaExistiam;falhas;observacao;puladas`). Exemplo real: `2026-09-30 18:52:38;coleta;1;2026-09-29;2026-09-30;2026-09-30 18:59:18;5;2;0;3;;0` (5 fontes, 2 baixados, 3 falhas).
- `daLinha(linha)` faz o caminho inverso (um método `static` que funciona como "construtor alternativo"): ao reabrir o programa, o histórico volta **sem** ter os `ResultadoColeta` originais. Por isso guarda `resumoSalvo` (um vetor de 4 inteiros): os métodos `fontes()`, `baixados()`, `jaExistiam()` e `falhas()` devolvem o resumo salvo **se existir**, senão calculam somando os resultados em memória.
- Há teste "histórico gravado e relido" e "ponto e vírgula da observação não quebra a linha".

P Por que guardar o estado em arquivos de texto e não num banco de dados?

R Banco não foi dado em aula, e o arquivo de texto **pode ser aberto e conferido por qualquer pessoa**. O custo é ter que cuidar do formato à mão (por isso `limpo()` e o `split(";", -1)`).

7. Servidor, Aplicacao, Principal e Testes

A parte "fora da aula", o orquestrador e o menu

Aplicacao.java (continuação) — coleta, threads e estado

Aplicacao.java

O "cérebro": processarPasta(), coletar(), descobrir() e a coleta em segundo plano.

pacote diarios
306 linhas · PDF 1, **pág. 81**

Já vimos `processarPasta()`. O resto da classe cuida de **coletar, descobrir, rodar em segundo plano e guardar o histórico**.

Atributos importantes:

- Constantes de pasta: `PASTA_DIARIOS = "dados/diarios"`, `PASTA_SAIDA`, `PASTA_ESTADO` (caminhos **relativos** à pasta de trabalho do programa).
- Resultados da última leitura: `diarios`, `cobertura`, `painel`, `grafo`, `interpretacao`, `falhasDeLeitura`, `ultimoProcessamento`.
- Histórico: `rodadas` (lista) e `ultimaRodada`.
- **Estado compartilhado entre threads**: `private volatile boolean ocupado` e `private volatile String progresso`.

Aplicacao.java · linhas 117–163

```

117 public Rodada coletar(int dias, boolean refazerTudo) {
118     int n = Math.max(1, Math.min(dias, DIAS_MAXIMO));
119     LocalDate hoje = LocalDate.now();
120     String desde = hoje.minusDays(n).toString();
121     String ate = hoje.toString();
122
123     ArrayList<EstadoFonte> alvo = fontes.ligadas();
124     Rodada rodada = new Rodada(agora(), "coleta", n, desde, ate);
125
126     int i = 0;
127     int falhasSeguidas = 0;
128     String interrompida = "";
129     for (EstadoFonte estado : alvo) {
130         i++;
131         if (!refazerTudo && estado.jaCobre(desde, ate)) {
132             ResultadoColeta pulada = new ResultadoColeta(estado.getMunicipio());
133             pulada.marcarPulada();
134             pulada.anotar(estado.getMunicipio().getNome() + ": pulada, ja coletada ("
135                 + estado.getStatus() + " de " + estado.getCoberturaDesde() + " a "
136                 + estado.getCoberturaAte() + ")");
137             rodada.adicionar(pulada);
138             continue;
139         }
140         progresso = "coletando " + estado.getMunicipio().getNome()
141             + " (" + i + " de " + alvo.size() + ")";
142         ResultadoColeta r = coletor.coletar(estado.getMunicipio(), desde, ate);
143         estado.registrarResultado(agora(), r, desde, ate);
144         rodada.adicionar(r);
145
146         falhasSeguidas = r.isFalhouRede() ? falhasSeguidas + 1 : 0;
147         if (falhasSeguidas >= FALHAS_SEGUIDAS_PARA_PARAR && i < alvo.size()) {
148             interrompida = "interrompida depois de " + FALHAS_SEGUIDAS_PARA_PARAR
149                 + " falhas seguidas; " + (alvo.size() - i)
150                 + " fonte(s) ficaram para a proxima rodada";
151             break;
152         }
153     }
154     fontes.salvar();
155
156     progresso = "lendo e interpretando os diarios da pasta";
157     processarPasta();
158
159     String obs = alvo.isEmpty() ? "nenhuma fonte ligada" : interrompida;
160     rodada.finalizar(agora(), obs);
161     registrarRodada(rodada);
162     return rodada;
163 }

```

coletar(dias, refazerTudo), **passo a passo:**

1. `n = Math.max(1, Math.min(dias, DIAS_MAXIMO))`: limita os dias a **1-90**. `desde = hoje - n`, `ate = hoje` (como String `aaaa-mm-dd`, via `java.time.LocalDate`).
2. `alvo = fontes.ligadas()`. Para cada fonte: se `!refazerTudo` e `estado.jaCobre(desde, ate)`, cria um `ResultadoColeta` marcado **pulada** e segue (não gasta pedidos à API).
3. Senão atualiza progresso ("coletando Recife (2 de 5)"), chama `coletor.coletar(...)`, `estado.registrarResultado(...)` e `rodada.adicionar(r)`.
4. **Freio**: `falhasSeguidas` conta falhas de rede consecutivas; ao chegar a 3 (`FALHAS_SEGUIDAS_PARA_PARAR`) e ainda faltando fontes, **interrompe** a rodada e registra "interrompida depois de 3 falhas seguidas; N fonte(s) ficaram para a próxima rodada".
5. `fontes.salvar()`, depois `processarPasta()` (lê tudo de novo, incluindo o que acabou de ser baixado), `rodada.finalizar(...)` e `registrarRodada(rodada)`.

`descobrir()` pergunta ao `Coletor` quem publicou nos últimos 3 dias (`DIAS_DESCOBERTA`) e adiciona os municípios novos em `Fontes` (desligados). Se der `ColetaException`, a rodada termina com "falhou: ...".

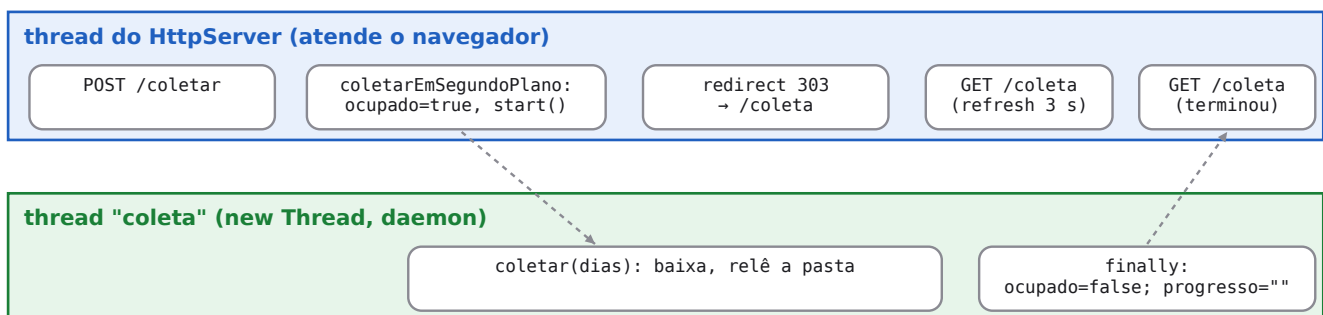
A thread de coleta (conteúdo fora da aula)

Aplicacao.java · linhas 189–205

```

189 public synchronized boolean coletarEmSegundoPlano(int dias, boolean refazerTudo) {
190     if (ocupado) {
191         return false;
192     }
193     ocupado = true;
194     Thread t = new Thread(() -> {
195         try {
196             coletar(dias, refazerTudo);
197         } finally {
198             ocupado = false;
199             progresso = "";
200         }
201     }, "coleta");
202     t.setDaemon(true);
203     t.start();
204     return true;
205 }

```



`volatile ocupado / progresso`: a outra thread vê o valor novo na hora. `synchronized`: troca atômica do resultado (processarPasta monta tudo em variáveis locais e só atribui dentro de `synchronized (this)`).

Por que uma Thread? Coletar demora (várias requisições com pausa de 1,5 s). Se o `HttpServer` fizesse isso dentro do tratador do clique, o navegador ficaria congelado até terminar. Então o método **inicia uma thread separada e devolve na hora**; a página recarrega a cada 3 s (`<meta http-equiv="refresh">`) e mostra progresso.

Peça por peça:

peça	o que é	por que está aqui
<code>new Thread(() -> { ... }, "coleta")</code>	cria uma linha de execução; o <code>() -> { ... }</code> é uma lambda : um jeito curto de passar um bloco de código (equivale a uma classe que implementa <code>Runnable</code>)	o corpo roda em paralelo ao servidor
<code>t.setDaemon(true)</code>	thread "de serviço": não impede o programa de encerrar	se o usuário fecha o painel, a coleta não segura o processo
<code>t.start()</code>	começa a executar o corpo da lambda	
<code>try { coletar(...) } finally { ocupado = false; progresso = ""; }</code>	o <code>finally</code> sempre roda	mesmo com erro, <code>ocupado</code> volta a <code>false</code> e o painel não fica "travado"
<code>synchronized</code> no método	só uma thread entra por vez no trecho <code>if (ocupado) ... ocupado = true</code>	garante uma coleta por vez : o 2º clique recebe <code>false</code> ("já existe uma coleta em andamento")
<code>volatile boolean ocupado</code>	toda leitura vê o valor mais recente escrito por qualquer thread	o servidor lê <code>isOcupado()</code> sem <code>synchronized</code> e precisa enxergar a mudança feita pela thread de coleta

Em uma frase para a banca: "A coleta roda numa linha de execução separada para a página não travar; duas linhas mexem nos mesmos dados, então `synchronized` deixa uma de cada vez e `volatile`

garante que a outra enxergue o valor atualizado. Entendo o motivo, mas não domino o detalhe."

P	Qual o papel do <code>synchronized (this)</code> no fim de <code>processarPasta</code>?
R	<code>processarPasta</code> monta tudo em variáveis locais (a leitura demora) e só no fim, dentro de um bloco <code>synchronized</code> , troca os campos de uma vez. Os getters (<code>getDiarios()</code> , <code>getCobertura()</code> ...) também são <code>synchronized</code> . Assim a página do painel nunca vê "diários novos com cobertura antiga".
P	O que é <code>this::tratarPainel e () ->?</code>
R	<code>this::tratarPainel</code> é uma referência a um método : diz ao servidor "chame este método do próprio objeto quando chegar um pedido neste endereço". <code>() -> { ... }</code> é uma lambda : um bloco de código sem nome passado como valor. As duas dão a uma interface de um só método (<code>HttpHandler</code> , <code>Runnable</code>) a implementação sem escrever uma classe.

Histórico e persistência: `registrarRodada (synchronized)` acrescenta `rodada.paraLinha()` ao arquivo `rodadas.txt` com `StandardOpenOption.CREATE, APPEND`; `carregarRodadas()` relê no começo (`Rodada.daLinha`). Se o arquivo estiver ilegível, o programa só avisa e começa vazio, sem cair.

Servidor.java — o `HttpServer` do JDK

Servidor.java

Sobe o `HttpServer` do JDK em localhost e liga cada endereço a um método.

pacote `diarios.servidor`
224 linhas · PDF 1, **pág. 68**

Objetivo. Subir um servidor web **local** que devolve páginas HTML geradas pelo programa. É a **classe principal** do `pom.xml`.

`Servidor.java` · linhas 70–102

```

70 public boolean iniciar(int portaDesejada) {
71     for (int p = portaDesejada; p < portaDesejada + TENTATIVAS_DE_PORTA; p++) {
72         try {
73             http = HttpServer.create(new InetSocketAddress("localhost", p), 0);
74             porta = p;
75             break;
76         } catch (BindException e) {
77             System.out.println("porta " + p + " ocupada, tentando a proxima...");
78         } catch (IOException e) {
79             System.out.println("nao consegui abrir o servidor: " + e.getMessage());
80             return false;
81         }
82     }
83     if (http == null) {
84         System.out.println("nenhuma porta livre entre " + portaDesejada + " e "
85             + (portaDesejada + TENTATIVAS_DE_PORTA - 1));
86         return false;
87     }
88
89     http.createContext("/", this::tratarPainel);
90     http.createContext("/coleta", this::tratarPaginaColeta);
91     http.createContext("/coletar", this::tratarColetar);
92     http.createContext("/descobrir", this::tratarDescobrir);
93     http.createContext("/processar", this::tratarProcessar);
94     http.createContext("/fonte/alternar", this::tratarAlternar);
95     http.createContext("/fonte/todas", this::tratarTodas);
96     http.createContext("/diario", this::tratarDiario);
97     http.createContext("/ato", this::tratarAto);
98     http.createContext("/busca", this::tratarBusca);
99     http.setExecutor(null);
100    http.start();
101    return true;
102 }
```

- `HttpServer.create(new InetSocketAddress("localhost", p), 0)` abre o servidor **só em** localhost (ninguém de fora acessa). Se a porta estiver ocupada (`BindException`), tenta a seguinte, até 10 portas (8080 a 8089).
- `http.createContext("/diario", this::tratarDiario)` liga **um endereço a um método**. Todo tratador tem a forma `void tratar...(HttpExchange troca) throws IOException`.

- `http.setExecutor(null)` usa o executor padrão do JDK; `http.start()` põe no ar.

endereço	método	o que faz
GET /	<code>tratarPainel</code>	painel principal (qualquer outro caminho → página 404)
GET /coleta	<code>tratarPaginaColeta</code>	"Coleta e fontes"
POST /coletar	<code>tratarColetar</code>	lê dias e refazer, dispara a coleta em segundo plano
POST /descobrir	<code>tratarDescobrir</code>	dispara a descoberta
POST /processar	<code>tratarProcessar</code>	"Reler a pasta" (só se não estiver ocupado)
POST /fonte/alternar	<code>tratarAlternar</code>	liga/desliga uma fonte (codigo)
POST /fonte/todas	<code>tratarTodas</code>	liga/desliga todas
GET /diario?arquivo=...	<code>tratarDiario</code>	página de um diário
GET /ato?arquivo=...&indice=...	<code>tratarAto</code>	página de um ato, com a prova de cada fato
GET /busca?q=...	<code>tratarBusca</code>	busca por palavra

Padrão POST → redirecionar → GET. Depois de um POST (clique em botão), o tratador **redireciona com status 303** (redirecionar) para uma página GET. Assim, se o usuário apertar F5, o navegador **não reenvia** o formulário (e não dispara a coleta de novo). `responder(...)` converte o HTML em `byte[]` UTF-8, define `Content-Type` e `Cache-Control: no-store`, e escreve no `OutputStream` dentro de um `try-with-resources`.

`main` lê as opções (`--sem-coleta`, `--sem-navegador`, `--porta=`), cria a `Aplicacao`, **lê a pasta**, sobe o servidor, abre o navegador (`java.awt.Desktop`, com plano B de só imprimir a URL) e, se não for `--sem-coleta`, dispara a coleta de 1 dia em segundo plano.

P O que é essa parte de servidor? Posso ignorá-la?

R É só **apresentação**: devolve o resultado no navegador. O trabalho do projeto está em `etapas/` e `extratores/`. O `HttpServer` vem no JDK; recebe o pedido do navegador, chama um método nosso e devolve uma página HTML. Sem JavaScript: cada botão é um formulário comum, o Java recebe o clique, faz a ação e devolve a página de novo.

Formulario.java — ler parâmetros de URL e de formulário

Formulario.java

Lê parâmetros de formulário/URL (`split`, `URLDecoder`).

pacote `diarios.servidor`
48 linhas · PDF 1, **pág. 71**

Objetivo. Ler `dias=7&refazer=sim` (corpo de um POST) ou `?arquivo=x&indice=3` (URL de um GET).

Formulario.java · linhas 9–23

```

9 public static String parametro(String dados, String chave) {
10     if (dados == null || dados.isEmpty()) {
11         return "";
12     }
13     for (String par : dados.split("&")) {
14         int igual = par.indexOf('=');
15         if (igual < 0) {
16             continue;
17         }
18         if (decodificar(par.substring(0, igual)).equals(chave)) {
19             return decodificar(par.substring(igual + 1)).trim();
20         }
21     }
22     return "";
23 }

```

- `dados.split("&")` separa os pares; `indexOf('=')` separa chave e valor; as duas partes passam por `URLDecoder` (+ vira espaço, `%C3%A3` vira `ã`); devolve o valor se a chave for a procurada, ou `""`.
- `inteiro(dados, chave, padrao)` converte e, se vazio ou inválido (`NumberFormatException`), devolve o `padrao`: **entrada ruim nunca derruba o servidor**.

- `codificar` usa `URLEncoder` para pôr um nome de arquivo dentro de um link.

PainelWeb.java — as páginas "Painel" e "Coleta e fontes"

PainelWeb.java

Páginas "Painel" e "Coleta e fontes" (HTML montado em `ArrayList<String>`).

pacote `diarios.servidor`
421 linhas · PDF 1, **pág. 72**

Objetivo. Montar as duas páginas principais como `ArrayList<String>` de linhas de HTML, juntadas com `String.join("\n", h)`.

- `montar(aviso)`: cabeçalho com 4 números ("tiles"), barra de estado, interpretação, **por município**, fornecedores, grafo, lista de diários, funil, portão e a lista de contratações. Muitas seções vêm de `PaginaHtml` (reaproveitadas).
- `montarColeta(aviso)`: formulários de coleta, histórico de rodadas e tabelas de fontes (ligadas e desligadas), com botão por linha.
- `abrir(titulo)` **acrescenta** `<meta http-equiv="refresh" content="3">` **só se** `app.isOcupado()`: é como a página se atualiza sozinha durante a coleta, sem JavaScript.
- Botões ficam `disabled` enquanto há coleta em andamento.
- A página é **montada a cada requisição**, a partir dos objetos atuais de `Aplicacao`: por isso reflete sempre o estado mais recente.

PaginaDiario.java — diário, ato e busca

PaginaDiario.java

Páginas de um diário, de um ato (com prova de cada fato) e a busca.

pacote `diarios.servidor`
281 linhas · PDF 1, **pág. 77**

Três páginas:

- `diario(arquivo)`: tabela dos atos (decisão, assunto, título, **leitura**) e, recolhidos, os textos bruto e limpo. Se o diário foi recusado, mostra o motivo e o texto bruto.
- `ato(arquivo, indice)`: a página mais importante para **demonstrar** o projeto: título, "o que este ato faz" (`LeituraDoAto`), ficha (identidade, assunto, decisão e motivo, posição), **cada fato com sua prova** (`f.provar(...)`) e o **texto do ato com os fatos marcados**.
- `busca(termo)`: procura em todos os diários utilizáveis (minúsculas e sem acento), até 200 ocorrências, e **liga cada uma ao ato** que contém aquela posição (`atoNaPosicao`).

PaginaDiario.java · linhas 171–194

```

171 private String destacar(Ato ato) {
172     String texto = ato.getTexto();
173     ArrayList<int[]> trechos = new ArrayList<>();
174     for (Fato f : ato.getFatos()) {
175         trechos.add(new int[] {f.getInicio() - ato.getPosicao(), f.getFim() - ato.getPosicao()});
176     }
177     ordenar(trechos);
178
179     StringBuilder s = new StringBuilder();
180     int cursor = 0;
181     for (int[] t : trechos) {
182         int ini = Math.max(0, Math.min(t[0], texto.length()));
183         int fim = Math.max(ini, Math.min(t[1], texto.length()));
184         if (ini < cursor) {
185             continue;
186         }
187         s.append(esc(Privacidade.mascararCpf(texto.substring(cursor, ini))));
188         s.append("<mark>").append(esc(Privacidade.mascararCpf(texto.substring(ini, fim))))
189           .append("</mark>");
190         cursor = fim;
191     }
192     s.append(esc(Privacidade.mascararCpf(texto.substring(cursor))));
193     return s.toString();
194 }
```

`destacar` é a parte algorítmica: coloca `<mark>` em volta de cada fato. Converte cada fato para posição **relativa ao ato** (`f.getInicio() - ato.getPosicao()`), **ordena** os trechos por início (`ordenar` é uma **ordenação por inserção** feita à mão sobre `ArrayList<int[]>`), e percorre o texto copiando o trecho "normal", depois o trecho marcado. Trechos que se **sobrepoem** a um já marcado são pulados (`if (ini < cursor)` continue). Tudo passa por `esc(...)` (contra injeção de HTML) e `Privacidade.mascararCpf(...)` (**CPF mascarado**).

Principal.java — o menu de terminal

Principal.java

Menu de terminal (Scanner + switch); chama a Aplicacao.

pacote diarios

486 linhas · PDF 1, **pág. 85**

Objetivo. A "porta" de terminal: um menu com `Scanner` e `switch` que chama a `Aplicacao`.

Principal.java · linhas 39–89

```
39 private void menu() {
40     Scanner teclado = new Scanner(System.in);
41     boolean rodando = true;
42
43     while (rodando) {
44         System.out.println();
45         System.out.println("=====");
46         System.out.println("  DIARIOS OFICIAIS - coleta, leitura e analise");
47         System.out.println("=====");
48         System.out.println("  1 - Coletar das fontes ligadas (escolher dias);");
49         System.out.println("  2 - Fontes: ver, ligar e desligar");
50         System.out.println("  3 - Descobrir municipios com publicacao recente");
51         System.out.println("  4 - Reler a pasta de diarios");
52         System.out.println("  5 - INTERPRETACAO: o que esses dados dizem");
53         System.out.println("  6 - Ver os arquivos (entrada e saida)");
54         System.out.println("  7 - Painel de contratacoes");
55         System.out.println("  8 - Grafo de normas");
56         System.out.println("  9 - Relatorio de cobertura");
57         System.out.println(" 10 - Consultar uma norma no grafo");
58         System.out.println(" 11 - Conferir um fato no texto original");
59         System.out.println(" 12 - Gravar relatorios em saida");
60         System.out.println(" 13 - Abrir o painel no navegador (localhost)");
61         System.out.println("  0 - Sair");
62         System.out.print("  escolha: ");
63
64         String escolha = teclado.hasNextLine() ? teclado.nextLine().trim() : "0";
65
66         switch (escolha) {
67             case "1": coletar(teclado); break;
68             case "2": gerenciarFontes(teclado); break;
69             case "3": descobrir(); break;
70             case "4": reler(); break;
71             case "5": mostrarInterpretacao(); break;
72             case "6": verArquivos(teclado); break;
73             case "7": mostrarPainel(); break;
74             case "8": mostrarGrafo(); break;
75             case "9": mostrarCobertura(); break;
76             case "10": consultarNorma(teclado); break;
77             case "11": conferirFato(teclado); break;
78             case "12": gravarTudo(); break;
79             case "13": abrirPainel(); break;
80             case "0": rodando = false; break;
81             default: System.out.println(" opcao invalida.");
82         }
83     }
84     if (servidor != null) {
85         servidor.parar();
86     }
87     System.out.println("  ate mais.");
88     teclado.close();
89 }
```

opção	método	o que faz
1	<code>coletar</code>	pergunta quantos dias e coleta das fontes ligadas
2	<code>gerenciarFontes</code>	lista, liga/desliga uma, t liga todas, n desliga todas
3	<code>descobrir</code>	descobre municípios com publicação recente

opção	método	o que faz
4	reler	processarPasta() e imprime o resumo por diário
5	mostrarInterpretacao	resumo e achados por nível
6	verArquivos	abre o Visualizador em diarios/ ou saida/
7	mostrarPainel	tabela de contratações (top 15)
8	mostrarGrafo	contagens por ação e normas mais mexidas
9	mostrarCobertura	relatório de cobertura
10	consultarNorma	quem mexeu numa norma + vizinhança em 2 níveis
11	conferirFato	escolhe um ato e mostra cada fato com a prova no texto
12	gravarTudo	grava interpretacao.txt, cobertura.txt, contratos.txt, grafo.txt, atos.txt e painel.html em dados/saida/
13	abrirPainel	sobe o servidor (porta 8080) enquanto o menu estiver aberto
0	—	sai (e para o servidor, se houver)

Detalhes de Java:

- `teclado.hasNextLine() ? teclado.nextLine().trim() : "0"`: lê com segurança (se a entrada acabar, vira "0" e o menu sai em vez de dar exceção).
- `switch` (escolha) sobre `String` com `case "1": ... break;` e `default`.
- `lerInteiro(teclado, padrao)`: converte e, se for inválido, avisa e usa o padrão (`try/catch` `NumberFormatException`).
- `quebrarTexto(texto, largura)`: quebra frase em linhas de até 72 caracteres, palavra por palavra, com `StringBuilder`.
- `main` com `--tudo` imprime interpretação, cobertura, painel e grafo, grava os arquivos e termina.

DICA — A demonstração de 1 minuto (opção 11)

No menu, escolha **11**, digite o número de um ato com contratação (por exemplo o **21** de BH, "Extrato do Termo de Patrocínio") e mostre: **a leitura** ("Contrata AÇÕES PARA A CIDADANIA... por R\$ 700.000,00"), **a identidade**, **a decisão com o motivo** e, para cada fato, **a posição** [`inicio..fim`] **e o trecho original**. É a prova de que o programa não inventa nada, e usa herança, `ArrayList`, `provar()` e o portão de uma vez. (A lista mostra só os 15 primeiros atos; o 21 você digita mesmo assim.)

Testes.java — como sei que funciona

Testes.java

85 verificações com um mini-framework próprio (sem JUnit).

pacote diarios
430 linhas · PDF 1, **pág. 91**

Objetivo. Verificar o comportamento sem JUnit: o **mini-framework** é feito à mão.

Testes.java · linhas 412–415

```
412 private static void conferir(String nome, Object esperado, Object obtido) {
413     marcar(esperado.equals(obtido), nome,
414         "esperado <" + esperado + ">, obtido <" + obtido + ">");
415 }
```

- `conferir(nome, esperado, obtido)` compara com `esperado.equals(obtido)` (valores primitivos entram como objetos por *autoboxing*) e imprime `ok` ou `FALHOU` com o `esperado` e o `obtido`; `conferirMaiorQue` verifica um mínimo; ambos chamam `marcar`, que atualiza os contadores `passou/falhou`.
- `main` roda cada grupo e, no fim, imprime `passou: 85` `falhou: 0` e **sai com código 1** se algo falhou (útil para automação).

grupo	testes	o que verifica
formatação de real	1	R\$ 8.323.312,07 mesmo com <code>Locale pt-BR</code>

grupo	testes	o que verifica
ExtratorValor	5	acha valor, converte, ignora R\$ 18, posição correta
ExtratorCnpj	3	acha o CNPJ; não confunde com CPF
ExtratorReferencia	4	lei:11065, sigla de órgão, grafias diferentes → mesma norma
Ingestao	2	município e data pelo nome do arquivo
Fonte	7	27 capitais, busca, apelido sem acento, padrão de 5
Json	9	edições, campos, null, ó, número e booleano sem aspas
papéis de CNPJ	4	multa não vira contrato (trecho real de BH)
Fontes ligar e lembrar	14	liga/desliga, descoberta, grava e relê
Fontes pular/coletar	10	regras de jaCobre
Formulario	5	parâmetros, decodificação, inteiro inválido
Privacidade	3	mascara, mantém tamanho, não mexe em CNPJ
Rodada	4	grava e relê o histórico
pipeline completo	14	os diários reais : recusa o escaneado, acha atos, fatos, identidades únicas, spans válidos, leitura determinística

O teste do pipeline (testarPipelineCompleto) é o mais forte: roda Ingestão → Limpeza → Fragmentação → Extração → Portão sobre dados/diarios, e confere, entre outras coisas, que **todo ato tem decisão e motivo**, que **nenhuma identidade se repete** no mesmo diário e que **todo span (inicio..fim) cabe no texto**.

NOTA — O que os testes não cobrem

Não há teste para Fragmentacao isolada (só pelo pipeline), para GrafoNormas.classificar com casos isolados, nem para Portao isoladamente. O defeito de "RESOLUÇÃO" (capítulo "Pontos de atenção") escapa exatamente por isso. Se perguntarem "como sabe que acerta?", responda: 85 testes sobre diários reais **mais** a página do ato, que mostra cada fato com o trecho original; e reconheça que a cobertura de testes não é total.

NOTA — O teste do pipeline depende de dados em dados/diarios

Ele lê a pasta real. Hoje há 4 diários (Aracaju é o escaneado, **recusa esperada = 1**) e ok.get(0) é o primeiro diário bom em ordem alfabética (BH). Se alguém colocar na pasta outro diário escaneado ou um município que venha antes de "belo-horizonte", alguns números esperados mudariam.

Arquivos de configuração e dados

pom.xml

Configuração do Maven: Java 17, nenhuma dependência, classe principal = Servidor.

raiz
43 linhas · PDF 1, **pág. 4**

- pom.xml: formato do **Maven** que o IntelliJ entende. Declara Java 17 (maven.compiler.release), UTF-8, **nenhuma dependência**, e a classe principal diarios.servidor.Servidor para o jar (maven-jar-plugin) e para exec. Se perguntarem: "formato que o IntelliJ entende; o projeto não tem dependência nenhuma".
- ABRIR_PAINEL.bat: usa o Java portátil de _sistema\ e o diarios.jar já compilado; chcp 65001 e -Dfile.encoding=UTF-8 para os acentos aparecerem certos no Windows.
- dados/diarios/*.txt: a **entrada** (um .txt por edição). dados/estado/fontes.txt e rodadas.txt: o **estado** (gerado). dados/saida/: relatórios gravados pelo menu (gerado).

8. Mapa de conceitos de Java

Onde cada conceito de aula aparece no código

Se o professor disser "**mostre onde você usou X**", abra a linha indicada. Todas as linhas foram calculadas automaticamente a partir do código, então batem com o PDF 1.

Orientação a objetos

conceito	onde	o que mostrar
Classe, objeto, construtor	Fato.java:16	construtor recebe os 4 valores e atribui com <code>this</code> .
Encapsulamento (<code>private final</code> + getters)	Fato.java:11 e Fato.java:23	ninguém altera o fato depois de criado; só lê
Herança (<code>extends</code>)	ExtratorCnpj.java:6	a filha herda <code>criar</code> , <code>casaMolde</code> , <code>semAcento</code>
Classe abstrata	Extrator.java:6	não pode ser instanciada
Método abstrato	Extrator.java:18	contrato que toda filha implementa
<code>super(...)</code>	ExtratorCnpj.java:11	a filha informa o tipo ao construtor da mãe
<code>@Override</code>	ExtratorCnpj.java:14	a filha sobrescreve <code>extrair</code>
Polimorfismo	Extracao.java:16 e Extracao.java:28	a lista guarda 5 classes diferentes como <code>Extrator</code> ; o laço chama <code>e.extrair(...)</code> sem saber qual é
Sobrecarga de construtor	ColetaException.java:10 e PaginaHtml.java:19	dois construtores, um chama o outro com <code>this(...)</code>
<code>equals</code> e <code>hashCode</code>	Relacao.java:28-37 e Relacao.java:40	comparar conteúdo, não referência (<code>instanceof</code> + <code>cast</code>)
<code>toString</code>	Fato.java:52	formato legível de um objeto
Membros static	Extrator.java:50 e Contrato.java:36	funções que não dependem de um objeto
Constantes <code>static final</code>	Portao.java:10 e Fato.java:5	parâmetros nomeados; os "tipos" são constantes <code>String</code> (não <code>enum</code>)
Composição (um objeto contém outros)	Diario.java:18 e Ato.java:19	Diario → Ato → Fato

Exceções

conceito	onde
Exceção própria (<code>extends Exception</code>)	DiarioIlegivelException.java:3
<code>throws</code> na assinatura	Limpeza.java:17
<code>throw new ...</code>	Limpeza.java:43
<code>try / catch</code>	Aplicacao.java:82
<code>catch</code> de várias exceções	Coletor.java:52 (junto com <code>catch (IOException e)</code> na mesma operação)
<code>finally</code>	Aplicacao.java:197
<code>try-with-resources</code>	Ingestao.java:25 e Relatorio.java:25
tratar <code>NumberFormatException</code>	Formulario.java:32

Texto, listas e arquivos

conceito	onde
<code>ArrayList<T></code> e <code>for-each</code>	<code>Ato.java:54-62</code>
<code>String: charAt, indexOf, substring</code>	<code>Extrator.java:42</code> e <code>Json.java:10</code>
<code>split, trim, startsWith, contains</code>	<code>Limpeza.java:18</code>
<code>toLowerCase + semAcento</code>	<code>ExtratorReferencia.java:21</code>
<code>StringBuilder</code>	<code>Portao.java:50</code> e <code>Contrato.java:42</code>
<code>String.format</code>	<code>Contrato.java:56</code>
Vetores e matrizes	<code>ExtratorData.java:8</code> , <code>PainelContratos.java:31</code> , <code>PainelContratos.java:302</code>
Ler arquivo	<code>Ingestao.java:40</code>
Gravar arquivo	<code>Relatorio.java:25</code> e <code>Aplicacao.java:258</code>
Listar pasta	<code>Ingestao.java:25</code>
<code>Scanner</code> e <code>switch</code>	<code>Principal.java:40</code> e <code>Principal.java:66</code>

Algoritmos

algoritmo	onde	custo
Ordenação por seleção	<code>PainelContratos.java:302</code> (ranking) e <code>GrafoNormas.java:136</code> (maisAlteradas)	$O(m^2)$
Ordenação por inserção	<code>PaginaDiario.java:196-206</code>	$O(n^2)$
Busca em largura (percurso em grafo)	<code>GrafoNormas.java:150-175</code>	$O(\text{níveis} \times \text{fronteira} \times \text{arestas})$
Busca linear por posição	<code>PaginaDiario.java:208-216</code>	$O(\text{atos})$
Agrupar por chave com <code>indexOf</code>	<code>PainelContratos.java:279-317</code>	$O(n \cdot m)$
Varredura de molde (sem regex)	<code>Extrator.java:36-48</code> e <code>ExtratorCnpj.java:15-24</code>	$O(\text{texto} \times \text{molde})$
Evitar duplicata com <code>contains</code>	<code>GrafoNormas.java:39</code>	$O(n)$ por inserção, $O(n^2)$ total

Para a pergunta "qual a complexidade?": todas são **$O(n^2)$ ou menos sobre dezenas ou centenas de itens** (96 atos, ~20 fornecedores, ~200 relações), então o tempo não aparece. Com milhares de itens valeria trocar por uma estrutura de busca rápida (`HashMap`), que ainda não foi dada.

Fora do conteúdo de aula (saiba a resposta curta)

o quê	onde	resposta curta e honesta
<code>Thread</code>	<code>Aplicacao.java:194</code>	"A coleta demora, então roda numa linha de execução separada para a página não travar."
<code>synchronized</code>	<code>Aplicacao.java:189</code>	"Duas linhas mexem nos mesmos dados; deixa uma de cada vez. Entendo o motivo, não domino o detalhe."
<code>volatile</code>	<code>Aplicacao.java:59</code>	"Garante que a outra linha enxergue o valor atualizado."
<code>lambda () -></code>	<code>Aplicacao.java:194</code>	"Jeito curto de passar um bloco de código; equivale a uma classe que implementa <code>Runnable</code> ."
referência de método <code>::</code>	<code>Servidor.java:89</code> e <code>Coletor.java:174</code>	"Diz qual método chamar, sem escrever uma classe."

o quê	onde	resposta curta e honesta
HttpClient	Coletor.java:36	"Vem no JDK; faz o GET na API."
HttpServer	Servidor.java:73	"Vem no JDK; recebe o pedido do navegador e devolve a página."
Optional	Coletor.java:174	"Lê o cabeçalho Retry-After, se existir; senão 0."
Locale.ROOT	Contrato.java:37	"No Windows em português o String.format usa vírgula; foi erro real."
java.time	Aplicacao.java:119	"Calcula as datas desde e ate da coleta."
expressão regular (replaceAll)	Coletor.java:132	"Só no Coletor (limpar nome de edição e tags de erro); os extratores não usam regex."
Maven (pom.xml)	pom.xml	"Formato que o IntelliJ entende; não tem dependência nenhuma."

9. Pontos de atenção

O que encontrei ao ler e executar o código (com evidência)

Esta é a parte que **ninguém te contou**. Para cada ponto: o que acontece, como reproduzi, o impacto e a **resposta honesta** para dizer se o professor encontrar a mesma coisa. Nenhuma linha do código foi alterada por mim; os experimentos foram feitos em **cópias** do projeto.

ATENÇÃO — Como usar este capítulo

Admitir uma limitação com precisão **soa como domínio do código**. Não admitir, e ser pego, soa como o contrário. Se perguntarem, diga o que o código faz, por que é assim e qual seria o próximo passo.

#	ponto	gravidade	onde
1	Espécies com acento ("RESOLUÇÃO", "RETIFICAÇÃO"...) não são reconhecidas	defeito	Fragmentacao.especieDaLinha
2	107 dos 129 títulos "sem corpo" de BH têm corpo e foram descartados por linhas curtas	limitação importante	Fragmentacao.temCorpo
3	Valor com 2 CNPJs é contado duas vezes	limitação	PainelContratos.processar
4	"R\$ 7,02 milhões" é lido como R\$ 7,02	limitação	ExtratorValor
5	O ramo REJEITADO do portão nunca é alcançado	observação	Portao
6	O ano é descartado na norma padronizada; o título cita a si mesmo	limitação	ExtratorReferencia
7	Nomes de empresa às vezes saem errados	limitação	PainelContratos
8	Datas de rodapé e CNPJ mascarado	limitação	ExtratorData, ExtratorCnpj
9	Pequenas imperfeições de texto e de código	cosmético	Interpretacao e outros
10	Documentação diverge do código em alguns números	documentação	README

1. Espécies com acento nunca são reconhecidas (defeito)

O código (Fragmentacao.java:94-109):

Fragmentacao.java · linhas 94–96

```

94 private String especieDaLinha(String linha) {
95     String s = Extrator.semAcento(linha.toUpperCase());
96     for (String e : ESPECIES) {

```

`linha.toUpperCase()` vem **antes** de `Extrator.semAcento`, e `semAcento` só troca letras **minúsculas** (ç ã á...). "RESOLUÇÃO" fica "RESOLUÇÃO" (o ç e o ã maiúsculos **não são trocados**) e nunca casa com a espécie "RESOLUCAO" da lista. O mesmo vale para INSTRUCAO NORMATIVA, ORDEM DE SERVICO, DELIBERACAO e RETIFICACAO.

Reprodução (num diário pequeno de teste, com dois títulos idênticos, um sem e outro com acento): RESOLUCAO Nº 10/2026 virou ato; RESOLUÇÃO Nº 11/2026 **não**.

No dado real: BH tem linhas como RESOLUÇÃO CMAS-BH Nº 028/2026 e várias RETIFICAÇÃO/Retificação (8 linhas assim no diário de BH, 3 no de Recife). Um exemplo concreto: o ato "Extrato do Termo de Patrocínio" **engole** uma RETIFICAÇÃO DO EXTRATO que deveria ser outro ato (aparece no meio do texto dele).

A correção seria de uma linha: `Extrator.semAcento(linha.toLowerCase()).toUpperCase()`. **Testei numa cópia, e não é neutra:**

	antes	depois da correção
BH: candidatos / atos	225 / 96	233 / 101
BH: ACEITO / DUVIDOSO	66 / 30	71 / 30
BH: contratações / valor	19 / R\$ 8.323.312,07	15 / R\$ 8.322.732,07
4 diários: atos	129	137
testes	85 passam	85 passam

Os números mudam porque os novos títulos **cortam** os atos vizinhos (inclusive separando, em alguns casos, o CNPJ do valor), e a contagem de contratações **cai** de 19 para 15. Por isso eu **não** apliquei a correção: ela muda todos os números que aparecem no README e que você vai apresentar.

DICA — O que fazer com isto até quarta

Opção 1 (mais seguro para a demonstração): não mexer, e conhecer o defeito para responder com naturalidade se aparecer. **Opção 2:** corrigir e recalcular todos os números que você vai citar (BH: 101 atos, 71/30/0, 15 contratações) e conferir o README. Se você escolher a opção 2, me peça e eu aplico e atualizo os números.

P O professor pergunta: "e se o título for RESOLUÇÃO?"

R Resposta honesta: "Aqui há um defeito: eu converto para maiúsculas antes de tirar o acento, e a função `semAcento` só trata letras minúsculas, então espécies acentuadas como RESOLUÇÃO não são reconhecidas. A correção é inverter a ordem (`semAcento(linha.toLowerCase()).toUpperCase()`). Eu medi o efeito: BH passa de 96 para 101 atos e as contratações caem de 19 para 15, porque os títulos novos cortam atos vizinhos. Deixei como está para não mudar os números sem revisar."

ATENÇÃO — Isto afeta o Exercício C (acrescentar "CONVENIO")

Testei: com "CONVENIO" na lista, o título CONVENIO Nº 10/2026 **vira ato**, mas CONVÊNIO Nº 11/2026 (com acento) **não**. Se o professor pedir esse exercício e testar com um texto acentuado, vai parecer que "não funcionou". Teste sempre **sem acento** na demonstração, ou corrija a ordem primeiro.

2. Os "129 títulos sem corpo" de BH: a maioria tem corpo

O README e o funil dizem que 129 dos 225 candidatos foram descartados por **não terem corpo** (sumário, itens vazios). Reproduzi a regra `temCorpo` sobre BH:

descartados	quantos	motivo
bloco com menos de 200 caracteres (realmente vazios)	22	falha em CORPO_MINIMO
bloco com 200 ou mais caracteres (contando o título)	107	em geral falha só em PARAGRAFO_MINIMO: nenhuma linha passa de 60 caracteres

As amostras que li dos 107 são **atos reais**: "Extrato do Termo de Execução Cultural", com Processo, Contratado(a), Objeto, Valor total: R\$100.000,00, Data assinatura. O PDF de BH tem colunas estreitas, então as linhas têm 30 a 55 caracteres. Resultado: **esses atos ficam fora de todos os totais** (e muitos têm CNPJ mascarado, 07.XXX.XXX/XXXX-58, que de qualquer forma o extrator não acharia).

Resposta honesta: "A regra foi desenhada para **nunca inventar atos** (por exemplo, a partir de um sumário). O custo é que, em PDFs de coluna estreita, atos reais ficam de fora, e isso aparece no funil (225 → 96). Um passo seguinte seria trocar 'uma linha com mais de 60 caracteres' por 'parágrafo com mais de N caracteres', juntando as linhas."

Atenção ao exemplo do README sobre o sumário de Aracaju (DECRETO NUMERADOS - 8512...): Aracaju é **recusado antes da fragmentação** (a limpeza o recusa), então esse exemplo **não** passa pela `Fragmentacao` no corpus atual; o efeito da regra do corpo é medido em BH.

3. Um valor e vários CNPJs: valor contado de novo

Testei um ato com CONTRATADA: ALFA... CNPJ 11.111.111/0001-11, CONTRATADA: BETA... CNPJ 22.222.222/0001-22 e um único Valor global: R\$ 100.000,00. Resultado: **2 contratos, total R\$ 200.000,00**. `PainelContratos.processar` liga cada CNPJ ao valor **mais próximo**, e com um só valor ele serve para os dois.

Resposta honesta: "É uma simplificação: cada CNPJ pega o valor mais próximo. Para atos com várias empresas e um valor global, o total pode estar inflado. O correto seria dividir o valor ou marcar o contrato como 'valor compartilhado'."

4. "R\$ 7,02 milhões" vira R\$ 7,02

`ExtratorValor` exige vírgula e 2 casas decimais e **não olha a palavra seguinte**. investimento total de R\$ 7,02 milhões → fato R\$ 7,02 → 7,02. (Testei também que R\$ 18 e R\$ 10,5 são ignorados.) Nos extratos, valores aparecem completos (R\$ 700.000,00); em texto de notícia, não.

5. O ramo `REJEITADO` nunca acontece

Veja o quadro no capítulo das etapas: `Fragmentacao` já exige corpo ≥ 200 e sempre define a espécie, então os dois primeiros testes do `Portao` são inalcançáveis; `REJEITADO` é **0** em todos os relatórios. É rede de segurança, e o README diz "três saídas" porque a estrutura existe; na prática o que se observa é **ACEITO** e **DUVIDOSO**.

6. Normas: ano descartado e autocitação

- `ExtratorReferencia.java:45-57` para na `/:` Portaria 073/2025 e Portaria 073/2026 viram o mesmo portaria:073.
- O **título do próprio ato** conta como referência: no grafo real há `decreto:19524 --CITA--> decreto:19524`. É ruído (`CITA` não altera nada), mas infla o número de relações (`CITA` = 190 no total dos 4 diários).

7. Nomes de empresa

A extração do nome usa heurística sobre texto livre. Na saída real aparecem `Item 01`, `Itens 01 a 04` e `licitante/lotado adjudicado`: `BEST CONE.`, e "(nao identificada)" em alguns. O **CNPJ** continua certo, e o ranking agrupa por CNPJ, então o dinheiro soma certo; o rótulo é que pode estar ruim.

8. Dados de página e CNPJ mascarado

- Datas de **rodapé de PDF** (24/03/2026 18:39:28) entram como fatos `DATA`, e inflam a contagem (524 datas em BH). `ExtratorData` também **não valida** a data (99/99/9999 seria aceita).
- `ExtratorCnpj` só aceita o formato **com pontuação e dígitos**; CNPJ mascarado (XXX) não é achado, e não confere dígitos verificadores.

9. Imperfeições pequenas (para não se surpreender)

- O resumo gera "**2 municípios publicou** 129 atos" (verbo no singular com sujeito plural).
- `Interpretacao.montarResumo` tem `atosDeContrato == 1 ? " deles ha " : " deles ha "` (os dois ramos são iguais), uma sobra sem efeito.
- `Interpretacao` e `PaginaHtml` **convertem de volta** o valor "`R$ 1.234,56`" em número (`valorDe`), porque `ranking()` devolve `String[]`. Funciona, mas é um desvio: quem guarda `double` e formata só na saída evita isso.
- `Municipio.getApellido` repete a tabela de acentos que já existe em `Extrator.semAcento`.
- `Ato.getFatos()` e outros getters devolvem a lista interna (não cópia).
- `Coletor.coletar` busca **só a primeira página** (`size=50`, sem `offset`): um município com mais de 50 edições no período teria o resto ignorado. E `baixarEdicao` **não tenta de novo** um download que falhou (só a consulta à API tem nova tentativa).
- O código **não tem comentários** (decisão do projeto): as razões estão no README e neste guia.

10. Documentação × código

onde	diz	o código faz
README.md	"roda os 75 testes", "75 testes sobre dados reais"	85 (LEIA-ME.txt, PREPARO_ARGUICA0.md já dizem 85)
README.md	" 1 segundo entre requisições"	PAUSA_MS = 1500 (1,5 s)
README.md	corpus de 2 diários (Aracaju e BH)	a pasta tem 4 (recife_2026-09-29.txt e ...-extra116.txt foram acrescentados)
README.md	contratações "19, somando R\$ 8.323.312,07"	verdadeiro só para BH ; nos 4 diários são 25 , R\$ 9.373.590,71
PREPARO_ARGUICA0.md	Exercício A "acha 4 CEPs reais " em BH	acha 4 fatos em BH, mas 2 são o mesmo telefone (98427-4362, falso positivo); os CEPs reais são 03333-050 e 31872-210 (veja Exercício A)

DICA — Se for citar números, cite o conjunto

"No diário de BH: 96 atos, 66 aceitos, 19 contratações." / "Nos 4 diários da pasta: 129 atos, 25 contratações, R\$ 9,37 milhões." Assim ninguém acha que você errou a conta.

10. Banco de perguntas e exercícios

Treine em voz alta: pergunta, resposta curta, onde apontar

Regra de ouro das respostas: duas frases, **o que faz** + **por que foi feito assim**, e **aponte a linha** enquanto fala. Se não souber, diga: *"essa parte foi além do que o senhor deu; eu entendo o que ela faz, mas não domino o detalhe"*. É muito melhor que inventar.

10.1 Visão geral

P	Resuma o projeto em 30 segundos.
R	Lê diários oficiais de municípios (arquivos de texto enormes), separa cada diário em atos (decretos, portarias, extratos), acha dentro de cada ato dados concretos (datas, valores, CNPJ, processos, normas citadas) e produz informação que não está escrita em nenhum documento: quanto cada município contratou e com quem, quais normas foram alteradas, o que cada ato faz em uma frase, e o que o programa não conseguiu ler. Em Java puro, sem biblioteca externa, e determinístico.
P	Quais são as etapas do pipeline? Onde elas estão ligadas?
R	Ingestão → Limpeza → Fragmentação → Extração → Portão → Análise. Estão ligadas em <code>Aplicacao.processarPasta()</code> (<code>Aplicacao.java:69-111</code>).
P	Por que existem <code>Aplicacao</code>, <code>Servidor</code> e <code>Principal</code>? Não bastava uma classe?
R	Para separar o trabalho (<code>Aplicacao</code>) das formas de mostrar (painel web e menu de terminal). As duas "portas" só chamam métodos da <code>Aplicacao</code> , e por isso os números são os mesmos nos dois.
P	Qual é a ideia mais importante do trabalho?
R	Que todo dado extraído guarda a posição onde foi achado (<code>Fato.inicio/fim</code>), então o programa mostra a prova em vez de pedir confiança. E que o programa recusa ou marca como duvidoso o que não consegue ler, em vez de fingir.
P	Por que não usou banco de dados? E <code>HashMap</code>?
R	Banco e <code>HashMap</code> não foram dados em aula; o estado fica em arquivos de texto, que qualquer pessoa pode abrir e conferir. Com dezenas ou centenas de itens a diferença de tempo entre <code>ArrayList</code> e <code>HashMap</code> não aparece.
P	Como você sabe que o programa acerta?
R	85 testes (incluindo um que roda o pipeline inteiro sobre diários reais), e a página de cada ato, que mostra cada fato com o trecho original. E o programa conta o que não sabe: os DUVIDOSO e os diários recusados aparecem no painel. Reconheço que a cobertura de testes não é total (por exemplo, <code>Fragmentacao</code> só é testada via pipeline).
P	Quais são as limitações do projeto?
R	Só funciona com diários que têm texto (PDF escaneado é recusado; precisaria de OCR); a regra de corpo da fragmentação descarta atos de coluna estreita; alguns nomes de empresa não são identificados; um valor com vários CNPJs é contado várias vezes; e uns 30% dos atos ficam em dúvida, e isso aparece no painel. (Detalhes no capítulo "Pontos de atenção".)

P	Por que um projeto de diários oficiais?
R	É dado público, real e bagunçado, que exige decisões de programação (limpar, separar, decidir o que é confiável), não só ler arquivo.

10.2 Orientação a objetos

P	Mostre onde há herança.
R	ExtratorCnpj extends Extrator (ExtratorCnpj.java:6); idem as outras quatro filhas. Também as exceções estendem Exception.
P	Mostre o polimorfismo.
R	Extracao.java:28: a lista tem cinco classes diferentes tratadas como Extrator, e e.extrair(...) executa a versão de cada uma. Para achar CEPs: uma classe nova e uma linha; nada mais muda.
P	O que é encapsulamento? Onde aparece?
R	Esconder os dados (private) e controlar o acesso por métodos. Em Fato: private final e só getters (Fato.java:11). Um fato não pode ser alterado depois de criado.
P	Para que servem equals e hashCode?
R	Para dizer quando dois objetos são "o mesmo" pelo conteúdo , e não pela referência. Relacao.equals compara origem, ação e alvo, e é usado em arestas.contains(nova) para não duplicar relações. hashCode tem de ser coerente com equals.
P	Diferença entre classe abstrata e concreta?
R	Abstrata não pode ser instanciada e pode ter métodos sem corpo; as filhas concretas são obrigadas a implementá-los. Extrator é abstrata; ExtratorCnpj é concreta.
P	O que significa static?
R	O membro pertence à classe e não a um objeto. Extrator.semAcento("São") é chamado direto na classe; Contrato.formatarReal(double) também.
P	Por que Ato tem getIdentidade() em vez de um id numérico?
R	A identidade vem do conteúdo (municipio:especie:numero:data), então é igual toda vez que se roda o programa. Um contador mudaria se a ordem mudasse, e o grafo e os contratos guardam essa identidade.

10.3 Exceções e arquivos

P	Mostre uma exceção própria e quem a lança e a captura.
R	DiarioIlegivelException. Lançada em Limpeza.limpar (Limpeza.java:43), capturada em Aplicacao.processarPasta (Aplicacao.java:82).
P	Por que uma exceção própria e não uma Exception qualquer?
R	Porque "diário ilegível" é uma situação do domínio : carrega motivo e evidência numérica, e o catch específico trata só isso, deixando outros erros passarem.

P	O que é try-with-resources e onde você usou?
R	Um try (recurso) { ... } que fecha o recurso sozinho , mesmo se der erro. Em Ingestao (DirectoryStream) e em Relatorio (BufferedWriter), e também em Servidor.responder (OutputStream).
P	Onde o programa lê e grava arquivos?
R	Lê em Ingestao (Files.readString, DirectoryStream), Fontes.carregar e Aplicacao.carregarRodadas (Files.readAllLines). Grava em Relatorio (BufferedWriter), Fontes.salvar (Files.write), Coletor.baixarEdicao (Files.writeString) e Aplicacao.registrarRodada (APPEND).
P	E se a pasta de diários não existir?
R	Ingestao.lerTodos captura a IOException, anota o motivo em falhas e devolve lista vazia. O programa não cai. (Na prática Aplicacao cria as pastas no construtor.)
P	E se o diário vier vazio ou for um PDF escaneado?
R	A Limpeza conta menos de 10 linhas longas e lança DiarioIlegivelException; o diário é registrado como RECUSADO com a evidência e o programa segue para o próximo.

10.4 Texto e extração

P	Como o casaMolde funciona?
R	9 é "qualquer dígito" e qualquer outro caractere tem de ser igual. Anda caractere por caractere (charAt) e devolve false na primeira diferença. Não uso expressão regular: só charAt, substring e laços.
P	Por que semAcento(texto.toLowerCase()) e as posições continuam valendo?
R	Porque as duas operações trocam caractere por caractere , então o texto transformado tem o mesmo tamanho; uma posição achada nele vale no original.
P	O que é um Fato e por que guarda a posição?
R	Um dado achado no texto (CNPJ, data, valor, processo, norma) mais o inicio e o fim . A posição permite provar de onde saiu (provar) e olhar o texto em volta.
P	Por que Extracao cria um novo Fato somando ato.getPosicao()?
R	Porque os extratores só enxergam o texto do ato e devolvem posições relativas a ele. Somando a posição do ato, o fato passa a apontar para o texto limpo do diário inteiro.
P	Como você acha datas por extenso?
R	Parto do nome do mês no texto sem acento e em minúsculas, volto para achar " de " e o dia, avanço para achar " de " e 4 dígitos do ano. Se tudo bate, o fato é o trecho original, "20 de março de 2026".

10.5 Fragmentação, portão e análise

P	Como a Fragmentação evita contar o sumário como atos?
R	temCorpo: o título só vira ato se depois dele houver pelo menos 200 caracteres e uma linha com mais de 60. Na dúvida, não corta.
P	Por que LEI COMPLEMENTAR vem antes de LEI na lista?
R	A busca para no primeiro que casa; se LEI viesse antes, "LEI COMPLEMENTAR" seria lida como LEI.
P	Por que o Portao tem três saídas?
R	Porque, com duas, o caso duvidoso é empurrado para um lado e some. Com três ele fica contável: é a lista do que o programa ainda não trata.
P	Como um ato chega a ACEITO?
R	Passando por todos os if (texto longo, espécie, pelo menos 2 fatos, número ou valor/CNPJ ou norma, pelo menos uma data). Nenhum caminho leva a aceite por omissão, e toda decisão grava o motivo.
P	Qual a complexidade do ranking() e por que isso não é problema?
R	$O(m^2)$ na ordenação por seleção, com m = número de fornecedores (dezenas). O tempo não aparece. Com milhares valeria outra estrutura.
P	Por que agrupar por CNPJ e não por nome da empresa?
R	O nome aparece escrito de várias formas e a extração do nome é imperfeita; o CNPJ é estável.
P	Como você sabe que um CNPJ é fornecedor e não a prefeitura multando?
R	Pelo papel escrito antes do CNPJ (NOTIFICANTE, NOTIFICADO, CONTRATADA...), numa janela de 320 caracteres: o marcador mais próximo decide. Isso nasceu de um erro real.
P	Explique o grafo de normas.
R	Cada citação de norma é uma aresta ato → norma com uma ação (ALTERA, REVOGA, PRORROGA ou CITA). classificar decide a ação olhando o texto em volta, em ordem: o que vem depois da citação, fundamento legal, verbo antes; na dúvida, CITA. vizinhanca percorre o grafo por níveis (busca em largura).
P	"Nenhuma interpretação usa IA": como se geram as frases?
R	Cada achado é um método com uma regra sobre números medidos (por exemplo, "os 3 maiores fornecedores somam $\geq 60\%$ "). O texto fixo é escrito no código; quais achados aparecem e com que números sai das regras. Mesma pasta, mesmas frases.

10.6 Fora da aula

P	O que é uma Thread e por que a usou?
R	Uma linha de execução paralela. A coleta demora (várias requisições com pausa), então roda numa thread separada para a página do painel não travar. (<code>Aplicacao.coletarEmSegundoPlano.</code>)
P	O que faz synchronized? E volatile?
R	<code>synchronized</code> deixa uma thread de cada vez entrar no trecho (evita que duas coletas comecem juntas e que se leia um estado pela metade). <code>volatile</code> garante que a outra thread veja o valor atualizado de ocupado/progresso. Entendo o motivo; não domino o detalhe.
P	O que é () -> e this::tratarPainel?
R	Lambda: um bloco de código passado como valor (aqui, o corpo da thread). Referência de método: indica qual método chamar para cada endereço do servidor. Ambos substituem escrever uma classe que implementa uma interface de um só método.
P	O que é HttpServer? Por que não usou Spring?
R	É um servidor web mínimo que já vem no JDK: recebe o pedido, chama um método nosso, devolve HTML. Spring seria uma biblioteca externa; o trabalho usa só o JDK.
P	Por que Locale.R00T em formatarReal?
R	No Windows em português, <code>String.format("%.2f")</code> usa vírgula decimal, e o método procura o ponto. Foi um erro real que só aparecia lá; o teste força o locale pt-BR para reproduzir.
P	Por que o painel não tem JavaScript?
R	Para ficar só com o que foi dado: cada botão é um formulário HTML; o Java recebe o clique, faz a ação e redireciona (303) para a página de volta. Enquanto há coleta, <code><meta http-equiv="refresh" content="3"></code> atualiza a página sozinha.

10.7 Perguntas de "mostre no código"

se o professor disser...	abra...
"Mostre a herança."	<code>Extrator.java</code> e <code>ExtratorCnpj.java</code>
"Mostre o polimorfismo."	<code>Extracao.java</code> (o laço sobre extratores)
"Mostre o tratamento de exceção."	<code>Limpeza.java</code> (lança) → <code>Aplicacao.processarPasta</code> (captura)
"Onde você usa arquivos?"	<code>Ingestao</code> (lê) e <code>Relatorio</code> (grava)
"Onde você usa ArrayList?"	<code>Ato.fatos</code> , <code>Diario.atos</code> , <code>Extracao.extratores...</code>
"Mostre que não inventou o dado."	menu 11, ou a página /ato no painel (<code>Fato.provar</code>)
"Mostre uma ordenação."	<code>PainelContratos.ranking</code> (seleção) ou <code>PaginaDiario.ordenar</code> (inserção)
"Mostre StringBuilder."	<code>Contrato.formatarReal</code> , <code>Portao.avaliarAto</code>
"Mostre Scanner/switch."	<code>Principal.menu</code>

10.8 Exercícios: "altere o código aqui na minha frente"

Muito professor pede isto. Treine os quatro até fazer sem olhar. **Todos foram testados** numa cópia do projeto; os resultados abaixo são medidos.

Exercício A — achar um tipo novo de dado (CEP)

1. Em `Fato.java`: `public static final String CEP = "CEP";` (junto das outras constantes).
2. Crie `extratores/ExtratorCep.java`:

```
package diarios.extratores;

import diarios.modelo.Fato;
import java.util.ArrayList;

public class ExtratorCep extends Extrator {

    private static final String MOLDE = "99999-999";

    public ExtratorCep() {
        super(Fato.CEP);
    }

    @Override
    public ArrayList<Fato> extrair(String texto) {
        ArrayList<Fato> achados = new ArrayList<>();
        for (int i = 0; i + MOLDE.length() <= texto.length(); i++) {
            if (casaMolde(texto, i, MOLDE)) {
                achados.add(criar(texto.substring(i, i + MOLDE.length()),
                                   i, i + MOLDE.length()));
                i += MOLDE.length() - 1;
            }
        }
        return achados;
    }
}
```

3. Em `Extracao.java`, no construtor: `extratores.add(new ExtratorCep());` e o `import diarios.extratores.ExtratorCep;`.

Resultado medido: compila, os **85 testes continuam passando** e o programa acha **5 fatos CEP** nos diários legíveis: 03333-050 e 31872-210 (BH), 50030-903 (Recife) e **dois falsos positivos**, ambos o mesmo telefone (31) 98427-4362 (o molde pega 98427-436). Se o professor notar, diga: *é uma limitação do molde; o passo seguinte seria checar o que vem antes e depois (não ser precedido nem seguido de dígito)*. **O que isso prova:** polimorfismo; só acrescentou uma classe e uma linha.

Exercício B — deixar o portão mais rígido

Em `Portao.java`, troque `FATOS_MINIMOS = 2` por `3` (`Portao.java:10`). **Resultado medido** em BH: de **66 aceitos / 30 duvidosos** para **58 / 38**. Mostre no painel ("Reler a pasta") depois de recompilar. **O que isso prova:** o limite é um **parâmetro**, e o número de "duvidosos" responde a ele.

Exercício C — reconhecer outro tipo de ato

Em `Fragmentacao.java`, acrescente `"CONVENIO"` à lista `ESPECIES` (`Fragmentacao.java:10`). **Atenção à ordem:** nomes mais longos antes dos curtos. **Resultado medido:** `CONVENIO Nº 10/2026` **vira ato**; mas `CONVÊNIO Nº 11/2026`, **com acento, não**, por causa do defeito do capítulo "Pontos de atenção". **Na demonstração, use texto sem acento**, ou explique o defeito.

Exercício D — mudar um limite da limpeza

Em `Limpeza.java`, troque `MINIMO_DE_LINHAS_LONGAS = 10` por `2000` (`Limpeza.java:9`). **Resultado medido:** **todos** os diários passam a ser recusados (BH tem 1.018 linhas de corpo; Recife 918 e 74; Aracaju 1), cada um com a evidência "...; o mínimo esperado é 2000". **O que isso prova:** a recusa depende de um parâmetro explícito, e a evidência numérica acompanha a decisão. (Volte para 10 depois.)

DICA — Antes de mexer no código na frente do professor

Faça uma **cópia** da pasta, ou use o Git para desfazer (`git checkout -- arquivo`). E lembre: o painel só reflete a mudança depois de **recompilar e reiniciar**; no LEIA-ME.txt está dito que o Java portátil de `_sistema` não tem compilador. Para editar, use o IntelliJ.

10.9 Autoteste de 10 minutos (em voz alta)

1. Explique `Aplicacao.processarPasta()`, em ordem.
2. O que é um `Fato` e por que guarda a posição?
3. Mostre herança e polimorfismo no projeto.
4. O que a `Limpeza` faz e quando lança exceção?
5. Por que o `Portao` tem três saídas? Por que `REJEITADO` é sempre 0?
6. Como a `Fragmentacao` evita contar o sumário? Qual o custo disso?
7. Por que `ranking()` é $O(n^2)$ e por que isso não é problema aqui?
8. Faça o Exercício A sem olhar.
9. O que `synchronized` faz e onde aparece?
10. O que acontece se a API estiver fora do ar?
11. Qual defeito você conhece na `Fragmentacao`, e como corrigiria?

Se errou mais de 3, releia os capítulos 3 e 4 deste guia e depure `processarPasta()` no IntelliJ (veja o plano de estudo).

11. Plano de estudo e roteiro de apresentação

Segunda, terça e a manhã de quarta

Hoje é **segunda-feira, 05/10**; a arguição é **quarta, 07/10**. Há tempo para dominar o essencial **sem decorar**: entenda e ensaie dizendo em voz alta. Os tempos abaixo são realistas para quem abre o código no IntelliJ ao lado do guia.

Plano em três blocos

quando	bloco	o que fazer	tempo
Segunda	1	"Resumo em uma página" + capítulo 1 (visão geral, seção 1.3 com o exemplo de BH)	20 min
	2	Capítulo 2: Fato, Ato, Diario (abra os três no IntelliJ)	25 min
	3	Capítulo 3: Extrator, ExtratorCnpj, Extracao. Faça o Exercício A sem olhar	35 min
	4	Capítulo 4: processarPasta(), Ingestao, Limpeza, Portao. Depure (veja abaixo)	50 min
	5	Perguntas 10.1 e 10.2 em voz alta	20 min
Terça	6	Fragmentacao (temCorpo, especieDaLinha) + "Pontos de atenção" 1 e 2	40 min
	7	Capítulo 5: PainelContratos, GrafoNormas.classificar, LeituraDoAto, Interpretacao	45 min
	8	Capítulos 6 e 7: Coletor (nova tentativa), Aplicacao (thread), Servidor, Testes	35 min
	9	Exercícios B, C e D; decidir o que fazer com o defeito	20 min
	10	Autoteste (10.9) e perguntas 10.3 a 10.6	25 min
Quarta (manhã)	11	Releia "Resumo em uma página" e "Pontos de atenção"	15 min
	12	Rode o programa e faça a demonstração (veja abaixo)	15 min
	13	Reveja só as perguntas que você errou	15 min

A IDEIA CENTRAL — Se sobrar pouco tempo

Domine a **herança e o polimorfismo** (capítulo 3) e o **pipeline** (processarPasta, capítulo 4). Com essas duas coisas você responde a maioria das perguntas. Depois, o capítulo 9 ("Pontos de atenção"): é o que mais diferencia quem leu o código de quem só leu o README.

Como depurar no IntelliJ (o melhor jeito de aprender)

1. Abra `Aplicacao.java` e vá a `processarPasta()`.
2. Clique na **margem esquerda** ao lado de `limpeza.limpar(d)`; aparece uma bolinha vermelha (*breakpoint*).
3. Rode `Servidor.java` com o botão do **inseto** (Debug), não o ►. Se não quiser baixar nada da internet, edite a configuração e use o argumento `--sem-coleta`.
4. O programa **para** na linha. Na aba **Variables**, abra `d` e veja o `textoBruto` do diário.
5. **F8** (Step Over) avança uma linha; **F7** (Step Into) entra no método.
6. Repita com `fragmentar(d)` e `extracao.extrair(d)`. Observe `d.getAtos()`: primeiro vazio, depois com 96 atos (em BH), depois cada ato com seus fatos.

Atalhos que ajudam: Ctrl + clique num nome (vai à definição), Alt + ← (volta), Ctrl + Shift + F (busca no projeto inteiro).

Roteiro de demonstração (3 minutos)

- 1. Suba o painel:** `Servidor.java` com `--sem-coleta` (evita rede). Abra `http://localhost:8080`.
- 2. Mostre o painel:** os quatro números do topo, "O que esses dados dizem", o "funil" (limpeza → fragmentação → portão) e **os diários recusados** (Aracaju).
- 3. Abra o diário de BH** (link na tabela "Diários lidos") e clique no ato **"Extrato do Termo de Patrocínio"**.
- 4. Mostre a página do ato:** "O que este ato faz", a decisão com o **motivo**, e **cada fato com a prova**: *o trecho recortado do texto original*. Mostre o texto do ato **com os fatos marcados** em amarelo.
- 5. Fale em uma frase:** "Cada fato guarda `inicio` e `fim`; esta prova é `Fato.provar`; por isso o programa não inventa nada."
- 6. Plano B (sem navegador):** `Principal`, opção **11**, ato **21**.

Quem defende o quê (sugestão para 4 pessoas)

pessoa	pacote	o que dominar
1	<code>modelo/</code> + <code>extratores/</code>	Fato, Ato, Extrator e as filhas (herança e polimorfismo)
2	<code>etapas/</code>	Limpeza, Fragmentacao, Portao, Ingestao, Relatorio
3	<code>analise/</code>	<code>PainelContratos</code> , <code>GrafoNormas</code> , <code>Interpretacao</code> , <code>LeituraDoAto</code>
4	<code>Aplicacao + coleta/</code> + <code>servidor/</code>	o fluxo e a parte "fora da aula"

Todos precisam saber explicar `Aplicacao.processarPasta()`.

Glossário

Termos do domínio e de Java usados neste projeto

termo	significado
Diário oficial	publicação diária (em PDF) em que o município divulga seus atos.
Ato	cada decisão publicada: decreto, portaria, extrato de contrato, edital, aviso... É a unidade de trabalho do programa (Ato).
Espécie	o tipo do ato (DECRETO, PORTARIA, EXTRATO...), vem do início do título.
Norma	lei, decreto, portaria, resolução: um texto que cria regras. O grafo mostra quais atos mexem em quais normas.
Fundamento legal	a norma citada só para justificar o ato ("nos termos do Decreto X"); não a altera. No programa: CITA.
Extrato (de contrato)	resumo publicado de um contrato: partes, objeto, valor, prazo.
Licitação	procedimento pelo qual o poder público escolhe quem vai fornecer (pregão, concorrência, dispensa...).
Ata de registro de preços	documento que registra preços de fornecedores para compras futuras; pode ter várias detentoras.
CNPJ	identificador da empresa (NN.NNN.NNN/NNNN-NN). CPF : da pessoa (NNN.NNN.NNN-NN); é mascarado na tela.
Número de processo	identificador do processo administrativo (31.00204095/2026-45); liga o contrato à licitação.
Querido Diário	projeto aberto (Open Knowledge Brasil) que converte diários em texto e tem API pública.
PDF escaneado	PDF que é imagem, sem texto por baixo; só um OCR conseguiria lê-lo. O programa o recusa .
OCR	reconhecimento óptico de caracteres: transformar imagem em texto (não implementado aqui).
Pipeline	sequência de etapas em que a saída de uma é a entrada da próxima.
Funil	a sequência de números que mostra quanto "sobra" em cada etapa (16.338 linhas → 96 atos).
Portão	a etapa que dá a cada ato ACEITO, DUVIDOSO ou REJEITADO, com o motivo.
Fato	dado achado no texto, com tipo, valor e posição (início, fim).
Span	o intervalo [início..fim] de um fato no texto.
Molde	padrão em que 9 significa "um dígito" e o resto é literal (99.999.999/9999-99).
Identidade (do ato)	texto municipio:especie:numero:data que identifica o ato de forma estável.
Grafo / aresta	conjunto de vértices (atos e normas) e de ligações (Relacao) entre eles.
Busca em largura	percurso de um grafo por níveis: primeiro os vizinhos, depois os vizinhos dos vizinhos.
Determinístico	mesma entrada, mesma saída; nada aleatório nem de IA.
Encapsulamento	esconder os dados (private) e dar acesso por métodos.
Polimorfismo	chamar o mesmo método em objetos de classes diferentes e cada um responder à sua maneira.
Classe abstrata	classe que não pode ser instanciada e define um contrato para as filhas.
Exceção verificada	extends Exception: o compilador obriga a tratar ou declarar com throws.
Thread	linha de execução paralela; lambda é um bloco de código passado como valor; synchronized deixa uma thread por vez; volatile garante visibilidade do valor atual.

termo	significado
Maven (<code>pom.xml</code>)	ferramenta de build; aqui só descreve o projeto (Java 17, sem dependências, classe principal).
HttpClient / HttpServer	cliente HTTP e servidor web que já vêm no JDK (fora do conteúdo de aula).